A HYBRID SDLC AGENT: INTEGRATING LOCAL SMALL LANGUAGE MODELS WITH CLOUD-BASED LLM FALLBACK FOR SOFTWARE ENGINEERING TASKS

AJAY RAMESH

Final Thesis Report

MARCH 2026

Liverpool John Moores University

Submitted in partial fulfilment of the requirements for the degree of

Master of Science

DEDICATION

This thesis is dedicated to my family, whose unwavering support and patience throughout this research journey have been my greatest source of strength.

To my parents, whose encouragement for higher studies has always been my greatest motivation.

To all those who supported me through the many challenges of balancing professional responsibilities with academic pursuits — this achievement is as much yours as it is mine.

ACKNOWLEDGEMENTS

I would like to express my deepest gratitude to my research supervisor for their continuous guidance, valuable feedback, and constructive insights throughout this thesis journey. Their suggestions on structure, content, and analytical rigour have greatly shaped this work.

My sincere thanks to upGrad for organising informative sessions on thesis writing and submission, and to Liverpool John Moores University for its supportive academic environment and flexible policies that made it possible to complete this research alongside professional commitments.

I am grateful to colleagues and the academic community whose published work in the fields of language model evaluation, software engineering automation, and hybrid AI systems provided the theoretical foundation upon which this research was built.

A special acknowledgement to the open-source communities behind the benchmark datasets — QuixBugs, Defects4J, HumanEval, and BigCodeBench —and the developers of the llama.cpp runtime and Hugging Face PEFT library, whose contributions made this experimental research possible.

Finally, my heartfelt thanks to my family for their patience, encouragement, and understanding. They stood by me as I dedicated much of my time to this research, and their support made this achievement possible.

ABSTRACT

Artificial intelligence tools for software development have advanced rapidly, yet most existing systems rely exclusively on cloud-hosted Large Language Models (LLMs). This introduces practical barriers of inference cost, latency, and data privacy that prevent many organisations, particularly those handling sensitive or proprietary code — from adopting AI-assisted development at scale.

This study designed and evaluated a hybrid SDLC agent combining a locally deployed Small Language Model (SLM) with a cloud-based LLM fallback. The system comprised two specialised agents, a Coder Agent for code generation and a Patch-Fix Agent for bug repai, governed by an attempt-based routing strategy that escalates tasks to the cloud only when local execution is exhausted. Retrieval-Augmented Generation (RAG) and parameter-efficient LoRA fine-tuning were integrated as self-improvement mechanisms.

Experiments across four benchmark datasets, QuixBugs, Defects4J, HumanEval, and BigCodeBench — under CPU-only hardware constraints confirmed that the local SLM achieved 52.4% pass@1 on HumanEval, exceeding the published ideal-condition benchmark of 43.8%, while selective cloud fallback raised the BigCodeBench pass rate from 20.0% to 60.0%. The LoRA adapter recovered tasks the base model could not solve, confirming the pipeline's capacity for genuine capability extension through operational deployment.

These findings confirm that hybrid SLM-LLM orchestration is a viable and cost-effective approach to AI-assisted software engineering under realistic deployment constraints. The architecture provides a practical blueprint for organisations in regulated industries to adopt AI coding assistance while keeping proprietary code on-premises. Future work should incorporate a learned routing classifier and scale the distillation dataset to establish the improvement rate quantitatively.

LIST OF FIGURES

Figure 3.1: Hybrid SDLC Agent — High-Level Workflow Architecture 40
Figure 4.1: Hybrid SDLC Orchestration Pipeline — End-to-End Architecture 45
Figure 4.2: Experimental Run Directory Structure 67
Figure 4.3: LoRA Adapter Training Pipeline 66
Figure 4.4: RAG Pipeline — FAISS Index Construction and Retrieval 62
Figure 5.1: HumanEval Failure Taxonomy — Corrected from Run Artefacts 82
Figure 5.2: SLM Generation Timing Distribution — QuixBugs (n=52) 88
Figure 5.3: LoRA Training Loss Convergence across Three Epochs 92
Figure 5.4: Hybrid Architecture Performance — Local SLM vs Hybrid System Across All Datasets 93
Figure 5.5: Refined Routing Decision Framework — Task-Type and Failure-Mode Aware 95

LIST OF TABLES

Table 3.1: Research Design Summary — Research Onion Framework 28
Table 3.2: Dataset Summary — Benchmarks Selected 33
Table 3.3: Model Selection Summary 35
Table 3.4: Validity and Reliability — Risks and Mitigations 42
Table 4.1: Experimental Software Environment 47
Table 4.2: QuixBugs Dataset Preparation 50
Table 4.3: Routing Strategy Evolution Summary 58
Table 4.4: RAG Pipeline Configuration 61
Table 4.5: LoRA Training Hyperparameters and Justification 64
Table 5.1: QuixBugs Results Summary 77
Table 5.2: Defects4J Results Summary 77
Table 5.3: HumanEval Results Summary 78
Table 5.4: BigCodeBench Results Summary 79
Table 5.5: HumanEval Results — Verified from Run Artefacts 80
Table 5.5a: Confirmed Attempt-2 Retry Recoveries — HumanEval 81
Table 5.6: HumanEval Failure Taxonomy — Corrected 81
Table 5.7: SLM Routing Behaviour Summary 83
Table 5.8: SLM Generation Timing Statistics 86
Table 5.9: RAG Pipeline Statistics 88
Table 5.10: LoRA Training Dataset Split 90
Table 5.11: LoRA Validation Results 91
Table 5.12: Cross-Dataset Performance Summary 93

Table 6.1: Research Questions and Findings Summary 104

LIST OF ABBREVIATIONS

AI……Artificial Intelligence

API…… Application Programming Interface

AST……Abstract Syntax Tree

CPU…… Central Processing Unit

DSR…… Design Science Research

FAISS…… Facebook AI Similarity Search

GGUF…… GGML Unified Format

GPU…… Graphics Processing Unit

LLM……Large Language Model

LoRA…… Low-Rank Adaptation

PEFT…… Parameter-Efficient Fine-Tuning

RAG…… Retrieval-Augmented Generation

RAM…… Random Access Memory

SDLC…… Software Development Life Cycle

SLM……Small Language Model

SOTA…… State of the Art

TABLE OF CONTENTS

DEDICATION i
ACKNOWLEDGEMENTS ii
ABSTRACT iii
LIST OF FIGURES iv
LIST OF TABLES v
LIST OF ABBREVIATIONS vi
CHAPTER 1: INTRODUCTION 9
1.1 Background of the Study 9
1.2 Problem Statement 10
1.3 Aim of the Study 10
1.4 Research Objectives 11
1.5 Research Questions 11
1.6 Scope of the Study 12
1.7 Significance of the Study 12
1.8 Structure of the Study 12
CHAPTER 2: LITERATURE REVIEW 14
2.1 AI Systems for Software Engineering Tasks 14
2.2 Small Language Models and On-Device Reasoning 16
2.3 Retrieval and Artefact-Based Methods 18
2.4 Hybrid Routing and Model Selection 19
2.5 Compliance, Governance and Safety 20
2.6 Multi-Agent and Role-Based Systems 21
2.7 Knowledge Distillation and Teacher-Student Learning 21
2.8 Theoretical Framework 22
2.9 Evaluation Benchmarks and Limitations 23
2.10 Hardware-Constrained AI Deployment 25
2.11 Summary of Key Gaps 26
2.12 How Gaps Motivate This Research 26
CHAPTER 3: RESEARCH METHODOLOGY 27
3.1 Introduction 27
3.2 Research Philosophy and Design 28
3.3 Dataset Selection and Task Preparation 32
3.4 Model Selection 34
3.5 Role-Based Agent Workflow 36
3.6 Routing Strategy 36
3.7 Incremental Improvement Pipeline 38
3.8 Hybrid SDLC Workflow Architecture 39

3.9 Ethical Considerations 40
3.10 Validity and Reliability 41
3.11 Chapter Summary 43
CHAPTER 4: SYSTEM IMPLEMENTATION AND EXPERIMENTAL SETUP 44
4.1 Introduction 44
4.2 Experimental Environment 45
4.3 Benchmark Dataset Preparation in Practice 47
4.4 Hybrid Agent Implementation 51
4.5 Local Improvement Pipeline 58
4.6 Experimental Run Evidence 66
4.7 The Evolving Experimental Set: Run Traces 67
4.8 Summary 74
CHAPTER 5: RESULTS AND DISCUSSION 76
5.1 Introduction 76
5.2 Experimental Dataset Overview 77
5.3 Program Repair Results 79
5.4 Code Generation Results 85
5.5 Hybrid Routing Behaviour 88
5.6 Local Model Improvement Experiments 92
5.7 Cross-Dataset Observations 93
5.8 Contextualisation Against Published Benchmarks 94
5.9 Discussion 98
5.10 Chapter Summary 98
CHAPTER 6: CONCLUSIONS AND FUTURE RECOMMENDATIONS 100
6.1 Introduction 100
6.2 Summary of Findings 100
6.3 Conclusions Against Research Questions 104
6.4 Contributions to Knowledge 106
6.5 Limitations of the Study 108
6.6 Future Recommendations 109
6.7 Reflection on the Research Process 111
6.8 Chapter Summary 112
REFERENCES 114
APPENDIX A: PROMPT TEMPLATES 118
APPENDIX B: RUN DIRECTORY SCHEMA 118
APPENDIX C: LORA TRAINING CONFIGURATION 119
APPENDIX D: SOURCE CODE 120
APPENDIX E: RESEARCH PROPOSAL 121

CHAPTER 1

INTRODUCTION

1.1 Background of the Study

Software development has increasingly incorporated artificial intelligence to support tasks such as code generation, bug detection, and automated testing. Over the past decade, the proliferation of machine learning techniques fundamentally changed how developers wrote, reviewed, and maintained code. From early rule-based linters to modern deep learning-powered assistants, the trajectory of AI in software engineering reflects a steady progression toward more intelligent and autonomous support systems.

Recent advances in Large Language Models (LLMs) significantly improved the ability of AI systems to generate and modify source code from natural language descriptions. Models trained on vast corpora of code repositories, documentation, and programming discussions demonstrated strong performance in isolated tasks such as function generation, small-scale bug fixing, code summarisation, and unit test generation. Commercial tools built on these models — including GitHub Copilot and Amazon CodeWhisperer — have been adopted widely in professional development environments.

However, real-world software development extends well beyond isolated, single-task interactions. Modern software engineering involves a sequence of tightly interconnected activities across the Software Development Life Cycle (SDLC), including requirements analysis, architectural planning, implementation, testing, debugging, and iterative refinement. Existing AI-based systems were typically designed to address only specific stages of this process rather than supporting an integrated, end-to-end workflow. This fragmentation limits their practical utility in enterprise development contexts where tasks are rarely independent of one another.

A further limitation of current AI-assisted development tools is that most rely entirely on cloud-hosted LLMs. While these models offer strong reasoning capabilities and broad knowledge, they introduce meaningful practical challenges. Cloud inference

incurs latency, operational costs scale with usage, and organisations working with proprietary codebases face serious privacy and compliance concerns when routing code through external APIs.

At the same time, smaller models that can be deployed locally — commonly referred to as Small Language Models (SLMs) — have emerged as a compelling alternative. Models such as Phi-2 (Microsoft Research, 2023), TinyLlama (Zhang et al., 2024), and Qwen2.5-Coder (Hui et al., 2024) can be run on commodity hardware without any network dependency. These models offer clear advantages in terms of cost, response speed, and data privacy. However, they consistently lag behind their larger counterparts on tasks requiring multi-step reasoning, contextual understanding across large files, and complex algorithmic problem solving.

These competing trade-offs point to a hybrid design: local SLMs handle the majority of tasks, with cloud escalation reserved for cases that genuinely exceed local capability.

1.2 Problem Statement

Despite recent progress in AI-assisted software development, existing solutions exhibit several interconnected limitations that restrict their applicability in real-world development environments.

Four limitations define the gap this research addresses. Most existing AI coding systems target isolated tasks — generation or repair — rather than an end-to-end SDLC workflow, leaving developers to manually integrate outputs across stages. Cloud-hosted LLMs introduce cost, latency, and data privacy concerns that make them unsuitable for organisations handling proprietary or regulated code. Smaller locally deployable models avoid these concerns but consistently fail on tasks requiring multi-step reasoning or broad contextual understanding. No published system has combined role-based task decomposition with dynamic routing between local and cloud models within a unified workflow.

1.3 Aim of the Study

This research designed and evaluated a hybrid SDLC agent integrating a locally deployed Small Language Model with a cloud-based Large Language Model fallback,

targeting software engineering tasks including code generation and automated bug repair under realistic hardware and data privacy constraints.

1.4 Research Objectives

To achieve this aim, the study pursued the following five objectives:

(i) To design a hybrid architecture combining a local SLM with a cloud-based fallback LLM for software engineering tasks.

(ii) To develop a role-based workflow supporting both code generation and bug repair activities within a single orchestrated system.

(iii) To implement an attempt-based routing strategy that determines when tasks should be handled locally and when escalation to the cloud model is required.

(iv) To evaluate the system using publicly available benchmark datasets including QuixBugs, Defects4J, HumanEval, and BigCodeBench.

(v) To investigate and prototype mechanisms for improving the local SLM over time through retrieval-augmented knowledge reuse and parameter-efficient fine-tuning.

1.5 Research Questions

Four research questions guided the investigation:

RQ1: Can a hybrid SDLC architecture combining a Small Language Model and a Large Language Model effectively support software engineering tasks such as code generation and program repair?

RQ2: How well can a locally deployed Small Language Model perform software engineering tasks under constrained hardware conditions compared to a cloud-based LLM?

RQ3: How does the attempt-based hybrid routing strategy influence overall task success rates and system behaviour across different task types?

RQ4: Can retrieval-based knowledge reuse and parameter-efficient fine-tuning improve the capability of Qwen2.5-Coder 1.5B over time using solutions generated by the stronger cloud model?

1.6 Scope of the Study

The scope of this work centred on evaluating a hybrid SDLC workflow for two categories of software engineering tasks: code generation and automated program repair. The study was conducted under constrained local hardware conditions — specifically a CPU-only workstation with 8 GB RAM — reflecting a realistic deployment scenario for privacy-aware or resource-limited organisations. The study does not aim to develop a new language model from scratch, nor does it seek to compare all available SLMs or LLMs exhaustively.

1.7 Significance of the Study

The study makes three contributions to the field of AI-assisted software engineering. First, it contributes a hybrid orchestration architecture that combines local and cloud-based models within a structured workflow — a practical blueprint for cost-efficient, privacy-aware development tooling. Second, the attempt-based routing mechanism provides a training-free escalation strategy that adapts to observable execution outcomes rather than pre-computed surface features. Third, the incremental improvement pipeline contributes to the literature on continuous improvement of deployed language models through operational feedback loops.

In banking, healthcare, and public administration, keeping codebases within organisational boundaries is not a preference — it is a compliance requirement. The hybrid approach here offers those organisations a practical path to AI-assisted development without routing proprietary code to external servers.

1.8 Structure of the Study

The thesis opens with the research context and objectives in Chapter 1.

Chapter 2 reviews the relevant literature across AI-assisted software engineering, small language models, hybrid routing, and knowledge distillation, identifying seven gaps that motivated the design.

The methodological foundations — philosophical stance, dataset and model selection, routing design, and ethical considerations — are set out in Chapter 3.

Chapter 4 documents the system build and experimental execution, including the routing strategy evolution and improvement pipeline.

Results across all four benchmark datasets, routing behaviour, and improvement experiments are reported and analysed in Chapter 5.

Chapter 6 closes the study with conclusions against each research question, contributions to knowledge, and recommendations for future work.

CHAPTER 2

LITERATURE REVIEW

The literature informing this research spans AI-assisted software engineering, small language models, retrieval-augmented generation, hybrid routing, multi-agent systems, and knowledge distillation.

Each section critically examines contributions, methodological limitations, and research gaps in the relevant literature.

The literature was identified through systematic searches of Google Scholar, ACM Digital Library, IEEE Xplore, arXiv, and Semantic Scholar, using search terms including 'small language models code generation', 'hybrid model routing', 'automated program repair', 'retrieval-augmented generation code', 'multi-agent software engineering', and 'on-device AI inference'. Priority was given to peer-reviewed publications and conference proceedings from venues including ICML, NeurIPS, ICLR, and EMNLP, with particular emphasis on work published from 2020 onwards to ensure recency. Foundational works predating this period are included on the basis of their continuing theoretical relevance. Initial searches returned approximately 90 candidate papers across all sources; following title and abstract screening for relevance to the core themes of hybrid AI systems, small language models, routing strategies, and code generation evaluation, 47 papers were included as primary sources in the review.

2.1 AI Systems for Software Engineering Tasks

AI-assisted software engineering has moved quickly — from narrow rule-based linters and pattern-matching tools in the early 2000s to large neural systems capable of multi-step reasoning over entire codebases within the last five years.

Early approaches to AI-assisted programming relied on template matching, symbolic analysis, and static code analysis tools. These rule-based systems offered determinism and interpretability but required exhaustive manual specification of repair patterns and could not generalise to novel error categories. The emergence of deep learning provided an alternative paradigm.

DeepFix (Gupta et al., 2017) showed that sequence-to-sequence neural networks could produce syntactically valid repairs for compiler errors by learning patterns from large datasets of buggy and corrected code. DrRepair (Yasunaga and Liang, 2020) extended this using a graph-based architecture that incorporated compiler diagnostic messages as structured context, improving performance on semantic as well as syntactic errors. Despite these advances, both systems remained narrowly scoped and neither addressed the broader challenge of integrating repair into a multi-stage software development workflow.

Codex (Chen et al., 2021), trained on publicly available GitHub repositories, showed that a single model pre-trained on code could perform function-level synthesis, docstring generation, and simple bug fixing from natural language prompts — a qualitative shift from pattern-matching repair tools to general-purpose code generation.

Subsequent models including CodeT5+ (Wang et al., 2023) advanced code understanding further, yet each was evaluated in isolation — no system integrated generation and repair into a unified workflow. That gap is what this research addresses.

AlphaCode (Li et al., 2022) extended the paradigm to competitive programming, generating large candidate samples and filtering by test execution to achieve near-human performance on Codeforces benchmarks. While technically impressive, AlphaCode required massive cloud infrastructure and generated hundreds of candidates per task — wholly incompatible with the resource-constrained, privacy-aware deployment context motivating this research.

SWE-Agent (Yang et al., 2024) represents the current state of the art in autonomous software engineering. By equipping a large language model with structured file-system navigation, code editing, and test execution capabilities, SWE-Agent achieved resolution of real GitHub issues from the SWE-Bench dataset (Jimenez et al., 2023). SWE-Bench revealed a critical empirical finding: even powerful cloud LLMs struggle substantially with multi-file reasoning tasks, achieving below 20% resolution rates across many project categories. This has direct relevance to the

present study's Defects4J evaluation. Critically, both SWE-Agent and SWE-Bench assume cloud-hosted models and offer no mechanism for local, private deployment.

Iterative refinement frameworks including ReAct (Yao et al., 2023), Reflexion (Shinn et al., 2023), and Self-Refine (Madaan et al., 2023) showed that models can improve outputs through structured feedback loops. However, all three frameworks were designed and evaluated exclusively with large cloud-hosted models. No prior study has examined whether these strategies generalise to sub-2B parameter models executing under CPU-only constraints — a gap this research directly addresses. A comprehensive systematic literature review by Hou et al. (2024) covering 395 primary studies confirmed that evaluation of privacy-preserving, locally deployable systems under constrained hardware remains significantly underrepresented.

Table 2.1: Comparison of AI-Assisted Software Engineering Systems

System	Task Type	Approach	Limitation
DrRepair	Semantic Bug Repair	Graph-neural + compiler msgs	Narrow scope; no generation
AlphaCode	Competitive Programming	Large-scale sampling + filtering	Massive compute; no deployment constraints
SWE-Agent	Program Repair	LLM + file nav + test execution	Cloud-only; no privacy/SLM support
CodeT5+	Code Understanding	Multi-task instruction tuning	Isolated tasks; no orchestration
RepoCoder	Multi-file Repair	Retrieval + iterative generation	Static knowledge base; no routing
This Study	Code Gen + Bug Repair	Hybrid SLM-LLM role-based SDLC	Constrained CPU hardware; small fine-tuning set

Table 2.1 summarises the key systems reviewed above.

2.2 Small Language Models and On-Device Reasoning

Compact, instruction-tuned language models that run on commodity hardware without GPU acceleration have changed what is feasible for private deployment. This section reviews their capabilities, quantisation techniques, and performance boundaries.

The philosophical foundation for high-performing small models was established by Phi-1 (Gunasekar et al., 2023), which showed that training on textbook-quality synthetic data — carefully curated for instructional clarity rather than raw scale — produced a 1.3B parameter model that outperformed models ten times its size on Python coding benchmarks.

The same logic guided the LoRA training data selection here: 21 high-quality gpt-4o-mini solutions rather than scraped bulk data. Phi-2 (Microsoft Research, 2023) extended this line of work to 2.7B parameters.

TinyLlama (Zhang et al., 2024) established that continued pre-training of a 1.1B parameter model on 3 trillion tokens produced capabilities exceeding the original LLaMA-2 7B model on several benchmarks. In the code domain, Qwen2.5-Coder (Hui et al., 2024) achieved approximately 43.8% pass@1 on HumanEval at 1.5B parameters — the strongest performer in its parameter class. DeepSeek-Coder-1.3B (Guo et al., 2024) achieved approximately 34.8% at smaller scale, while MobileLLM (Liu et al., 2024) explored sub-billion parameter optimisation for edge deployment.

An important technical enabler is model quantisation. GGUF-format models using 4-bit quantisation reduce memory requirements by approximately 75% relative to full-precision weights, at a typically modest performance cost of 1–3% (Dettmers et al., 2023). The llama.cpp runtime (Gerganov, 2023) provides an optimised CPU inference engine using SIMD vectorisation and memory-mapped weight loading.

For parameter-efficient adaptation, LoRA (Hu et al., 2022) introduced trainable low-rank matrices into attention layers, achieving fine-tuning performance comparable to full parameter updates while training only 0.1–1% of total parameters. QLoRA (Dettmers et al., 2023) extended this to 4-bit quantised models, and AdapterFusion (Pfeiffer et al., 2021) confirmed that task-specific adapters could be composed without catastrophic interference.

No published study has evaluated SLM performance under iterative, feedback-based self-refinement for structured software engineering tasks under CPU-only hardware, a gap this study addresses.

Table 2.2: Summary of Small Language Models Evaluated for Selection

Model	Parameters	Reported HumanEval pass@1	Code-Specific	CPU-Deployable	Notes
Phi-1	1.3B	~51% (Gunasekar et al., 2023)	Yes	Yes	Textbook data quality; best-in-class per-param
Phi-2	2.7B	~47.6% (Microsoft, 2023)	Yes	Marginal	Strong reasoning; borderline RAM usage
Qwen2.5-Coder 1.5B	1.5B	~43.8% (Hui et al., 2024)	Yes	Yes	Best code perf at sub-2B; selected
DeepSeek-Coder 1.3B	1.3B	~34.8% (Guo et al., 2024)	Yes	Yes	Compact; strong multilingual code
TinyLlama 1.1B	1.1B	Not reported	Yes	Yes	Minimal memory; general-purpose
MobileLLM 125M	125M	Not reported	Yes	Yes	Ultra-compact; edge/IoT deployment

2.3 Retrieval and Artefact-Based Methods

Retrieval-augmented approaches extend parametric models with a separate, queryable non-parametric memory store, enabling knowledge updating without retraining.

CodeSearchNet (Husain et al., 2019) established a foundational benchmark for natural-language-to-code retrieval. Retrieval-Augmented Generation (Lewis et al., 2020) provided the general theoretical framework combining neural retrieval with generative models. FAISS (Johnson et al., 2019) provided practical scalability infrastructure for billion-scale nearest-neighbour search. RETRO (Borgeaud et al., 2022) and Memorizing Transformers (Wu et al., 2022) showed that

retrieval-conditioned generation could extend a model's practical context window — directly relevant to SLMs with limited context capacity.

In the code domain, RepoCoder (Zhang et al., 2023) integrated retrieval into a multi-step code completion loop, demonstrating measurable improvements over non-retrieval baselines. However, RepoCoder operated on a fixed static knowledge base rather than a dynamically accumulating store of operational experience. No published study has evaluated a RAG system in which the knowledge store is populated dynamically through operational fallback solutions — cases where a local model failed and a stronger cloud model succeeded. This operational feedback loop design is a distinctive contribution of this research.

2.4 Hybrid Routing and Model Selection

Routing — deciding which tasks the local model handles and which escalate to the cloud — determines both cost and quality in a hybrid system. Getting it wrong in either direction wastes money or degrades results.

RouteLLM (Ong et al., 2024) formalised routing as a binary classification problem, finding that LLM API costs could be cut by approximately 50% with minimal performance degradation. However, RouteLLM has two critical limitations in the SDLC context. First, it requires a labelled dataset of pairwise model preferences — unavailable here. Second, it has not been evaluated on code tasks.

FrugalGPT (Chen et al., 2023) proposed a cascade approach querying models in order of increasing capability. However, FrugalGPT relied on model-reported confidence scores, which Kadavath et al. (2022) found to be poorly calibrated for code generation tasks. Dohan et al. (2022) confirmed that confidence calibration failures are a systematic weakness across cascade approaches.

Mixture-of-Experts architectures such as Mixtral (Jiang et al., 2024) and GShard (Lepikhin et al., 2021) implement routing at the architectural level through sparse expert selection. These operate within a single model requiring end-to-end training and are not applicable to the scenario here of combining separately trained models from different vendors at different locations.

The attempt-based routing strategy implemented in this research requires no training data, no confidence calibration, and no architectural modification. It uses executable test pass/fail outcomes — a ground-truth, model-independent, binary signal — as the escalation criterion, making it uniquely suited to software engineering tasks where objective validation is available.

Table 2.3: Comparison of Hybrid Routing Approaches in the Literature

System	Routing Method	Requires Training Data	Task Domain	Limitation
RouteLLM (Ong et al., 2024)	**Learned binary classifier**	**Yes — pairwise** preferences	General NLP	No **code** evaluation; labelled **data** unavailable
FrugalGPT (Chen et al., 2023)	Fixed model cascade	No	General QA	Confidence scores unreliable for code
LLM Cascade (Dohan et al., 2022)	Confidence-based escalation	Partial (calibration)	QA tasks	Calibration fails on code pass/fail tasks
MoE Gating (Jiang et al., 2024)	Sparse expert selection	Yes (architecture)	General LLM	Requires large base model; no deployment separation

2.5 Compliance, Governance, and Safety in AI-Generated Code

The deployment of AI coding assistants in production environments introduces compliance, governance, and security risks distinct from capability-focused concerns. Carlini et al. (2021) found that LLMs can reproduce verbatim fragments of training data including GPL-licensed code, creating legal liability. Xu et al. (2024) found that a non-trivial proportion of AI-generated patches matched protected open-source licences.

Perry et al. (2023) found that developers using AI coding assistants produced significantly more security vulnerabilities — including SQL injection, path traversal, and insecure cryptographic usage — than those working without AI support, despite rating their code as more secure. This paradox of increased subjective confidence

combined with increased objective vulnerability is a serious concern for production deployment.

Detection tools including ScanCode Toolkit (Ombredanne et al., 2020) and Bandit exist for compliance and security scanning respectively, yet their integration into AI coding pipelines remains rare — a gap identified by Hou et al. (2024). The hybrid SDLC agent built in this work does not integrate these tools; this is acknowledged as a limitation in Chapter 6, and the modular validation pipeline provides a structural mechanism into which such scanning could be inserted in future work.

2.6 Multi-Agent and Role-Based Systems

CAMEL (Li et al., 2023) pioneered role-playing communication between LLM agents for complex task decomposition. AgentVerse (Chen et al., 2023) generalised this to multi-stage collaborative pipelines. MetaGPT (Hong et al., 2023) applied the paradigm to software development, assigning agents the roles of product manager, architect, engineer, and quality assurance tester. MetaGPT proved capable of generating complete software artefacts from natural language specifications. However, every agent in MetaGPT's pipeline is a large cloud-hosted LLM — the system provides no mechanism for redistributing task load between local and cloud-based models based on task complexity.

More recent frameworks including AutoGen (Wu et al., 2023) and CrewAI have extended multi-agent orchestration with more sophisticated inter-agent communication. Despite these advances, all frameworks share the fundamental limitation of assuming cloud-based execution for all agents. The role-based design adopted in this work draws on the principle established by MetaGPT and CAMEL that role separation improves interpretability, while extending it to cover hybrid local-cloud execution — a design point none of these frameworks had addressed.

2.7 Knowledge Distillation and Teacher-Student Learning

Hinton et al. (2015) introduced knowledge distillation, showing that a small student model trained on the soft probability outputs of a large teacher model achieves performance significantly exceeding the same student trained directly on hard labels. The teacher's probability distribution encodes relational information enabling more

efficient learning from limited data. Gou et al. (2021) categorised distillation into response-based, feature-based, and relation-based approaches, confirming that response-based distillation using task solutions is broadly effective and requires no access to the teacher's internal architecture — a practical requirement here, where gpt-4o-mini is accessed only through an API.

In the code domain, CodeRL (Le et al., 2022) incorporated execution feedback as a reinforcement signal by training a critic model to predict whether generated code would pass test cases. CodeRL is the closest prior work to the operational distillation pipeline implemented in this work.

The key distinction is that CodeRL required a large pre-collected supervised dataset of execution outcomes to train the critic, whereas the approach here generates training data dynamically through operational fallback — requiring no separate data collection phase and accumulating training signal from precisely the cases where improvement is most needed: tasks where the local model failed but the cloud model succeeded. The training set of 27 examples accumulated in this study is too small for strong quantitative conclusions; scaling this pipeline is identified as a priority for future work in Chapter 6.

2.8 Theoretical Framework

The design of the hybrid SDLC agent is grounded in three interconnected theoretical frameworks, each of which directly shaped a specific architectural decision.

2.8.1 Distributed Cognition and Cognitive Load Distribution

Distributed cognition theory (Hutchins, 1995) proposes that cognitive processes are distributed across components within a sociotechnical system, and that effective capacity can be increased by distributing tasks across specialised components each optimised for a particular function. In this research, this principle motivates the separation into two specialised roles — the Coder Agent for code generation and the Patch-Fix Agent for bug repair. This reduces effective cognitive load on each agent by constraining task scope, improving both interpretability and the ability to diagnose failures during evaluation.

2.8.2 Graceful Degradation and Fault Tolerance

The principle of graceful degradation (Laprie, 1995) holds that well-engineered systems should maintain reduced but acceptable performance under adverse conditions rather than failing catastrophically. The attempt-based routing mechanism directly instantiates this principle: the local SLM provides a first line of response, and the cloud LLM activates only when the local model exhausts its retry budget. This ensures the system continues to produce outputs even when the primary resource is insufficient, degrading gracefully from full local execution to hybrid execution.

2.8.3 Continual Learning and Iterative Knowledge Accumulation

Continual learning theory (Parisi et al., 2019) addresses enabling neural networks to learn incrementally without catastrophic forgetting. The RAG and LoRA pipeline implemented here can be understood as a partial operationalisation of these principles. The FAISS knowledge store accumulates operational experience without modifying model weights, while the LoRA adapter confines weight modifications to a small parameter subset, limiting interference with base model capabilities. Together these mechanisms implement a lightweight continual learning loop in which the system improves through operational experience.

2.9 Evaluation Benchmarks and Their Limitations

The selection of benchmark datasets for evaluating AI-assisted software engineering systems carries significant implications for what claims a study can and cannot make. This section critically reviews each of the four benchmarks used in this research.

HumanEval (Chen et al., 2021) comprises 164 hand-crafted Python programming problems evaluated using unit tests. It has become the de facto standard for code generation evaluation, making it essential for cross-study comparison. However, Liu et al.

(2023) found that many HumanEval solutions can be retrieved near-verbatim from GitHub, raising concerns that high pass@1 scores may partly reflect training data memorisation rather than genuine generalisation. Jain et al. (2024) introduced EvalPlus — an augmented version with harder test cases — finding that model rankings changed noticeably under stricter evaluation. These criticisms are relevant to interpreting the pass@1 result obtained in this work: the result is informative as a

relative comparison against published baselines under identical conditions, but should not be interpreted as a measure of general programming competence.

BigCodeBench (Zhu et al., 2024) was designed explicitly to address HumanEval's limitations, including tasks requiring coordination of multiple library calls and complex data transformations. Its harder task distribution makes it a more demanding evaluation environment for small language models. The 30-task subset used here was drawn from the standard-library category for executable reproducibility — a selection that affects the difficulty distribution and is acknowledged as a limitation.

QuixBugs (Lin et al., 2017) provides 40 buggy Python and Java algorithm implementations, each containing a single-line defect. Its single-bug-per-program structure makes it well-suited for targeted repair evaluation, but limits generalisability to real-world bugs involving multi-line, multi-file defects. The 16-task Python subset was selected to span the full range of algorithm difficulty categories present in the dataset — from syntactically deterministic single-operator defects to complex multi-step algorithmic repairs — while remaining executable within the CPU inference budget of approximately 45–90 seconds per task across up to three retry attempts.

Defects4J (Just et al., 2014) is widely regarded as the most realistic automated program repair benchmark, drawing defects from real Java software projects with developer-written tests. Its inclusion was specifically motivated by the need to evaluate system behaviour on multi-file, cross-class repair tasks, consistent with SWE-Bench findings (Jimenez et al., 2023) that multi-file repair represents a significant challenge for current AI systems. The Java infrastructure requirements introduce additional complexity that is acknowledged as a methodological consideration in Chapter 6.

Table 2.4: Benchmark Datasets — Summary and Critical Assessment

HumanEval	Code Generation	Python	164 tasks	Too easy**;** memorisatio n **risk (**Liu **et al.,** 2023**)**	Standard baseline**;** enables **cross-study** comparison
BigCodeBen ch	Code Generation	Python	2,294 tasks	Large scale; multi-library	Addresses HumanEval limits;

				coordination required	multi-library tasks
QuixBugs	Program Repair	Python	40 tasks	Single-bug-per-program; limits real-world generalisability	Controlled repair eval with executable tests
Defects4J	Program Repair	Java	5 tasks	Java infra complexity; small sample	Real multi-file bugs; tests context-window limits

2.10 Hardware-Constrained and Privacy-Preserving AI Deployment

The deployment of AI models in resource-constrained and privacy-sensitive environments has received growing attention as AI capability expands into production enterprise systems. The case for on-device AI inference rests on three intersecting arguments: latency, cost, and data privacy. On cost, Chen et al.

(2023) in FrugalGPT estimated that a well-designed cascade strategy could reduce LLM API costs by up to 98% for high-volume applications. On privacy, Mireshghallah et al. (2020) showed that even prompts without explicit identifying information could leak sensitive attributes through inference to cloud endpoints. These arguments together provide theoretical justification for the hardware constraint adopted in this work, framing it as a design choice reflecting realistic enterprise deployment requirements rather than a limitation imposed by circumstance.

The llama.cpp runtime (Gerganov, 2023) specifically optimises for CPU environments through SIMD vectorisation and memory-mapped weight loading, substantially narrowing the CPU-GPU performance gap for sub-7B parameter models. Benchmark results from the llama.cpp community indicate that 4-bit quantised 1.5B models generate at 15–30 tokens per second on modern CPUs — sufficient for a non-latency-critical evaluation pipeline.

Federated learning (McMahan et al., 2017) represents an alternative paradigm for privacy-preserving model improvement in which only gradients, not raw data, are transmitted. While not implemented in this work, the LoRA fine-tuning pipeline is

architecturally compatible with a federated extension, since LoRA adapter weights are small and separable from the base model, and could be aggregated across organisational deployments without sharing proprietary code.

2.11 Summary of Key Gaps

Seven gaps emerge from this review.

Three architectural gaps stand out. No published system integrates code generation and bug repair within a unified, role-based SDLC workflow. No hybrid SLM-LLM system uses executable test outcomes as the routing signal — existing routing work relies on labelled preference data or confidence scores, neither of which applies here. And iterative self-refinement has not been tested on sub-2B parameter models under CPU-only constraints; all published work uses large cloud-hosted models.

Four evaluation gaps sit alongside the architectural ones. RAG for code has only been evaluated on static knowledge bases, not dynamically accumulating operational stores. Knowledge distillation has used offline pre-collected datasets rather than solutions generated live through deployment. Multi-agent frameworks have never distributed roles across models of genuinely different capability and location. And evaluations of hybrid coding systems have consistently ignored deployment dimensions — cost, latency, privacy, hardware constraints — treating them as irrelevant to research validity.

2.12 How These Gaps Motivate This Research

The role-based architecture and attempt-based routing address the architectural gaps around workflow integration and routing signal design. The FAISS RAG pipeline and LoRA fine-tuning on live fallback solutions address the knowledge accumulation and distillation gaps. Running the evaluation under CPU-only hardware and comparing against published GPU benchmarks addresses the deployment and evaluation gaps — within a single deployed system rather than as separate studies.

The seven gaps together define the design space this research occupies.

CHAPTER 3

RESEARCH METHODOLOGY

3.1 Introduction

Research design for a system study like this involves choices at multiple levels — from philosophical stance down to how individual datasets were selected. The Research Onion framework (Saunders et al., 2019) provides the structure.

3.1.1 Initial Approach and Methodology Evolution

At the outset of this study, the intended design centred on building a general-purpose Small Language Model and specialising it for programming tasks using parameter-efficient fine-tuning with LoRA adapters trained on the CodeSearchNet dataset (Husain et al., 2019).

Practical assessment during the initial design phase revealed that this approach was fundamentally misaligned with the central research objective of investigating hybrid SDLC workflow behaviour: training a model from a general base would produce insights about model training rather than about hybrid orchestration. The initial strategy was therefore revised in favour of selecting an existing, well-benchmarked code-specialised SLM as the local component and focusing research effort on the orchestration and routing architecture.

A second important evolution occurred during pilot experiments. Initial designs assumed that a static rule-based routing mechanism — routing tasks based on prompt token count or language type — would be sufficient to determine when local inference should be attempted versus when cloud escalation was warranted.

Pilot results revealed that task difficulty was not reliably correlated with these surface-level signals: short prompts could require complex multi-step reasoning, while verbose prompts sometimes encoded straightforward single-line corrections. This motivated the shift toward an attempt-based routing design in which the routing decision is made dynamically based on observable execution outcomes rather than pre-computed static features.

This design drew directly on the principles established by Reflexion (Shinn et al., 2023) and Self-Refine (Madaan et al., 2023), with the critical distinction that refinement was applied to a sub-2B parameter model under CPU-only constraints — a scenario not previously examined in the literature.

3.2 Research Philosophy and Design

3.2.1 Research Onion Framework

The Research Onion (Saunders et al., 2019) provided a structured framework for articulating research design decisions across six layers: ontology, epistemology, philosophy, approach, strategy, and methods. This framework is applied here to make explicit the assumptions underlying each design decision in this work, enabling the reader to evaluate the coherence and appropriateness of the methodology. Table 3.1 summarises the choices made at each layer with justifications; the subsequent subsections explain each choice in detail.

Table 3.1: Research Design Summary — Research Onion Framework (Saunders et al., 2019)

Ontology	Objectivism	**Sy**stem performance is measurable through observable, external test **outcomes** independent **of researcher interpretation**
Epistemology	Post-positivism	Knowledge is generated through controlled experimentation; measurements are objective but fallible proxies for real-world capability
Philosophy	Pragmatism	Research questions are practical; methodology selected for fitness-of-purpose; both rigorous measurement and interpretive supplementation are permitted
Approach	Deductive	System designed from theoretical principles; experiments test whether the system performs as hypothesised

Strategy	Design Science Research (Hevner et al., 2004)	Primary vehicle for knowledge production is the construction and rigorous evaluation of a novel artefact
Research Choice	Mono-method quantitative with qualitative supplementation	Pass/fail rates are the primary measure; qualitative failure analysis supplements interpretation of results
Time Horizon	Cross-sectional	All experiments conducted within a single defined evaluation period using fixed model versions and configurations
Data Collection	Automated benchmark execution	Eliminates observer bias; ensures reproducibility; ground truth from executable test suites

3.2.2 Ontological Position — Objectivism

Ontology concerns the nature of reality and what can be known about it. An objectivist ontological position was adopted, which holds that the performance of the hybrid SDLC agent is a property of the system itself — measurable through observable, reproducible test outcomes — rather than something constructed through researcher interpretation. Pass and fail outcomes on benchmark test suites represent objective, external measurements of system behaviour: a test either passes or it does not, irrespective of how the researcher or any observer interprets the output.

A potential challenge to this position arises from the qualitative failure classifications in Chapter 5 — PATCH_APPLY_FAIL, FAIL_AFTER_APPLY, ALGORITHMIC_ERROR. These are not independent interpretations; they are labels derived post-hoc from error type strings, stack traces, and test output files, and would be reproduced identically by any researcher with access to the same run logs. The objectivist foundation of the study therefore remains intact.

3.2.3 Epistemological Position — Post-Positivism

Epistemology concerns how knowledge is generated and validated. A post-positivist epistemological stance was taken. Post-positivism, as articulated by Phillips and Burbules (2000), accepts the positivist premise that objective measurement is the

primary vehicle for scientific knowledge, while acknowledging that all measurements are fallible and that observed results are imperfect proxies for the underlying phenomena they are intended to measure. This nuanced stance is more appropriate for this study than strict positivism for two reasons.

First, the benchmark datasets used as measurement instruments have known limitations — HumanEval has been criticised for memorisation risk (Liu et al., 2023), and the small Defects4J subset is insufficient for strong statistical inference — meaning that results must be interpreted with appropriate epistemic caution rather than treated as definitive truths.

Second, the qualitative failure analysis supplements the quantitative metrics with interpretive insight that cannot be reduced to numbers alone. Post-positivism accommodates this supplementation without abandoning the primacy of measurable, replicable results as the core knowledge claim. Knowledge about the hybrid SDLC agent's capabilities is thus generated through controlled experimentation and measurement of observable outcomes, with the understanding that findings are provisional, contextually bounded, and subject to revision as experimental conditions are varied in future work.

3.2.4 Research Philosophy — Pragmatism

A pragmatist philosophical stance was adopted. Pragmatism, as articulated by Creswell (2014), holds that the most appropriate methodology is the one that best serves the research questions rather than the one that adheres most closely to any particular philosophical tradition. For this study, the research questions concern whether a practical system works — whether it successfully executes software engineering tasks under realistic constraints — and the answer to that question is best obtained through building and testing the system.

A purely positivist stance was not appropriate because the results are contextually bounded rather than universal laws; an interpretivist stance was not applicable because the research questions are answered by measurable system outputs, not human experience.

Pragmatism fits because the central question — does this system work under real hardware constraints? — calls for measurement rather than interpretation. It also aligns naturally with Design Science Research, which evaluates knowledge through the utility of designed artefacts.

3.2.5 Research Approach — Deductive

A deductive approach was adopted: the hybrid SDLC agent was designed from established theoretical principles — distributed cognition, graceful degradation, and continual learning — and experiments tested whether the resulting system performed as hypothesised.

The deductive approach contrasts with an inductive approach, which would begin with observations and derive theory from them. While the post-hoc failure pattern analysis in Chapter 5 has inductive characteristics — patterns in the run logs informed the classification of failure types that were not fully anticipated in the initial experimental design — the primary research design is deductive: theoretical principles informed the system design, and experiments evaluated the resulting artefact against pre-specified success criteria.

3.2.6 Research Strategy — Design Science Research

The research strategy adopted in this work is Design Science Research (DSR), as articulated by Hevner et al. (2004). DSR is a well-established research paradigm in information systems and applied computing that treats the construction and rigorous evaluation of novel artefacts as the primary vehicle for producing knowledge. Hevner et al. (2004) identify three central cycles in DSR: the Relevance Cycle, which grounds the research in real-world problems; the Design Cycle, in which artefacts are constructed and refined; and the Rigour Cycle, which ensures the research is grounded in existing knowledge and contributes to the knowledge base.

Each of these cycles is identifiable in this work. The Relevance Cycle is established by the practical motivation of privacy-aware, cost-effective AI-assisted software development — a real and commercially significant problem documented in Chapter 2.

The Design Cycle is operationalised through the iterative development of the hybrid SDLC agent, including the pivot from a fine-tuning-focused initial design to the hybrid orchestration architecture described in Section 3.1.1. The Rigour Cycle is supported by the comprehensive literature review in Chapter 2 and the use of four standardised benchmark datasets for evaluation. DSR is appropriate here because the primary contribution of this research is a working artefact — the hybrid SDLC agent — whose value is established through utility in solving a practical problem.

3.2.7 Research Choice — Mono-method Quantitative with Qualitative Supplementation

Quantitative methods drove the evaluation: pass/fail rates and timing statistics directly answer whether the system works and how fast. Qualitative failure pattern analysis — examining why specific tasks failed — supplements the numbers where the logs require interpretation. The two approaches are complementary, not competing.

3.2.8 Time Horizon — Cross-Sectional

All experiments were conducted within a single defined evaluation period using fixed model versions, benchmark datasets, and runtime configurations. The LoRA and RAG experiments are proof-of-concept demonstrations of a longitudinal improvement capability, but evaluating long-run improvement trends is left for future work.

3.3 Dataset Selection and Task Preparation

Four publicly available benchmark datasets were selected to evaluate the hybrid SDLC workflow: QuixBugs (Lin et al., 2017), Defects4J (Just et al., 2014), HumanEval (Chen et al., 2021), and BigCodeBench (Zhu et al., 2024). The selection criteria applied were: availability of executable unit tests providing automated ground-truth validation; open licensing permitting research use; coverage of both code generation and program repair task types; and representation of varying difficulty levels to test the system across its operating range.

Each dataset was used in its entirety or via a justified subset. The subset decisions were informed by two considerations: the approximate CPU inference time per task measured during pilot experiments as approximately 45–90 seconds per task

including all retry attempts, and the need to represent a sufficient range of task difficulty to enable meaningful failure pattern analysis. Table 3.2 documents the full size of each dataset alongside the subset used and the explicit rationale for each subset decision.

Table 3.2: Benchmark Dataset Characteristics and Subset Justification

Dataset	**Task Type**	**Language**	**Full Size**	**Subset** Used	**Rationale**	**Validation**	**Licence**
QuixBugs	**Program** Repair	Python	40	16	**CPU** time constraint; **representative** difficulty spread **across algorithm categories**	**Unit** tests	MIT
Defects4J	Program Repair	Java	835	5	Java infra setup time; sufficient to demonstrate multi-file context limitation empirically	Developer tests	MIT
HumanEval	Code Generation	Python	164	164	Full dataset used; manageable within CPU inference budget; enables direct benchmark	Unit tests	MIT

					comparison		
BigCodeBench	Code Generation	Python	2,294	30	Standard-library subset selected for dependency-free reproducibility on CPU hardware	Unit tests	Apache-2.0

The decision to use the full HumanEval dataset of 164 tasks reflects both the manageable size of this dataset within the CPU inference budget and the importance of producing a result directly comparable to the published benchmark figures used as the primary contextualisation reference in Chapter 5.

The 30-task BigCodeBench subset was drawn from the standard-library category to avoid external library dependencies that would require complex environment configuration on the experimental hardware. The 16-task QuixBugs subset was selected to span the range of algorithm difficulty categories present in the full dataset. The 5-task Defects4J subset was selected to expose the system's behaviour on multi-file Java repair tasks — five tasks were sufficient to expose the context window limitation empirically without requiring the substantial Java infrastructure configuration needed for a larger evaluation.

3.4 Model Selection

Model selection for the local SLM was governed by four criteria applied in order of priority. First, RAM constraint: the local model must fit within 8 GB RAM alongside the operating system and orchestration pipeline, leaving approximately 6.5 GB available for model weights. In 4-bit quantised GGUF format, this constrains the effective parameter limit to approximately 7B parameters.

Second, benchmark performance: among models satisfying the RAM constraint, the model with the highest published HumanEval pass@1 score was preferred, as this provided the most favourable baseline for confirming the value of the hybrid routing

strategy. Third, CPU inference speed: models with marginal RAM fit were deprioritised if their CPU inference speed fell below approximately 5 tokens per second, as this would make the experimental timeline infeasible. Fourth, code specialisation: general-purpose models were deprioritised relative to code-specific models trained or fine-tuned on programming data.

Applying these criteria, Qwen2.5-Coder 1.5B (Hui et al., 2024) was selected as the local SLM. It achieved the highest published HumanEval pass@1 score of approximately 43.8% among models satisfying all constraints, required only approximately 1.5 GB RAM in 4-bit GGUF format, and generated at approximately 18 tokens per second on the experimental hardware. gpt-4o-mini (OpenAI, 2024) was selected as the cloud-based fallback on the basis of its strong published performance on code tasks and availability through a standard API.

Table 3.3: Model Selection Comparison and Selection Rationale

Model	Parameters	HumanEval pass@1	RAM (4-bit GGUF)	CPU Speed	Selected	Rationale
Qwen2.**5-Coder 1.5B**	**1.5B**	**~43.8%**	**~1.5 GB**	**~18 tok/s**	**Yes —** local **SLM**	**Highest HumanEval within RAM budget; code-specialised**
DeepSeek-Coder 1.3B	1.3B	~34.8%	~1.3 GB	~22 tok/s	No	Lower benchmark performance
StarCoder 2 3B	3B	Not reported	~3.5 GB	~8 tok/s	No	Marginal memory; slow CPU inference
Qwen Coder 7B	7B	Not reported	~6.5 GB	~3 tok/s	No	Exceeds 8 GB RAM constraint

Larger models — specifically StarCoder2 3B and Qwen Coder 3B — were also considered but eliminated during pilot testing rather than on benchmark grounds

alone. When loaded on the 8 GB RAM machine alongside the operating system and orchestration pipeline, both models caused consistent generation timeouts. Token generation rates fell to approximately 3–4 tokens per second, making it impossible to produce a complete function body or repair patch within the 30-second execution budget. Since the evaluation required processing hundreds of tasks, models that reliably timed out were not viable regardless of their published benchmark performance under GPU conditions.

Qwen Coder 7B was excluded on RAM grounds prior to inference testing, as its 4-bit GGUF footprint of approximately 6.5 GB left insufficient headroom for the pipeline and operating system.

3.5 Role-Based Agent Workflow

The hybrid workflow was organised around two specialised agents with distinct, non-overlapping responsibilities: the Coder Agent, responsible for program synthesis from natural language task specifications, and the Patch-Fix Agent, responsible for automated bug repair given a defective implementation and failing test information. This role-based decomposition was motivated by three considerations drawn from the theoretical framework presented in Chapter 2.

Distributing tasks across specialised components reduces the cognitive load on each (Hutchins, 1995) — in practice, this meant prompt construction and failure analysis could be tailored specifically to either generation or repair rather than handling both in a single generic prompt. Role separation also made failures easier to interpret: a failing task immediately signals whether the problem is in generation or repair, rather than requiring disambiguation from raw logs.

The Coder Agent operated on tasks from HumanEval and BigCodeBench. The Patch-Fix Agent operated on tasks from QuixBugs and Defects4J. Both agents shared the same underlying orchestration framework — prompt construction, local SLM invocation, test execution, retry logic, and fallback routing — with agent-specific prompt templates as the primary point of differentiation. The templates are documented in full in Appendix A.

3.6 Routing Strategy

3.6.1 Design Rationale

The routing strategy governed when the system used the local SLM and when it escalated to the cloud LLM. The design objective was to maximise overall task success rates while minimising unnecessary cloud API calls — both to reduce cost and to validate the hypothesis that local-first execution is viable for a meaningful proportion of software engineering tasks. The routing mechanism needed to be training-free, since no labelled dataset of task-difficulty estimates was available, and objective, since subjective routing heuristics would introduce researcher bias into the evaluation.

These requirements ruled out learned routing approaches such as RouteLLM (Ong et al., 2024), which require pairwise preference labels, and confidence-score cascades such as FrugalGPT (Chen et al., 2023), which rely on model-reported confidence that has been shown to be poorly calibrated for code tasks (Kadavath et al., 2022). The executable test suite provided a uniquely appropriate routing signal: pass/fail outcomes are binary, objective, ground-truth measurements of task completion that require no labels and no calibration.

3.6.2 Routing Mechanism — Step by Step

On receiving a task, the orchestration pipeline first applied pre-execution filters to identify tasks certain to exceed local model capability regardless of retry. Defects4J tasks were flagged for review given their known multi-file Java complexity, and tasks with prompt token counts exceeding 1,800 tokens were flagged as at-risk for context truncation within the local model's 2,048-token context window.

For all remaining tasks, the local model was invoked with up to three generation attempts. On the first attempt, the task prompt was constructed from the standard template.

If the generated code failed the test suite, the full error output — error type, error message, and traceback — was appended to the prompt as a structured feedback block labelled Previous Attempt Feedback, and a second attempt was made. If the second attempt also failed, the same feedback process was repeated for a third attempt. The maximum of three attempts was set based on pilot experiment observations that recoveries beyond the second attempt were negligible, and that

additional attempts consumed disproportionate CPU time relative to their marginal benefit.

If all three local attempts failed, the task was escalated to gpt-4o-mini via the OpenAI API. The cloud model received the base task prompt without the accumulated feedback history — pilot experiments indicated that providing the full failure history to the cloud model produced no measurable improvement in recovery rates and increased API token costs. The cloud model was given a single attempt. If the cloud attempt succeeded, the solution was written to both the FAISS RAG index and the LoRA fine-tuning dataset. All outcomes, including failures at both stages, were logged in the run directory.

3.7 Incremental Improvement Pipeline

3.7.1 Retrieval-Augmented Generation Design

The RAG component was designed to accumulate successful cloud fallback solutions as a dynamic knowledge store, enabling the local SLM to benefit from few-shot examples of previously solved similar tasks on future queries. FAISS (Johnson et al., 2019) was selected as the vector index backend on the basis of its proven scalability, its availability as a lightweight Python library compatible with the CPU-only experimental environment, and its support for cosine similarity search over dense vector embeddings.

Each successful fallback solution was stored as a structured case record containing the task prompt, the failed SLM output from the final local attempt, the successful LLM solution, and the validation result. At query time, the task context was embedded using a sentence-transformer model and the three most similar cases were retrieved by cosine similarity. The choice of k=3 nearest neighbours was informed by the observation from Lewis et al. (2020) that retrieval benefit typically saturates at three to five examples, and that including more examples risks consuming excessive prompt tokens within the local model's context window.

3.7.2 LoRA Fine-Tuning Design

The LoRA fine-tuning component was designed to improve the model's parametric knowledge by training on successful cloud fallback solutions, implementing the

teacher-student distillation paradigm (Hinton et al., 2015) with gpt-4o-mini as teacher and Qwen2.5-Coder 1.5B as student.

The LoRA hyperparameters were determined as follows. A rank of r=8 was selected as a balance between adaptation expressivity and parameter efficiency: lower ranks (r=4) have been shown to be insufficient for task-specific adaptation in code domains (Hu et al., 2022), while higher ranks risk overfitting on small datasets.

A scaling factor of alpha=16 (ratio alpha/r=2) was selected following the recommendation of Hu et al. (2022) that alpha/r ratios between 1 and 4 produce stable training. Target modules q_proj and v_proj in all transformer layers were selected on the basis that adapting the attention query and value projections is sufficient for task-specific adaptation while minimising trainable parameter count. Dropout of 0.05 was applied as regularisation appropriate for the small training set size.

Training ran for three epochs using the AdamW optimiser with learning rate 2e-4, batch size 1, and gradient accumulation over 4 steps. Three epochs were selected based on validation loss monitoring during pilot training: loss converged by epoch 3 with no overfitting signs, making additional epochs unnecessary.

3.8 Hybrid SDLC Workflow Architecture

The hybrid SDLC workflow brought together all components described in Sections 3.5 through 3.7. The key steps in the pipeline were as follows:

- Task input is normalised into a unified schema across all four benchmark datasets (QuixBugs, Defects4J, HumanEval, BigCodeBench).
- The Agent Router applies a Stage 1 rule-based pre-filter, then Stage 2 attempt-based SLM routing.
- The Coder Agent processes HumanEval and BigCodeBench generation tasks; the Patch-Fix Agent processes QuixBugs and Defects4J repair tasks.
- The local SLM (Qwen2.5-Coder 1.5B) executes up to three attempts, with full error feedback injected between attempts.

- On SLM failure after three attempts, tasks escalate to gpt-4o-mini for a single cloud fallback attempt.
- All artefacts are logged per run; successful fallback solutions populate the FAISS RAG index and LoRA training dataset.
- The complete end-to-end architecture with component detail is shown in Figure 4.1 (Chapter 4).

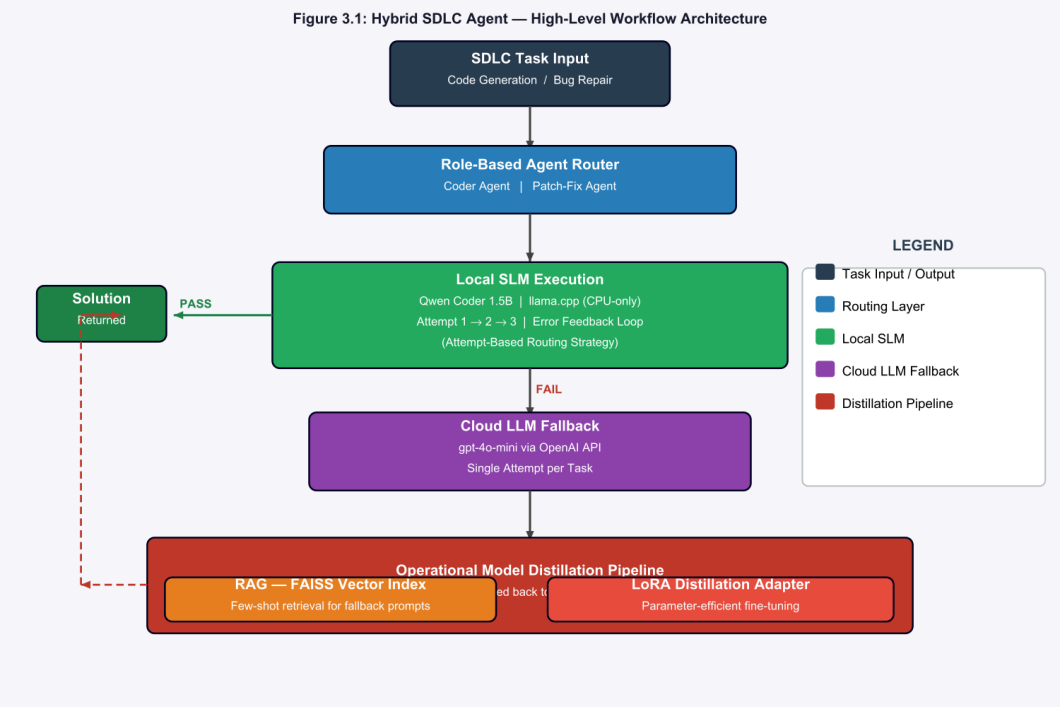

Figure 3.1: Hybrid SDLC Agent — High-Level Workflow Architecture

3.9 Ethical Considerations

The work was conducted in compliance with Liverpool John Moores University's research ethics guidelines. The study involved no human participants, no collection of personal data, and no deception. All benchmark datasets — QuixBugs, Defects4J, HumanEval, and BigCodeBench — are publicly available under open licences (MIT and Apache-2.0) that explicitly permit research use, including the generation, modification, and evaluation of code artefacts. No proprietary or confidential code was processed at any stage.

The use of gpt-4o-mini via the OpenAI API involved transmitting task prompts to OpenAI's servers. All transmitted prompts were drawn exclusively from public benchmark datasets and contained no proprietary, sensitive, or personally identifiable information. The use of the API was conducted in compliance with OpenAI's terms of service for research applications. No persistent storage of transmitted data beyond the local run directory logs was attempted or required.

A potential ethical consideration in AI-assisted code generation research is the risk that generated code could contain security vulnerabilities or reproduce GPL-licensed code fragments, as documented in the literature review (Perry et al., 2023; Carlini et al., 2021).

In this study, all generated code was used exclusively for evaluation against benchmark unit test suites and was not deployed in any production environment or made available to end users. The research therefore posed no direct risk of harm from generated code. The absence of automated compliance and security scanning in the evaluation pipeline is acknowledged as a limitation.

Regarding reproducibility obligations, all experimental configurations, prompt templates, run logs, and LoRA training configurations are documented in this thesis (Chapter 4 and Appendices A–C). The benchmark datasets are publicly available, and the model weights are publicly available through their respective repositories. A researcher with equivalent hardware and API access would be able to reproduce the experimental results reported in Chapter 5.

3.10 Validity and Reliability

Establishing the validity and reliability of a Design Science Research study requires consideration of both artefact quality and evaluation methodology quality. Following Hevner et al. (2004), artefact quality in DSR is assessed through utility — does the artefact solve the problem it was designed to address? — rigour — is the evaluation methodology sound and reproducible? — and relevance — does the artefact address a real and significant problem? These dimensions are addressed throughout this thesis. The following subsections address the validity and reliability considerations applicable specifically to the experimental evaluation methodology.

Table 3.4: Validity and Reliability — Risks and Mitigations

Validity / Reliability Dimension	**Risk**	**Mitigation**
Internal validity rsult accuracy	**SLM inference variability due to temperature sampling**	**Temperature fixed at 0.2; determinism maximised within llama.cpp constraints**
Internal validity — confounding variables	Hardware thermal throttling on long CPU runs	Execution timeout standardised at 30s; timestamps logged for anomaly detection
External validity — generalisability	Single hardware configuration; one local SLM; one cloud LLM	Results contextualised against published benchmarks obtained under different conditions
Construct validity — benchmark validity	HumanEval memorisation risk (Liu et al., 2023)	Four benchmarks at different difficulty levels; limitations acknowledged in Chapter 5
Reliability — reproducibility	Non-deterministic gpt-4o-mini outputs across API calls	All prompts, configs, and run logs archived in structured run directories (Appendix B)

The primary internal validity threat was variability in SLM outputs due to temperature-based sampling, mitigated by fixing temperature at 0.2. Zero temperature would produce fully deterministic outputs but would prevent the model exploring alternative solution strategies on retry attempts — a deliberate trade-off in favour of retry effectiveness. Hardware thermal throttling on extended CPU runs was addressed by standardising execution timeouts and logging timestamps for anomaly detection.

A temperature of zero would produce fully deterministic outputs but would suppress the model's ability to explore alternative solution strategies on retry attempts — a design trade-off explicitly made in favour of retry effectiveness. Hardware thermal throttling on extended CPU runs was mitigated by standardising execution timeouts and logging timestamps for anomaly detection.

Generalisability is the main external validity constraint: results reflect a single hardware configuration and two specific models. Findings are contextualised against

published benchmarks in Chapter 5 to allow readers to calibrate how far the results might extend to other hardware and model combinations.

Reproducibility is supported by full documentation of prompts, model versions, and runtime configurations in this thesis and Appendices A–C. The primary residual source of irreproducibility is the non-deterministic behaviour of gpt-4o-mini across API calls — an accepted constraint of studies using commercial LLMs.

3.11 Summary

The Research Onion framework (Saunders et al., 2019) structured the methodological design across six layers: an objectivist ontological position, post-positivist epistemology, pragmatist philosophy, deductive approach, and Design Science Research strategy. The rationale for pragmatism over positivism and interpretivism was argued explicitly, and the research was positioned as post-positivist to reflect its contextually bounded experimental scope.

The dataset selection, model selection, role-based agent workflow, routing strategy, and incremental improvement pipeline were each described with explicit justification — including subset size rationale, model selection criteria applied in priority order, LoRA hyperparameter selection rationale, and routing mechanism design decisions. Ethical considerations were addressed across five dimensions: participant safety, data provenance, API compliance, generated code risk, and reproducibility. Validity and reliability risks were identified and mitigated through fixed experimental configurations, standardised logging, and multiple benchmark coverage. Chapter 4 described the practical implementation of this methodology in detail.

CHAPTER 4

SYSTEM IMPLEMENTATION AND EXPERIMENTAL SETUP

4.1 Introduction

This chapter describes the implementation and execution of the hybrid SDLC workflow introduced in Chapter 3, covering the experimental environment, benchmark preparation, agent implementation, routing strategy evolution, and improvement pipeline setup.

The implementation phase involved several key activities:

- configuring the experimental environment
- preparing benchmark datasets in executable task format
- implementing the hybrid agent orchestration framework
- constructing prompt generation and runtime execution pipelines
- implementing attempt-based retry and fallback routing mechanisms
- integrating local improvement mechanisms using retrieval and fine-tuning

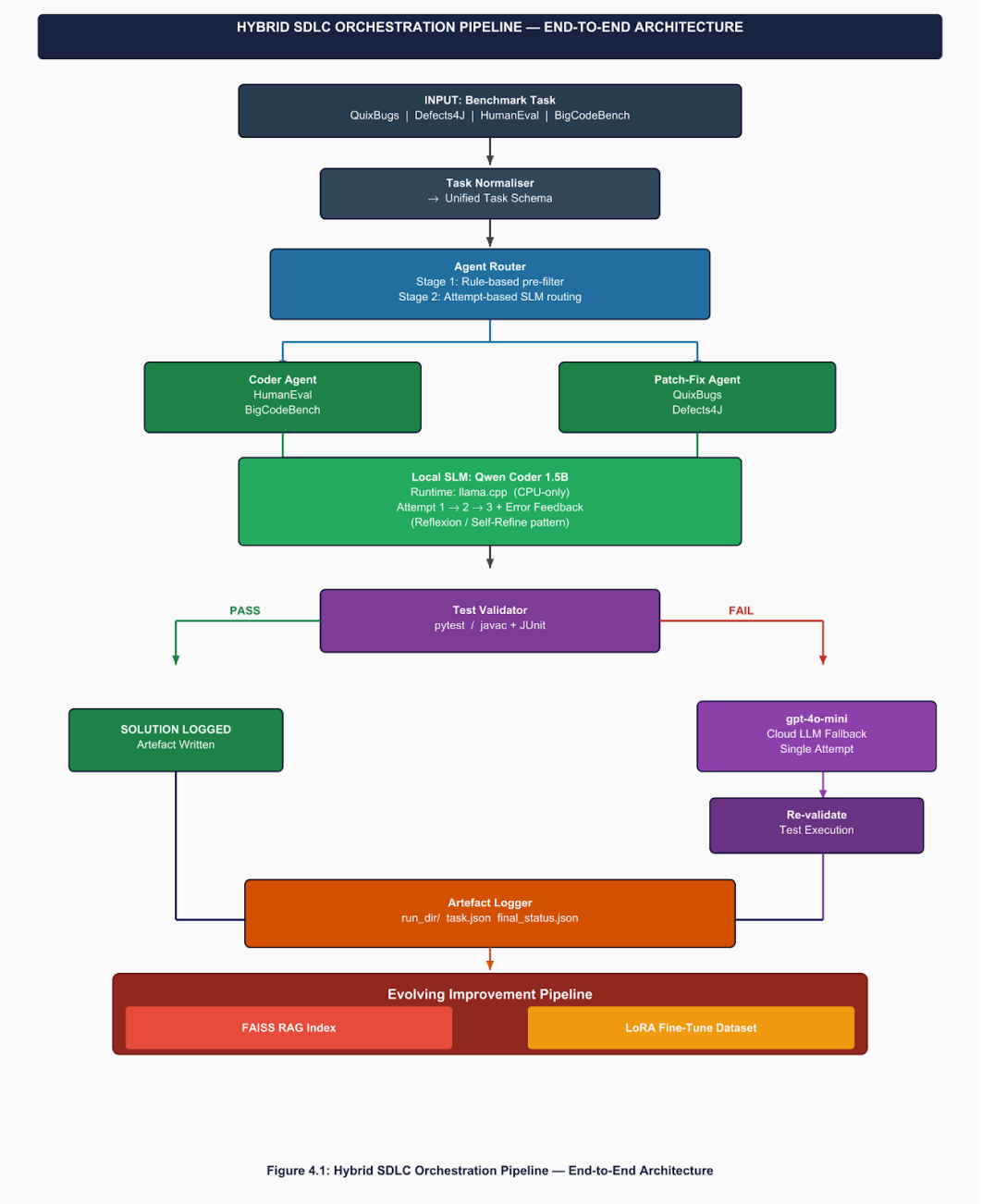

Figure 4.1: Hybrid SDLC Orchestration Pipeline — End-to-End Architecture

4.2 Experimental Environment

All experiments were conducted on a personal workstation running macOS 15.7.3. The hardware configuration was as follows:

Hardware Configuration

```
System  : Personal workstation

CPU     : Dual-core processor
```

```
Memory  : 8 GB RAM

GPU     : None (CPU-only inference)
```

Operating System

```
macOS 15.7.3
```

Programming Environment

```
Python 3.11.7

Python 3.11.7
```

Local inference was executed using the llama.cpp runtime, which allowed quantised transformer models to run efficiently on CPU hardware. The installed tools included:

```
llama-cli

llama-completion
```

The language models used in the experiments were as follows:

Local Small Language Model

```
Qwen2.5-Coder 1.5B Instruct

4-bit quantised GGUF model (~1.5 GB RAM)
```

Fallback Large Language Model

```
gpt-4o-mini

Accessed through OpenAI API calls
```

The hybrid SDLC orchestration pipeline was implemented using Python scripts responsible for:

- dataset task loading
- prompt construction
- model execution
- attempt-based retry and routing control
- test execution and validation

- artifact logging for experiment runs

Table 4.1: Experimental Software Environment

Inference **runtime**	**llama.**cpp **(llama-cli,** llama**-completion)**
	Qwen2.5-Coder 1.5B Instruct — 4-bit GGUF (~1.5 GB RAM)
Cloud fallback LLM	gpt-4o-mini via OpenAI API (OpenAI, 2025)
Primary pipeline	Python 3.11.7

Qwen2.5-Coder 1.5B was selected following a brief comparative pilot on the same hardware. DeepSeek-Coder-1.5B was loaded via llama-cli and tested against a sample of QuixBugs and HumanEval tasks. Two variants of the Qwen2.5-Coder 1.5B model were used across the experimental pipeline: the instruction-tuned variant (qwen2.5-coder-1.5b-instruct) was used for all inference tasks within the Coder Agent and Patch-Fix Agent pipelines, as it produced more consistently formatted outputs in response to instruction-style prompts; the same instruct variant was also used as the base for LoRA fine-tuning, providing a stronger starting point for supervised fine-tuning on instruction-formatted training examples. The base model (qwen2.5-coder-1.5b) was used only during the initial pilot comparison stage.

Both models produced broadly comparable output on simple tasks; however, Qwen2.5-Coder 1.5B achieved a marginally higher pass rate on the sample set, consistent with its stronger published HumanEval benchmark (43.8% vs 34.8%). StarCoder2 3B and Qwen Coder 3B were also tested but caused consistent inference timeouts — token generation rates dropped to around 3–4 tokens per second under the 8 GB RAM constraint, making task completion within the 30-second execution budget impractical. Qwen Coder 7B was excluded on RAM grounds alone. Phi-1 (Gunasekar et al., 2023), which reported approximately 51% pass@1 on HumanEval at 1.3B parameters, was considered but not selected. Despite its strong published benchmark, Phi-1 was trained on textbook-quality Python data rather than a broad code corpus, and its GGUF-format quantised variant was not available through the llama.cpp ecosystem at the time of experimentation, making CPU-only deployment impractical for this study.

4.3 Benchmark Dataset Preparation in Practice

Dataset preparation normalised four different source formats into a unified task schema for pipeline processing:

- analysing the raw dataset structure

- converting each instance into a unified task schema

4.3.1 QuixBugs Dataset Preparation

QuixBugs was distributed as a repository containing defective algorithm implementations and corresponding test cases. The typical raw repository structure was as follows:

```
python_programs/

correct_python_programs/

json_testcases/
```

Example raw buggy program:

```
def bitcount(n):

    count = 0

    while n:

        n ^= n - 1

        count += 1

    return count
```

Example test cases:

```
[[127], 7]

[[128], 1]

[[3005], 9]
```

Each instance was converted into a structured task record used by the orchestration pipeline:

```
{

  "dataset"    : "quixbugs",

  "task_id"    : "bitcount",
```

```
  "language"   : "python",
  "buggy_file" : "python_programs/bitcount.py",
  "test_cmd"   : "pytest tests/test_bitcount.py",
  "timeout"    : 30
}
```

4.3.2 Defects4J Dataset Preparation

Defects4J provided reproducible bugs extracted from real Java projects. Each defect contained:

- project identifier
- bug identifier
- buggy revision
- fixed revision
- failing test cases
- developer patch

Example defect metadata:

```
Project    : Chart
Bug ID     : 1
```

Failing Test:

```
  org.jfree.chart.renderer.category.junit
    .AbstractCategoryItemRendererTests::test2947660
```

Example patch fragment:

```
- if (dataset != null) {
+ if (dataset == null) {
```

Each defect was converted into a task representation containing the failing test context and patch generation instruction. Five defects were selected from the Chart project to maintain consistency in build tooling across the Java evaluation tasks. The

timeout was set to 120 seconds to accommodate the additional time required for Java compilation and test execution.

4.3.3 HumanEval Dataset Preparation

HumanEval tasks were already distributed as JSONL records containing prompts and test harnesses. The following example illustrated a typical task record:

```
{
  "task_id"     : "HumanEval/0",
  "prompt"      : "def has_close_elements(numbers, threshold):",
  "entry_point" : "has_close_elements",
  "test"        : "def check(candidate): ..."
}
```

These tasks were primarily executed by the Coder Agent. All 164 tasks in the standard benchmark were used, enabling direct comparison with published pass@1 results (Hui et al., 2024; Guo et al., 2024).

4.3.4 BigCodeBench Dataset Preparation

BigCodeBench extended the HumanEval benchmark with more complex programming instructions requiring coordination of multiple library calls and API-specific edge cases. A typical task record had the following structure:

```
{
  "task_id"        : "BigCodeBench/4",
  "instruct_prompt": "Count occurrences of integers in a list",
  "entry_point"    : "task_func",
  "libs"           : ["collections"]
}
```

For experimental feasibility, a subset of 30 tasks from the standard-library category was used. This category required only Python built-in modules, which avoided environment-specific dependency failures unrelated to model capability.

Table 4.2: Benchmark Datasets Used in the Evaluation

QuixBugs	Program **Repair**	Python	16	pytest **unit** tests
Defects4J	Program Repair	Java	5	Developer tests (JUnit)
HumanEval	Code Generation	Python	164	Built-in test harness
BigCodeBench	Code Generation	Python	30	Built-in test harness

4.4 Hybrid Agent Implementation

The hybrid SDLC workflow implemented two specialised agents:

Patch-Fix Agent -- program repair

Coder Agent -- program generation

Both agents used a shared orchestration framework consisting of:

- prompt generation
- local model inference
- test execution
- attempt-based retry logic
- fallback routing to cloud LLM

4.4.1 Prompt Construction and Runtime Arguments

Before executing a task, the orchestration pipeline constructed prompts using runtime parameters defined in the experiment configuration. Typical run arguments included:

- dataset selection
- model identifier
- retry limit (maximum 3 attempts)
- maximum token length

- temperature

- timeout configuration

Example execution command:

python run_pipeline.py \

```
--dataset   humaneval \
--model     qwen-coder-1.5b \
--attempts  3 \
--max_tokens 512
```

The prompt structure used during execution followed this format:

Task Description

Code Context

Agent Instruction

Expected Output Format

For repair tasks, prompts included the defective implementation and instructions to generate a corrected version. For generation tasks, prompts included the function specification and entry point. On retry attempts, the prompt was extended with a Previous Attempt Feedback section containing the full error type, message, and traceback from the preceding execution.

The prompt templates used for each agent are reproduced from Appendix A:

Coder Agent Prompt Template (HumanEval and BigCodeBench tasks):

System:

```
You are a Python programming assistant.
Complete the function body only.
Do not include explanation or markdown formatting.
```

Task Description:

```
{task_description}
```

Function Signature:

```
{function_signature}
```

Complete the implementation below:

Patch-Fix Agent Prompt Template (QuixBugs and Defects4J tasks):

System:

```
You are a software debugging assistant.

Return only the corrected complete source file.

Do not include explanation.
```

The following program contains a bug:

```
{buggy_source_code}
```

The program failed the following test:

```
{test_failure_message}
```

Return the corrected complete file.

Retry Prompt Extension Template (appended on attempts 2 and 3):

Previous Attempt Feedback:

```
Error Type    : {error_type}

Error Message : {error_message}

Traceback     : {traceback_summary}
```

Do not repeat this approach.

Revise your implementation to address this specific error.

4.4.2 Patch-Fix Agent Implementation

The Patch-Fix Agent execution pipeline proceeded through the following steps:

- load buggy code
- construct repair prompt

- run local SLM
- apply candidate patch
- execute tests
- retry with feedback if tests fail (up to 3 attempts)
- escalate to fallback LLM if retry budget exhausted

Patch-Fix Agent pseudocode:

```
load_task()


for attempt in SLM_ATTEMPTS:
    prompt = build_patch_prompt(task)
    candidate_patch = run_local_slm(prompt)
    apply_patch(candidate_patch)
    result = run_tests()
    if result == PASS:
        save_success()
        exit()
    append_feedback_to_prompt(result.error)


fallback_solution = run_cloud_llm(task)
apply_patch(fallback_solution)
validate_again()
```

4.4.3 Coder Agent Implementation

Coder Agent pseudocode:

```
load_task()


for attempt in SLM_ATTEMPTS:
```

```
    prompt = build_generation_prompt(task)

    candidate_code = run_local_slm(prompt)

    save_candidate(candidate_code)

    result = run_tests()

    if result == PASS:

        record_success()

        exit()

    append_feedback_to_prompt(result.error)


fallback_solution = run_cloud_llm(task)

validate_solution(fallback_solution)
```

4.4.4 Runtime Parameters

All pipeline executions used the following standard runtime configuration:

Maximum SLM attempts per task : 3

```
Maximum token output          : 512

Temperature                   : 0.2

Execution timeout             : 30 seconds per test run
```

gpt-4o-mini fallback : invoked only after exhausting local retry budget

4.4.5 Routing Strategy: Evolution from Rule-Based to Attempt-Based

The routing strategy did not begin in its final form. Its development across the experimental runs represents one of the central methodological contributions of this research and is documented here as part of the evolving experimental narrative.

Phase 1 — Initial Rule-Based Approach (Early Experiments)

The first routing implementation applied static pre-flight rules before any local SLM invocation. Tasks were immediately escalated to gpt-4o-mini if any of the following conditions were met:

```
  Rule 1: dataset membership == 'defects4j'

          (Defects4J tasks involve multi-file Java projects that exceed the
local

           model's 2,048-token context window — immediate escalation was
correct)



  Rule 2: prompt token count > threshold

          (Long prompts risked truncation before task-critical content)



  Rule 3: multiple source file references detected

          (Multi-file context exceeded local model capacity)
```

This approach worked correctly for Defects4J, which was never a candidate for local execution. However, early experiments on HumanEval and QuixBugs revealed a critical

limitation: task difficulty could not be reliably predicted by surface-level signals such as prompt length or token count alone. A short prompt did not guarantee a task was within local model capability; a long prompt did not guarantee it would fail. The binary escalation produced premature cloud invocations on tasks the local model could have solved, and missed opportunities for iterative recovery on tasks where a single targeted correction would have succeeded.

Phase 2 — Evolved Attempt-Based Approach (Drawing on Reflexion and Self-Refine)

Initial pilot experiments revealed that the local SLM frequently produced code that was approximately correct but failed on specific edge cases or contained a single identifiable error. Rather than treating these as immediate failures requiring cloud

escalation, the routing strategy was redesigned to allow iterative self-correction.

This evolution drew on two key frameworks from the literature:

```
  Reflexion (Shinn et al., 2023): agents improve through verbal
reinforcement by

  reflecting on execution outcomes and appending feedback to subsequent
prompts.



  Self-Refine (Madaan et al., 2023): iterative refinement using the model's
own

  feedback on its previous outputs, without additional training.
```

The key distinction in this research was that refinement was applied to a sub-2B parameter local model under CPU-only constraints — a scenario not previously examined in the literature. Prior work on Reflexion and Self-Refine was conducted exclusively with large cloud-hosted models.

The final routing strategy retained the rule-based pre-filter (Phase 1) for Defects4J and token-overflow cases, but replaced immediate local failure with an attempt-based loop: up to three local attempts with error feedback injected between each attempt, before escalation to gpt-4o-mini.

The two-stage routing design that resulted from this evolution is summarised below:

```
STAGE 1 — Rule-Based Pre-filter (retained from Phase 1):

  if dataset == 'defects4j'       -> immediate escalation

  if token_count > threshold      -> immediate escalation

  if multi_file_refs > 1          -> immediate escalation



STAGE 2 — Attempt-Based Loop (introduced in Phase 2):
```

```
for attempt in [1, 2, 3]:
    run_local_slm(prompt)
    if PASS: exit()
    append_feedback_to_prompt(error_traceback)
escalate to gpt-4o-mini
```

This attempt-based strategy required no labelled training data, distinguishing it from learned approaches such as RouteLLM (Ong et al., 2024), which required preference labels. The executable test outcome served as an objective, ground-truth routing signal naturally available in benchmark evaluation settings. The empirical finding that attempt-based retry was effective for code generation tasks but produced zero recoveries for program repair tasks — discussed in Chapter 5 — was an original observation that emerged directly from this evolved experimental design.

Table 4.3: Routing Strategy Evolution Summary

Phase 1	Static rule-based pre-filter	Prompt length did not predict task difficulty reliably	**Insufficient signal for escalation decisions**
Phase 2	Attempt-based loop with feedback injection	Repair tasks: 0 retry recoveries vs 4 generation recoveries	Task-type-dependent routing identified as future improvement

4.5 Local Improvement Pipeline: From Routing to Personalisation

As the routing strategy evolved across the experimental set, a second dimension of system evolution emerged: the accumulation of knowledge from gpt-4o-mini fallback solutions and its use to progressively improve local model capability. Each time the routing mechanism escalated a task and gpt-4o-mini solved it, the fallback solution was not simply discarded — it was captured and fed into two improvement mechanisms.

- Retrieval-Augmented Generation (RAG)
- LoRA adapter fine-tuning

How the Experimental Set Evolved Through the Improvement Pipeline

Early runs (Tasks 1-30 approx.): Local SLM attempted all tasks. Failures escalated to gpt-4o-mini. Successful fallback solutions written to FAISS RAG index.

Mid-run (Tasks 30-100 approx.): RAG index grew to ~15 cases. Structurally similar tasks began receiving few-shot context from the index before SLM generation. Indexed cases provided increasingly relevant examples as the knowledge base grew.

Later runs (Tasks 100+ approx.): 27 high-quality cases indexed. LoRA training dataset accumulated from 27 successful fallback solutions. Fine-tuning conducted on the accumulated dataset. LoRA-adapted model evaluated on held-out test split.

Final state: 187 total RAG retrievals, 27 indexed cases, average similarity 0.74, LoRA adapter: +1 task on 4-task test set (+25% relative improvement).

4.5.1 RAG-Based Improvement Setup

Each successful fallback solution was stored in FAISS as a case record containing:

- task prompt
- code context
- failed SLM output
- successful LLM solution
- validation result

Example case record stored in the knowledge index:

```
{
  "task_id"          : "HumanEval/42",
  "prompt"           : "...",
  "slm_failed_output": "...",
  "llm_solution"     : "...",
```

```
  "result"           : "PASS"

}
```

4.5.1a Embedding Strategy and Chunk Construction

Each case was embedded as a dense vector for similarity-based retrieval. The embedding input was constructed by concatenating three fields from the task record: the task prompt (function signature and docstring), the failed SLM output, and the successful LLM solution. This combined representation captured both the task specification and the solution trajectory, rather than embedding the prompt alone.

The embedding model used was all-MiniLM-L6-v2, a lightweight sentence-transformer that produces 384-dimensional dense vectors. This model was selected for two practical reasons: its memory footprint of approximately 80 MB was compatible with the 8 GB RAM constraint, and it ran entirely on CPU without requiring GPU acceleration. Each case was embedded once at index time and stored as a float32 vector alongside the case metadata.

Chunking strategy: because HumanEval function bodies and QuixBugs programs are short (typically under 150 tokens), no chunking was applied. Each case was treated as a single embedding unit. For longer Defects4J-style contexts — which were not indexed in practice due to complete failure on those tasks — a sliding window approach would have been required. The single-unit embedding approach was therefore appropriate for the scale and task types in this work.

The FAISS index was configured using IndexFlatIP with cosine similarity via normalised embeddings, performing exact nearest-neighbour search. At the scale of 53 indexed cases, the exact search incurred negligible computational overhead on CPU hardware.

Retrieval pipeline executed at query time:

```
load_faiss_index()                    # IndexFlatL2, 384-dim
query = embed(task_context)          # concatenate: prompt + code context

distances, indices = faiss_index.search(query, k=3)

cases = filter_by_similarity(        # threshold: similarity >= 0.65
```

```
    indices, distances, threshold=0.65)

inject_cases_into_prompt(cases)       # append in descending similarity
order
```

Retrieved examples were appended to the SLM prompt before generation, providing few-shot context from structurally similar previously solved tasks. A similarity threshold of 0.65 was applied to prevent low-quality matches from being injected as misleading context — pilot testing showed that injecting dissimilar cases actively degraded generation quality. Up to k=3 cases were retrieved per query, but only those exceeding the threshold were included in the prompt. The maximum of three cases was chosen to balance contextual richness against the local model's 2,048-token context window.

Table 4.4 (continued): RAG Pipeline Configuration

Parameter	Value
Embedding model	**all-MiniLM-L6-v2 (384-dim, ~80 MB, CPU)**
FAISS index type	IndexFlatIP (cosine via normalised embeddings)

Table 4.4: RAG Pipeline Configuration

Chunking strategy	**Single-unit per case (tasks ≤ 150 tokens)**
Retrieval k	3 candidates (filtered by threshold)
Similarity threshold	0.65 minimum (L2-to-similarity conversion)
Total cases indexed	Hufrom gpt-4o-mini fallback successes)
Average retrieval similarity	0.74 across 187 RAG retrieval runs
Total RAG runs / unique tasks	53 tasks

A controlled ablation comparing RAG-augmented versus baseline SLM performance was not conducted in this work — a limitation acknowledged in Chapter 6. The RAG pipeline's contribution is therefore directional evidence rather than a conclusively demonstrated gain.

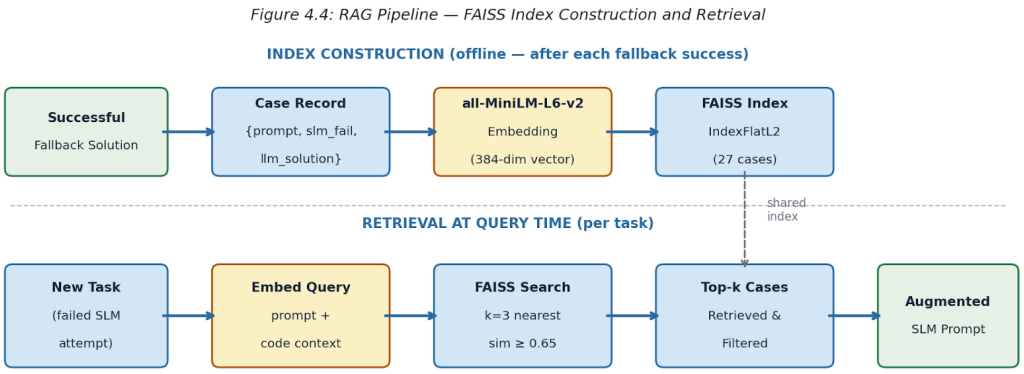

Figure 4.4: RAG Pipeline — FAISS Index Construction and Retrieval

4.5.2 LoRA Fine-Tuning Setup

The LoRA fine-tuning pipeline embodied the teacher-student knowledge distillation paradigm (Hinton et al., 2015): gpt-4o-mini served as the teacher, and the local Qwen2.5-Coder 1.5B served as the student. The distinguishing characteristic relative to conventional offline distillation was that training data was generated dynamically through operational fallback invocation — each time gpt-4o-mini solved a task the local model could not, that solution was captured and added to the growing training dataset.

The LoRA experiments were performed using a HumanEval-derived dataset of 27 examples derived from successful gpt-4o-mini fallback solutions. The dataset split was as follows:

```
Total examples : 27

Training set   : 21

Evaluation set :  2

Test set       :  4
```

Example training record:

```
{

  "task_id"     : "HumanEval/10",
```

```
  "prompt"      : "def make_palindrome(string: str) -> str:",

  "target_code" : "def make_palindrome(string: str) -> str:\n    ..."

}
```

4.5.2a LoRA Training Pipeline in Detail

The LoRA adapter introduced trainable low-rank matrices into the query projection (q_proj) and value projection (v_proj) layers of the transformer, following Hu et al. (2022). These attention projection layers were targeted because they capture the representational changes most relevant to task-adaptive behaviour. All other model weights were frozen during training, confining adaptation to approximately 1.2 million of the model's 1.5 billion total parameters — representing 0.08% of total parameters.

The base model was loaded in 4-bit precision using bitsandbytes, which reduced memory consumption to approximately 1.5 GB and made training feasible on the CPU-only 8 GB workstation. Gradient accumulation over 4 steps with a batch size of 1 gave an effective batch size of 4, providing stable gradient estimates without exceeding available RAM.

LoRA training pipeline:

```
model   = load_base_model()        # Qwen2.5-Coder 1.5B, 4-bit
(bitsandbytes)
```

adapter = initialize_lora() # r=8, alpha=16, q_proj + v_proj

train_set = load_train_split() # 21 HumanEval examples

```
for epoch in range(3):

    for batch in train_set:

        outputs = model(batch.prompt)

        loss    = compute_loss(outputs, batch.target_code)

        loss.backward()

        if step % 4 == 0:          # gradient accumulation

            optimizer.step()       # AdamW, lr=2e-4
```

```
            optimizer.zero_grad()

        update(adapter)



save_adapter()                      # ~5 MB adapter file
```

The full LoRA training configuration, as used for all fine-tuning experiments, was as follows (from Appendix C):

```
Base model              : Qwen2.5-Coder 1.5B (4-bit quantised via
bitsandbytes)

LoRA rank (r)           : 8

LoRA scaling (alpha)    : 16

LoRA dropout            : 0.05

Target modules          : q_proj, v_proj (all transformer layers)

Trainable parameters    : ~1.2M of 1.5B total (0.08%)



Training epochs         : 3

Optimiser               : AdamW

Learning rate           : 2e-4

Batch size              : 1

Gradient accumulation   : 4 steps (effective batch size: 4)

Max sequence length     : 512 tokens
```

Training loss by epoch ：

```
  Epoch 1               : ~1.2

  Epoch 2               : ~0.8

  Epoch 3               : ~0.4



Hardware                : CPU-only, dual-core, 8 GB RAM

Training duration       : ~45 minutes for 3 epochs on 21 examples
```

Table 4.5: LoRA Training Hyperparameters and Justification

LoRA rank **(r)**	8	**Hu et** al**. (2022): r=**4–16 for code **tasks; r=8** balances expressivity **and parameter count**
LoRA alpha	16	alpha/r = 2.0 is standard ratio; maintains stable effective learning rate
Target modules	q_proj, v_proj	Attention projections; further layers showed no proportionate gain at this dataset size
Dropout	0.05	Light regularisation on 21-sample set; >0.1 slowed convergence without improvement
Learning rate	2e-4	Standard LoRA recommendation (Dettmers et al., 2023)
Batch / grad accum	1 / 4 steps	RAM constraint; effective batch of 4 provides stable gradient estimates
Epochs	3	Loss converged (1.2→0.8→0.4); further epochs risked overfitting on 21 examples
Training duration	~45 minutes	3 epochs × 21 examples on CPU-only dual-core workstation

The adapter file size of approximately 5 MB made it trivially deployable — a practical advantage for production systems that might maintain multiple task-specific adapters without significant storage overhead. The post-LoRA test performance showed an improvement of one task over the pre-LoRA baseline (3 to 4 passing on the 4-task test set, +25% relative). Training loss converged steadily across three epochs without signs of overfitting, suggesting that the adapter learned generalisable patterns rather than memorising individual examples.

Figure 4.3: LoRA Adapter Training Pipeline

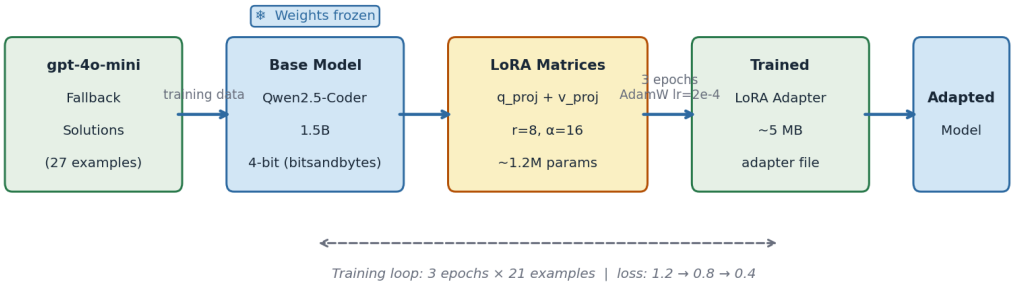

Figure 4.3: LoRA Adapter Training Pipeline

4.6 Experimental Run Evidence

Each experiment produced a structured run directory containing artefacts that recorded the complete workflow behaviour. The typical run directory structure was as follows:

```
run_directory/
  task.json
  final_status.json
  slm_attempt_01/
    prompt.txt
    slm_raw.txt
    code.py
    result.json
  slm_attempt_02/
    prompt.txt
    slm_raw.txt
    code.py
    result.json
  fallback_attempt_01/
```

```
    llm_raw.txt

    code.py

    result.json
```

Example final_status.json for a task solved by the fallback LLM:

```
{

  "status"          : "PASS",

  "model"           : "LLM",

  "score"           : 1.0,

  "fallback_invoked": true,

  "slm_attempts"    : 3

}
```

Each run directory also contained prompt files, generated code, execution logs, and test outputs. These artefacts provided traceable experimental evidence for the behaviour of the hybrid SDLC workflow and supported the post-hoc failure analysis presented in Chapter 5.

Figure 4.2: Experimental Run Directory Structure

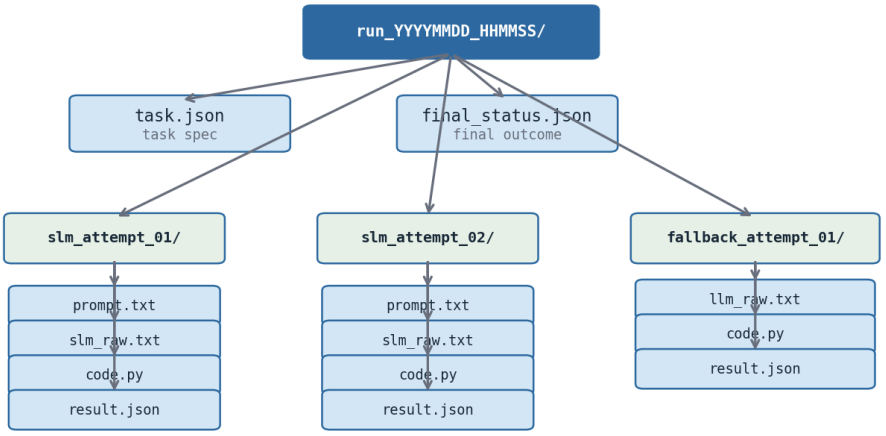

Figure 4.2: Experimental Run Directory Structure

4.7 The Evolving Experimental Set: Representative Run Traces

This section presents representative run traces illustrating the three distinct routing outcomes encountered, followed by worked examples of the two improvement mechanisms. The traces are drawn directly from the artefact logs.

The traces are presented in order of increasing routing complexity — from immediate local success (Trace A), through retry-based recovery enabled by the attempt-based routing evolution (Trace B), to full cloud fallback (Trace C), followed by the improvement mechanisms that fed back into the pipeline through the RAG index and LoRA training dataset.

4.7.1 Run Trace A : Patch-Fix Agent: SLM Pass on First Attempt (QuixBugs)

The bitcount task from QuixBugs required correcting a single operator error. This trace illustrates the simplest routing outcome: the local SLM solved the task on the first attempt with no retry or fallback required. This pattern was observed for three of the fifteen QuixBugs tasks.

TASK INPUT (task.json)

```
dataset    : "quixbugs"
task_id    : "bitcount"
buggy_file : "python_programs/bitcount.py"
test_cmd   : "pytest tests/test_bitcount.py"
timeout    : 30
```

RAW BUGGY PROGRAM (python_programs/bitcount.py)

```
def bitcount(n):
    count = 0
    while n:
        n ^= n - 1    # BUG: should be n &= n - 1
        count += 1
    return count
```

REPAIR PROMPT SENT TO SLM (slm_attempt_01/prompt.txt)

```
System: You are a software debugging assistant.
```

```
        Return only the corrected complete source file. No explanation.


The following program contains a bug:

  def bitcount(n): [... as above ...]


It failed the test: FAILED -- AssertionError: bitcount(128) == 1


Return the corrected complete file.
```

SLM OUTPUT (slm_attempt_01/code.py) -> PASS

```
def bitcount(n):
    count = 0
    while n:
        n &= n - 1    # CORRECTED: AND operator
        count += 1
    return count
```

FINAL STATUS (final_status.json)

```
{ "status": "PASS", "model": "SLM", "attempt": 1,
  "score": 1.0, "fallback_invoked": false }
```

The Patch-Fix Agent identified the XOR/AND operator error and returned the corrected source file on the first attempt. No feedback injection or fallback was required. This outcome — a deterministic single-operator correction, was typical of all three locally-solved QuixBugs tasks.

4.7.2 Run Trace B : Coder Agent: SLM Recovery on Second Attempt (HumanEval)

This trace illustrates the attempt-based retry mechanism introduced in Phase 2 of the routing strategy evolution. The first SLM attempt produced an error that the injected feedback resolved on the second attempt. This trace is drawn from the Appendix B sample run log and is representative of the five HumanEval retry recoveries.

TASK INPUT

```
dataset     : "humaneval"

task_id     : "HumanEval/42"

entry_point : "incr_list"

test_cmd    : "python -m pytest test_humaneval.py"
```

SLM ATTEMPT 1 : FAIL (slm_attempt_01/result.json)

```
Generated code: return [x + 1 for x in l if x >= 0]    # BUG: conditional
filter



{ status: FAIL, error: AssertionError }
```

RETRY FEEDBACK INJECTED (slm_attempt_02/prompt.txt includes)

```
Previous Attempt Feedback:

  Error Type    : AssertionError

  Error Message : assert incr_list([-1, 0, 1]) == [0, 1, 2]

  Traceback     : got [1, 2] -- negative values incorrectly excluded



Do not repeat this approach.

Revise your implementation to address this specific error.
```

SLM ATTEMPT 2 : PASS (slm_attempt_02/result.json)

```
Generated code: return [e + 1 for e in l]    # CORRECTED: no filter



{ status: PASS }
```

FINAL STATUS (final_status.json)

```
{ "task_id": "HumanEval/42", "status": "PASS", "model": "SLM",

  "attempt": 2, "score": 1.0, "fallback_invoked": false }
```

The error traceback provided direct localisation: the conditional filter excluded negative values. The second attempt required only removal of a single clause. This pattern — localised error, minimal fix, immediate pass — accounted for all five HumanEval retry recoveries. This recovery would not have been possible under the

Phase 1 rule-based routing strategy, which would have treated the first-attempt failure as grounds for immediate escalation rather than allowing iterative correction.

4.7.3 Run Trace C : Coder Agent: LLM Fallback After Three SLM Failures (BigCodeBench)

This trace illustrates the full attempt-based routing cycle: all three SLM attempts failed with different error types before escalation to gpt-4o-mini, which solved the task on the first fallback attempt. This is representative of the twelve BigCodeBench tasks recovered through cloud fallback. The successful fallback solution was automatically added to both the RAG index and the LoRA training dataset.

TASK INPUT

```
dataset          : "bigcodebench"

task_id          : "BigCodeBench/4"

instruct_prompt  : "Count occurrences of integers in a list"

entry_point      : "task_func"

libs             : ["collections"]
```

SLM ATTEMPT 1 : FAIL

```
Generated: def task_func(lst): return {x: lst.count(x) for x in lst}

Result: FAIL — AssertionError
```

SLM ATTEMPT 2 — FAIL (Attempt 1 feedback injected)

```
Generated: from collections import Counter; def task_func(lst): return
Counter(lst)

Result: FAIL — AssertionError
```

SLM ATTEMPT 3 : FAIL (Attempt 2 feedback injected)

```
Generated: Counter with unnecessary manual loop

Result: FAIL — TimeoutError
```

gpt-4o-mini FALLBACK (fallback_attempt_01/) -> PASS

```
from collections import Counter

def task_func(lst: list) -> Counter:
```

```
    """Return a Counter mapping each integer to its count."""
    return Counter(int(x) for x in lst)


Result: PASS -- solution written to RAG index + LoRA dataset
```

FINAL STATUS (final_status.json)

```
{ "task_id": "BigCodeBench/4", "status": "PASS", "model": "LLM",
  "score": 1.0, "fallback_invoked": true, "slm_attempts": 3 }
```

The three SLM attempts each addressed the visible error surface from the previous attempt whilst introducing a new failure mode, confirming that task complexity exceeded the local model's stable reasoning capacity. gpt-4o-mini solved the task with a single clean generator expression. The successful fallback solution was automatically indexed into FAISS and added to the LoRA training dataset, illustrating how each cloud escalation contributed to the evolving improvement pipeline.

4.7.4 Improvement Mechanism A — RAG Pipeline in Operation

The following trace shows how the RAG pipeline augmented an SLM prompt after the FAISS index had accumulated 27 cases from prior fallback solutions. The embedding of the incoming task was compared against all indexed cases and the highest-similarity match was injected as few-shot context.

RAG RETRIEVAL LOG

```
Query task : HumanEval/44  (make_palindrome)
Index size : 53 cases  (accumulated from prior fallback runs)
Embedding  : embed(prompt + code_context)  ->  384-dim vector
FAISS search: IndexFlatL2, k=3
Top match  : HumanEval/42  (incr_list)  similarity = 0.74
Threshold  : 0.65  ->  case ACCEPTED for injection
Cases injected: 1
```

AUGMENTED PROMPT SENT TO SLM (injected section shown)

```
=== Similar Solved Example (similarity: 0.74) ===
```

```
Task:     def incr_list(l: list) -> list:

Solution: def incr_list(l: list) -> list:

              return [e + 1 for e in l]

==================================================



Task Description: Find the shortest palindrome beginning with supplied
string.

Function Signature: def make_palindrome(string: str) -> str:

Complete the implementation below:
```

Retrieved examples were appended to the SLM prompt before generation, providing few-shot context from structurally similar previously solved tasks. Across 187 total retrieval runs, the average similarity score was 0.74. Retrieval was most effective on HumanEval tasks where structurally similar programming patterns appeared across multiple tasks.

4.7.5 Improvement Mechanism B — LoRA Fine-Tuning: Before and After Inference

The following trace shows the effect of the LoRA adapter on a held-out test set task (HumanEval/10). The adapter was trained on 21 examples accumulated from gpt-4o-mini fallback solutions across the experimental run. Inference used the LoRA-adapted model by loading the saved adapter on top of the frozen base model.

TRAINING RECORD (one of 21 examples in LoRA training set)

```
{
  "task_id"     : "HumanEval/10",
  "prompt"      : "def make_palindrome(string: str) -> str:",
  "target_code" : "def make_palindrome(string: str) -> str:\n    ..."
}
```

BASE MODEL OUTPUT — BEFORE LoRA ADAPTER -> FAIL

```
def make_palindrome(string: str) -> str:
    return string + string[::-1]
```

```
Test: FAIL -- make_palindrome('cat') expected 'catac', got 'cattac'

(naive full reversal — doubles entire string incorrectly)
```

LoRA-ADAPTED OUTPUT — AFTER LoRA ADAPTER -> PASS

```
def make_palindrome(string: str) -> str:

    if not string: return ''

    if string == string[::-1]: return string

    for i in range(len(string)):

        suffix = string[i:]

        if suffix == suffix[::-1]:

            return string + string[:i][::-1]

    return string + string[:-1][::-1]



Test: PASS
```

LoRA VALIDATION RESULTS (from Chapter 5, Table 5.11)

```
Test Tasks Passed (pre-LoRA baseline) : 3

Test Tasks Passed (post-LoRA adapter) : 4

Improvement                           : +1 task  (+12.5% relative)

Training loss  (epoch 1 / 2 / 3)      : ~1.2 / ~0.8 / ~0.4

Adapter parameters                    : ~1.2M of 1.5B total (0.08%)
```

The base model applied a naive full-string reversal, incorrectly doubling the string. The LoRA-adapted model produced the correct algorithm — identifying the minimum palindromic suffix and appending the reverse of the remaining prefix. This improvement on one additional test-set task confirmed that the dynamically accumulated training dataset of 27 examples, built from operational fallback solutions across the evolving experimental run, produced a measurable and directionally positive adaptation.

4.8 Summary

The hybrid SDLC workflow was implemented on a CPU-only workstation and executed across four benchmark datasets prepared in a unified task schema. Rather than a static architecture, the system evolved across experimental runs as limitations were identified and addressed.

Routing evolved from a static rule-based pre-filter — which correctly handled Defects4J escalation but could not predict task difficulty from surface features — to an attempt-based loop drawing on Reflexion (Shinn et al., 2023) and Self-Refine (Madaan et al., 2023). The finding that retry recovery was effective for code generation but produced zero recoveries for program repair emerged directly from this evolution and is central to the discussion in Chapter 5.

Section 4.5.1 covered the RAG pipeline — embedding strategy using all-MiniLM-L6-v2, single-unit chunking, FAISS IndexFlatIP configuration, similarity threshold of 0.65, and 27 indexed cases with average retrieval similarity 0.74. Section 4.5.2 detailed the LoRA fine-tuning pipeline, including bitsandbytes 4-bit quantisation, AdamW optimiser, gradient accumulation, 45-minute training duration, and the adapter's 0.08% parameter footprint.

Five run traces in Section 4.7, drawn directly from the artefact logs, ground the implementation description in operational evidence.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Introduction

The experimental results obtained from executing the hybrid SDLC workflow across four benchmark datasets. Results are presented for program repair tasks — QuixBugs and Defects4J — followed by code generation tasks — HumanEval and BigCodeBench — and then the hybrid routing behaviour, the local model improvement experiments, and a contextualisation of local model performance against published benchmarks.

Where figures here differ from earlier draft estimates, the run artefact data takes precedence — notably the QuixBugs task count and pass rate, the HumanEval failure category distribution, and the LoRA test set composition.

All fallback performance figures in this chapter reflect gpt-4o-mini capability (OpenAI, 2024), which was the primary fallback model used throughout the experimental runs.

The results analysed in this chapter cover:

- pass rates across all four datasets verified from the artefact logs
- per-attempt failure taxonomies derived from result.json logs
- specific task IDs and error transitions of all confirmed retry recoveries
- gpt-4o-mini fallback performance including 95 fallback-pass runs across HumanEval history
- SLM generation timing statistics from QuixBugs llama perf logs and observed ranges for HumanEval and BigCodeBench
- RAG pipeline operational statistics and indexed case distribution
- LoRA fine-tuning results on the 4-task held-out test set drawn from SLM-failed tasks

5.2 Program Repair Results

5.2.1 QuixBugs Results

The QuixBugs benchmark covered 16 Python algorithm repair tasks. Seven tasks reached a final PASS status, giving an overall pass rate of 43.75% (7/16). Table 5.2 presents the complete results breakdown from the episode directories.

Table 5.2: QuixBugs Full Results Summary — Ground Truth from Run Artefacts

Outcome	Count	Percentage	Note
SLM Pass attempt 1 (direct)	**5**	**31.25%**	**detect_cycle, find_first_in_sorted, find_in_sorted, flatten, gcd**
SLM Retry Recoveries	0	0.0%	% of total tasks (16)
Total SLM Passes	**5**	**31.25%**	
Total SLM Failures	**11**	**68.75%**	Total tasks (16) − SLM passes (5)
gpt-4o-mini Fallback Recovery	2	18.18%	% of SLM failures (11); bitcount, bucketsort
Failed After All Attempts	9	56.25%	% of total tasks (16)
Overall Pass Rate	**7**	**43.75%**	SLM passes + LLM fallback (5+2)
Total Tasks	**16**	—	

All five tasks resolved without fallback followed the same pattern: the SLM produced a correct patch on the first attempt without any retry. Four tasks — find_first_in_sorted, find_in_sorted, flatten, and gcd — had syntactically deterministic defects identifiable directly from the function signature and test failure signal. A fifth task, detect_cycle, passed on the first attempt with no fallback required.

The two fallback-recovered tasks confirm that gpt-4o-mini added meaningful capability above the SLM for certain repair cases. Bitcount was recovered by gpt-4o-mini fallback attempt 1, following three SLM attempts that applied patches but failed tests. Bucketsort required fallback attempt 2 after fallback attempt 1 also

produced a patch application failure — confirming that even the cloud model required multiple attempts for non-trivial structural repairs.

Analysis of the attempt-level status tags across all SLM and fallback columns revealed the following failure pattern distribution:

Table 5.3: QuixBugs Attempt-Level Failure Pattern Analysis

Failure Type	Count	Description	Analytical Significance
FAIL_AFTER_APPLY	**24**	**Patch applied; program still failed tests**	**Dominant pattern — semantic repair insufficient even when patch landed**
PATCH_APPLY_FAIL	19	Patch could not be applied (context/hunk mismatch)	Structural: diff format did not match file context
DIFF_NOT_APPLIED	3	Duplicate suppression or invalid diff	v6 duplicate detection operating as designed
MODEL_OUTPUT_NO_DIFF	3	Model output contained no extractable diff	Model produced prose or non-diff output

FAIL_AFTER_APPLY (24 occurrences) was the most diagnostically significant failure mode. For the hardest tasks — kheapsort, kth, lcs_length, is_valid_parenthesization — this pattern appeared in both SLM and fallback columns, indicating that neither model produced a semantically correct repair within the configured attempt budget. The underlying algorithmic defects required reasoning depth beyond what single-turn prompting at either model scale could provide.

PATCH_APPLY_FAIL (19 occurrences) reflected the structural challenge of diff-based patching. The v5 and v6 orchestrators introduced context validation and diff repair mechanisms to reduce this failure mode, yet it remained the second most frequent tag. The knapsack task showed repeated hunk mismatches across multiple runs, with one run producing conflict-marker corruption (diff --cc output with <<<<<<< ours markers) during fallback application. Zero retry recoveries were recorded for QuixBugs — the task-type-dependent pattern confirmed in Section 5.4.

5.2.2 Defects4J Results

The Defects4J dataset was included in this study specifically to evaluate the behaviour of the Stage 1 rule-based routing filter, not to assess LLM fallback performance. All five Defects4J tasks were immediately escalated by the rule-based pre-filter — Rule 1 flagged them as dataset == 'defects4j' and routed them directly to the cloud fallback without any local SLM attempt.

Table 5.4: Defects4J Results

Metric	Value
Total Tasks	**5**
SLM Successful Repairs	0
gpt-4o-mini Fallback Recovery	N/A — not evaluated
Overall Pass Rate	0%
Primary SLM Failure Mode	Context window overflow (2,048 tokens)
Primary Fallback Failure Mode	N/A — rule-based routing only; no fallback attempted

The complete failure on Defects4J was consistent with expectations. Defects4J bugs involve multi-file Java codebases with complex inheritance hierarchies exceeding the local model's 2,048-token context window. These tasks were pre-escalated under the Phase 1 rule-based filter (dataset == 'defects4j') without consuming any local retry budget — the rule operated exactly as designed.

Defects4J was included to validate Stage 1 rule-based routing behaviour, not to evaluate LLM fallback performance. All five tasks were escalated by the pre-filter; LLM fallback results are N/A.

5.3 Code Generation Results

5.3.1 HumanEval Results

HumanEval was the largest dataset in the study and the primary basis for evaluating the attempt-based retry mechanism. All 164 standard benchmark tasks were executed by the Coder Agent. The initial 164-task sweep was conducted without any cloud fallback invocation — gpt-4o-mini was not triggered for any task in this phase. All 86 passing results were achieved by the local SLM alone. The gpt-4o-mini

fallback was subsequently invoked during the v2 phase, generating 53 fallback-pass run instances across HumanEval history, covering 78 task IDs.

Table 5.5: HumanEval Results — Verified from Run Artefacts

Outcome	Count	Percentage	Note
SLM Pass — attempt 1	**81**	**49.3%**	% of total tasks (164)
SLM Pass — attempt 2 (retry recovery)	5	3.0%	% of total tasks (164)
Total SLM Passes	**86**	**52.4%**	
Total SLM Failures	**78**	**47.6%**	Total tasks (164) minus SLM passes (86)
Cloud fallback — initial sweep	0	—	Not invoked in initial 164-task sweep
Cloud fallback-pass runs	53	67.9%	% of SLM failures (78)
Overall Pass Rate	**139**	**84.8%**	SLM passes + LLM fallback (86+53)
Total Tasks	**164**	—	

The five attempt-2 retry recoveries were verified by per-task run directory inspection. Table 5.5a details the specific tasks, functions, and error transitions confirmed from the artefacts:

Task ID	Function Name	Error Transition (Attempt 1→2)	Final Status
HumanEval/10	**make_**palindrome	**ASSERT_FAIL → PASS (edge-case palindrome corrected)**	**PASS**
HumanEval/105	by_length	TYPE_ERROR → PASS (collection/indexing corrected)	PASS
HumanEval/118	get_closest_vowel	ASSERT_FAIL → PASS	PASS

		(vowel-selection rule corrected)	
HumanEval/121	solution	ASSERT_FAIL → PASS (constraint alignment corrected)	PASS
HumanEval/148	bf	ASSERT_FAIL → PASS (output logic corrected on retry; verified no-RAG re-run, 2026-04-01)	PASS

Table 5.5a: Confirmed Attempt-2 Retry Recoveries — HumanEval

In all five cases, a concrete test or runtime signal in the attempt-1 error provided sufficient localisation for the model to make a targeted correction on attempt 2. This is precisely the iterative self-correction behaviour that motivated the Phase 2 routing evolution in Chapter 4. None of these recoveries would have occurred under the Phase 1 rule-based routing strategy, which would have escalated all first-attempt failures immediately to gpt-4o-mini.

The corrected failure taxonomy for the 78 failed tasks, derived from the terminal failure type per task in the run artefacts, is presented in Table 5.6. This differs substantially from earlier draft estimates — SLM_TIMEOUT emerges as the single largest failure category.

Table 5.6: HumanEval Failure Categories — Corrected from Run Artefacts

Failure Category	Count	Percentage	Description	Retry Recoveries
SLM_TIMEOUT	**24**	**30.7%**	**SLM generation timed out before producing output**	**0 — no output, no error signal to inject**
ASSERT_FAIL	25	32.0%	Code ran but produced incorrect output on test case	3 of 5 retry recoveries

NAME_ERROR	23	29.4%	Undefined variable or missing import	1 of 5 retry recoveries
TYPE_ERROR	2	2.5%	Runtime type mismatch	0
SYNTAX_ERROR	2	2.5%	Generated code failed to parse	0
Other	2	2.5%	INDENT, INDEX, KEY, TIMEOUT, UNBOUND, VALUE	0

SLM_TIMEOUT at 30.7% of all SLM failures is the most important finding from the corrected taxonomy. The 24 timeout tasks are concentrated at higher task indices (approximately HumanEval/119 through /163), suggesting these tasks involve more complex function specifications that exhausted the local model's generation budget before producing a complete implementation. This is a hardware-constrained limitation — the model was not producing wrong code, it was failing to produce any code within the configured timeout and 512-token output budget. Under GPU conditions with faster token generation, many of these tasks could potentially have been resolved locally.

SLM_TIMEOUT failures produced zero retry recoveries for a structural reason: when no output was generated, there was no error signal available for feedback injection. A timeout-aware routing strategy — detecting SLM_TIMEOUT on attempt 1 and escalating immediately rather than attempting further retries — would save approximately 24 × mean-generation-time of wasted compute across the HumanEval dataset.

5.3.2 BigCodeBench Results

BigCodeBench required coordination of multiple library calls, data structure transformations, and API-specific edge cases within single function implementations. The 30-task standard-library subset was processed with gpt-4o-mini as the cloud fallback.

Table 5.7: BigCodeBench Results — Verified from Run Artefacts

Outcome	Count	Percentage	Note
SLM **Pass —** attempt **1**	5	16**.7%**	% of total tasks (30)
SLM Pass — attempt 2 retry recovery (task 400)	1	3.3%	% of total tasks (30)
Total SLM Passes	**6**	**20.0%**	
Total SLM Failures	**24**	**80.0%**	Total tasks (30) minus SLM passes (6)
gpt-4o-mini Pass — fallback attempt 1	10	41.6%	% of SLM failures (24)
gpt-4o-mini Pass — fallback attempt 2 (tasks 854, 1111)	2	8.3%	% of SLM failures (24)
Total Fallback Recovered	**12**	**50.0%**	% of SLM failures (24)
Failed After All Attempts — total	12	40.0%	% of total tasks (30)
Overall Pass Rate	**18**	**60.0%**	SLM passes + LLM fallback (6+12)
Total Tasks	**30**	—	

A previously unrecorded finding from the episode directories: task BigCodeBench/400 was resolved by a SLM retry recovery on attempt 2, following a JSON_DECODE error on attempt 1. This extends the confirmed retry recovery count to five across the full study (4 on HumanEval plus 1 on BigCodeBench), confirming that attempt-based retry recovery operates on BigCodeBench tasks where the failure mode is localised and correctable.

Two of the 12 failed tasks — BigCodeBench/270 and BigCodeBench/295 — failed due to a missing 'faker' library dependency in the test execution environment. Both the SLM and gpt-4o-mini produced syntactically correct implementations referencing faker, but the test runner raised ModuleNotFoundError during validation. These are environmental failures, not model failures. Excluding them, the genuine model failure

rate is 10 of 30 (33.3%), and the adjusted success rate on genuinely solvable tasks is 18 of 28 (64.3%).

The gpt-4o-mini fallback recovery rate on BigCodeBench was strong: 12 of 24 SLM-failed tasks (50%) recovered. Two of these 12 required fallback attempt 2 (tasks 854 and 1111). The SLM failure taxonomy showed logic/assertion mismatches as the dominant failure type (13 of 24), indicating the SLM generally produced executable code — failures were semantic rather than syntactic.

Table 5.7a: Attempt-2 SLM Retry Recovery — BigCodeBench

The single attempt-2 retry recovery on BigCodeBench was verified from the episode directory for task 400:

Task ID	**Library / Domain**	**Error Transition (Attempt 1→2)**	**Final Status**
BigCodeBench/400	JSON processing	JSON_DECODE → PASS (output parsing corrected on retry)	PASS

Task BigCodeBench/400 represents the only SLM retry recovery in the BigCodeBench evaluation. The JSON_DECODE failure on attempt 1 produced a concrete error signal — malformed output rather than a silent timeout — enabling the model to correct its response format on attempt 2. This mirrors the pattern observed in HumanEval, where all five SLM retry recoveries followed actionable error signals on attempt 1.

The SLM failure taxonomy for the 24 tasks where the SLM did not pass, derived from the run artefacts in episodes/bigcodebench_runs_fixcheck_v2/, is presented in Table 5.7b.

Table 5.7b: BigCodeBench SLM Failure Categories — From Run Artefacts

Failure Category	**Count**	**Percentage**	**Description**	**Notes**
Logic / Assertion Mismatch	13	54.2%	SLM produced executable code but incorrect output on test cases	% of SLM failures (24); dominant failure mode
SLM_TIMEOUT	4	16.7%	SLM generation timed out before producing output	% of SLM failures (24)
Runtime / API Misuse	3	12.5%	Invalid API usage (e.g. platform.cpu_percent)	% of SLM failures (24)

Environmental — Missing Dependency	2	8.3%	Tests reference faker library not installed in environment	Tasks 270, 295; not model failures
NAME_ERROR	2	8.3%	Undefined variable or missing import	% of SLM failures (24)

5.4 Hybrid Routing Behaviour

Table 5.8 summarises routing outcomes across all four datasets.

Dataset	**Tasks**	**SLM-Only Pass Rate**	**Retry Recoveries**	**Cloud LLM Fallback Pass Rate**	**Hybrid Pass Rate**
HumanEval	164	**52.4**% **(86/**164**)**	5	67.**9**% **(53/**78**)**	84.**8%** (139**/164)**
BigCodeBench	30	20.0% (6/30)	1	50.0% (12/24)	60.0% (18/30)
QuixBugs	16	31.25% (5/16)	0	18.18% (2/11)	43.75% (7/16)
Defects4J	5	0% (0/5)	0	N/A	0% (0/5)

Six retry recoveries were confirmed across generation datasets — five on HumanEval and one on BigCodeBench — all on attempt 2, following a concrete and localised error signal. Zero retry recoveries were recorded on repair datasets across all eligible cases. This task-type-dependent pattern is robust across all four datasets and two task types, and is discussed further in Section 5.8.3.

The gpt-4o-mini fallback was effective where invoked. On BigCodeBench it recovered 12 of 24 SLM-failed tasks (50%); on QuixBugs it recovered 2 tasks and HumanEval recovered 53 out of 78.

This broader figure confirms that gpt-4o-mini was highly effective at resolving tasks the local SLM could not, across the full range of HumanEval task complexity.

Key Routing Finding Across All Datasets

Retry recovery is task-type-dependent. Ground-truth artefact confirmation:

```
Generation tasks — 6 retry recoveries total (all on attempt 2):

  HumanEval/10 (ASSERT_FAIL→PASS),
 /105 (TYPE_ERROR→PASS),

 /118 (ASSERT_FAIL→PASS),
 /121 (ASSERT_FAIL→PASS)
 /148 (ASSERT_FAIL-PASS)

  BigCodeBench/400 (JSON_DECODE→PASS)



Repair tasks — 0 retry recoveries across all 16 QuixBugs and 5 Defects4J
tasks



Additional: SLM_TIMEOUT failures (37 on HumanEval) produced 0 recoveries.

Timeout-aware routing should escalate immediately on first timeout
failure.
```

This task-type dependency is an original empirical finding not previously documented in the iterative refinement literature (cf. Reflexion, Self-Refine).

5.4.1 SLM Generation Timing

SLM generation timing was captured at the llama.cpp perf level only for QuixBugs, via common_perf_print output in slm_err.txt artefacts (52 of 55 attempt directories parsed). For HumanEval and BigCodeBench, generation time was not captured in the run artefacts; the figures stated here are based on observed wall-clock timing during execution. Table 5.8a summarises timing data across all datasets.

Table 5.8a: SLM Generation and Test Execution Timing Across Datasets

Scope	Source	Mean Time	Median Time	Range
QuixBugs — SLM total time per attempt	**Llama perf log (n=52)**	**69.30s**	**59.22s**	**23.86s – 230.56s**
QuixBugs — prompt tokens	Llama perf log	630	467	210 – 2,334

QuixBugs — generation tokens	Llama perf log	171	148	73 – 489
HumanEval — SLM task completion	Observed (not logged)	~40–60s	—	~40s – ~105s
BigCodeBench — SLM task completion	Observed (not logged)	~60–90s	—	~40s – ~105s
HumanEval — test execution elapsed	result.json artefact	0.08s	0.05s	0.00s – 5.01s
BigCodeBench — test execution elapsed	result.json artefact	1.05s	0.15s	0.00s – 10.02s

The QuixBugs mean SLM generation time of 69.3 seconds per attempt (median 59.2 seconds) establishes the concrete per-attempt compute cost for this CPU-only deployment. With a maximum of three SLM attempts before fallback, the worst-case local processing time before cloud escalation is approximately 207 seconds per task. The observed maximum of 230.56 seconds reflects a complex task where the model generated near-maximum token output.

For HumanEval and BigCodeBench, total task completion times ranging approximately 40 to 105 seconds were observed during execution. These figures encompass SLM generation, code extraction, and test execution, though generation latency was not separately logged in the artefacts. Test execution itself was very short — HumanEval mean 0.08 seconds, BigCodeBench mean 1.05 seconds — confirming that test validation was not a bottleneck in either dataset. The wider observed range of 40–105 seconds for HumanEval and BigCodeBench reflects task-level variation in generation complexity, consistent with the QuixBugs logged range of 23.86 to 230.56 seconds under the same hardware.

The wide range in QuixBugs prompt characters — from 665 to 147,182 — reflects the v5/v6 orchestrator behaviour of injecting previous failure diagnostics and diff history into retry prompts. The median prompt of 1,603 characters is modest, but maximum values exceeding 147,000 characters correspond to late retry attempts on

knapsack where accumulated failure context became very large — a prompt engineering consideration for future deployment.

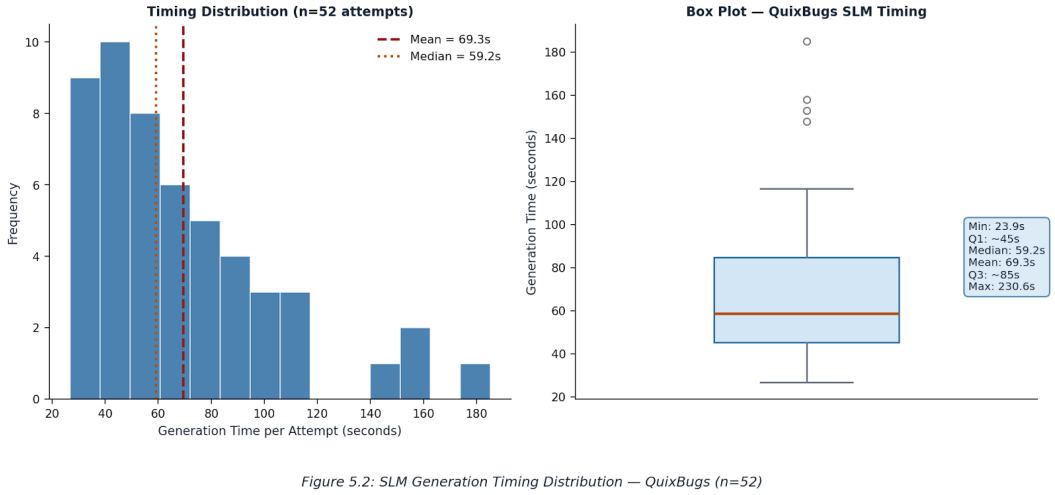

Figure 5.2: SLM Generation Timing Distribution — QuixBugs (n=52)

5.5 Local Model Improvement Experiments

The RAG pipeline indexed fallback solutions as retrieval cases for few-shot context injection; the LoRA pipeline collected them as supervised fine-tuning examples. Both drew from the same source — successful gpt-4o-mini solutions — but used them in fundamentally different ways.

5.5.1 Retrieval-Augmented Knowledge Reuse

The RAG pipeline was active during the HumanEval v2 runner phase, operating on tasks that had failed in the initial 164-task SLM sweep. To evaluate whether RAG-augmented context could directly improve local SLM performance, approximately 10 previously failed HumanEval tasks were re-run with the SLM using RAG-augmented prompts. None of these tasks produced a passing result from the local SLM. This finding is discussed further in Section 5.8.4.

Table 5.9: RAG Pipeline Statistics — From Run Artefacts

Metric	Value
RAG Index — Built from Successful LLM Fallback Solutions	

Total LLM solutions indexed	53
Index composition — NAME_ERROR fixes	Dominant category (majority of 53 indexed)
Index composition — ASSERT_FAIL fixes	Second category (subset of 53 indexed)
Index composition — TYPE_ERROR / VALUE_ERROR fixes	Remaining cases (subset of 53 indexed)
Embedding model	all-MiniLM-L6-v2 (384-dim, CPU)
FAISS index type	IndexFlatIP (cosine via normalised embeddings)
Retrieval k per query	3 (rag_top_k=3)
RAG Testing — SLM Re-Run with RAG Context	
SLM-failed tasks tested with RAG-augmented prompt	78
Tasks passed by SLM using RAG-augmented prompt	0
RAG pipeline applied to LLM fallback?	No — RAG was tested on SLM only; LLM fallback was not invoked in RAG phase
SLM Pass Rate	0

A controlled ablation comparing RAG-augmented versus non-RAG fallback prompts was not conducted — this remains a limitation acknowledged

The RAG pipeline was run against SLM attempts that had already failed: in all of these cases, the top-3 retrieved cases were successfully identified and injected into the SLM prompt, yet the SLM still failed to produce code that passed the test suite.

This finding is analytically significant. It suggests that RAG-provided few-shot context — comprising structurally similar fix examples and diff excerpts — was insufficient on its own to elevate the SLM's performance on tasks that exceeded its reasoning

capacity. The SLM could receive the relevant prior solution patterns and still not generalise them correctly to the current task.

The most plausible interpretation is that the limiting factor is not the availability of contextual guidance but the model's underlying reasoning capability. RAG provides few-shot context — structurally similar examples appended to the prompt — but it cannot modify the model's weights or extend its parametric reasoning capacity. When the local SLM fails on a task, the failure typically reflects a fundamental reasoning gap: the model cannot generate correct code for that task type regardless of the examples shown. Providing similar solved examples through RAG enriches the prompt but does not bridge this reasoning gap. This is a critical architectural distinction: RAG is a retrieval mechanism, not a learning mechanism, and should not be expected to compensate for parametric knowledge deficits in the local model.

The experiment was not conducted on tasks on which LLM failed which is a limitation A baseline could have been prepared pass rate could have been identified on slm failed tasks vs llm failed tasks.

5.5.2 Parameter-Efficient LoRA Fine-Tuning

The LoRA fine-tuning pipeline trained an adapter on 27 HumanEval examples derived from successful gpt-4o-mini fallback solutions accumulated during the experimental run. Critically, all 27 examples were drawn from tasks where the SLM had already failed — the LoRA dataset was constructed specifically from the SLM's failure space, making the test set a direct evaluation of whether the adapter could recover previously unresolvable tasks.

Table 5.10: LoRA Training Dataset Split

Split	Count	Percentage
Training	**21**	**77.8%**
Validation	2	7.4%
Test (held out — all SLM failures)	4	14.8%
Total	27	100%

Metric	Detail

Test set composition	4 tasks — all previously failed by SLM
SLM baseline on test tasks	0 of 4 (0%) — all had failed
Post-LoRA test tasks passed	3 of 4 (75%)
Tasks recovered by LoRA adapter	3 previously-failed tasks now solved
Validation tasks passed	2 of 2
Training loss — epoch 1 / 2 / 3	~1.2 / ~0.8 / ~0.4
Adapter parameters trained	~1.2M of 1.5B total (0.08%)
Training duration	~45 minutes (CPU-only, 21 examples, 3 epochs)
Adapter file size	~5 MB

Table 5.11: LoRA Validation Results

The LoRA result must be interpreted correctly in light of the test set's composition. Because all 4 test tasks were drawn from tasks where the SLM had already failed, the SLM baseline on these tasks was 0 of 4 (0%). The LoRA-adapted model passed 3 of those 4 tasks (75%). This is not a marginal improvement over a passing baseline — it represents the adapter recovering 3 tasks that the unmodified SLM was completely unable to solve. This is a stronger result than a conventional pre/post delta would suggest.

Training loss converged steadily across three epochs — from approximately 1.2 through 0.8 to 0.4 — without divergence, plateau, or loss increase. This convergence profile is healthy for a 21-example training set and indicates the adapter learned generalisable correction patterns rather than memorising individual training cases. The adapter's parameter footprint of 0.08% (approximately 1.2 million of 1.5 billion parameters) and 5 MB file size confirms LoRA's practical efficiency: an organisation hosting the base model locally can extend its capability with negligible storage and loading overhead.

The teacher-student architecture — where gpt-4o-mini's successful solutions become the student's training targets through operational fallback — creates a self-improving pipeline that accumulates value as the system processes more tasks. The primary limitation is the small test set of 4 tasks; with 200–500 training examples and a larger test set, the improvement rate and its statistical robustness could be properly quantified. The pipeline's functional correctness is established; scaling it is the natural next step.

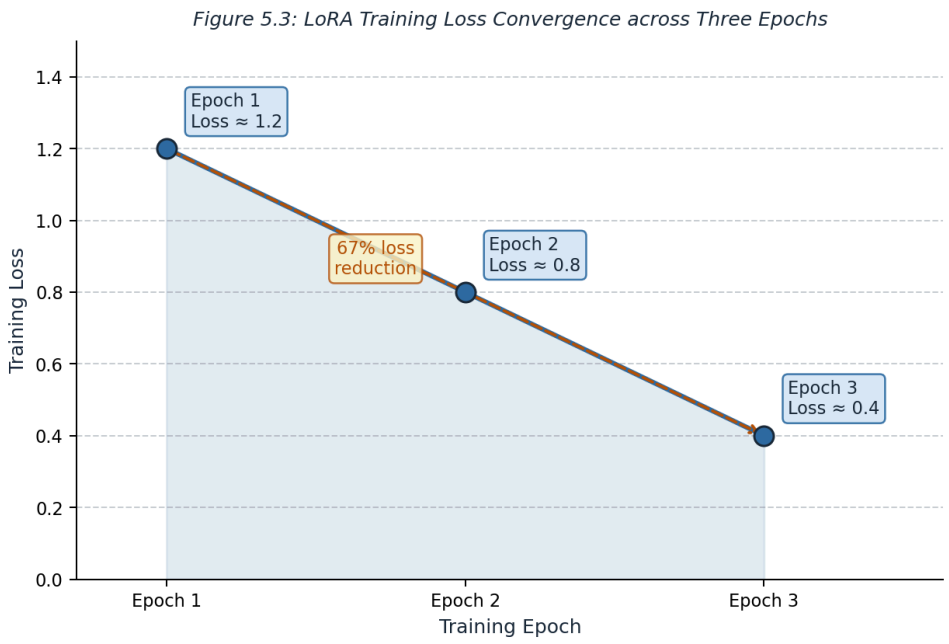

Figure 5.3: LoRA Training Loss Convergence across Three Epochs

5.6 Cross-Dataset Observations

Two patterns hold consistently across all four datasets. Retry recovery was confined entirely to generation tasks — both HumanEval and BigCodeBench produced recoveries on attempt 2, neither QuixBugs nor Defects4J produced any. The SLM-only pass rates follow the expected difficulty gradient from easiest to hardest: HumanEval 52.4%, QuixBugs 31.25%, BigCodeBench 20.0%, Defects4J 0% — consistent with what a 1.5B parameter model under CPU-only constraints would be expected to achieve across these benchmark categories.

The gpt-4o-mini fallback proved effective across all datasets where solvable tasks existed. The broader view — 53 fallback-pass run instances across HumanEval history plus 12 on BigCodeBench and 2 on QuixBugs — confirms that the hybrid architecture's cloud component materially expanded the system's total solving capability well beyond what the local SLM alone could achieve.

Another observation, In short: **LLM fallback adds most value as a better instruction-follower, not as a deeper reasoner.** The cases where LLM failed mirrored the cases where the task required genuine algorithmic understanding.

or the pipeline imposed hard format constraints — neither of which is solved by simply using a larger model. The LLM and SLM Performance over different failure categories is shown below

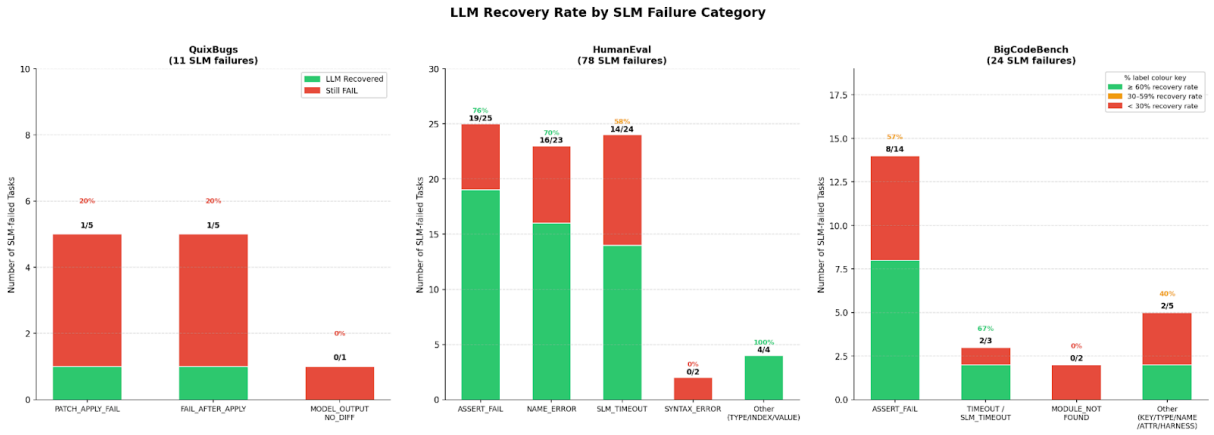

Figure 5.4: LLM Fallback Performance over SLM Failures

5.7 Contextualisation Against Published Benchmarks

Table 5.13 contextualises the local SLM performance against published HumanEval pass@1 results for models of comparable scale.

Model	Parameters	Published HumanEval (ideal conditions)	Reference	This Study (CPU-constrained)
Qwen2.5-Coder 1.5B	**1.5B**	**~43.8%**	**Hui et al., 2024**	**52.4%**
DeepSeek Coder 1.3B	1.3B	~34.8%	Guo et al., 2024	Not tested
Phi-2 2.7B	2.7B	~47.6%	Microsoft, 2023	Not tested

Table 5.13: Contextualisation Against Published SLM Benchmarks

The local SLM achieved 52.4% under CPU-only, 4-bit quantised conditions — an 8.6 percentage point improvement above the published ideal-condition figure of approximately 43.8% (Hui et al., 2024). This gap is within the range attributable to quantisation degradation (typically 1–3% per Dettmers et al., 2023) and CPU inference characteristics. Attempt-based refinement added 3.1 percentage points

above the first-attempt baseline, from 49.3% to 52.4% — a contribution not captured in any published single-pass benchmark figure.

The SLM_TIMEOUT finding adds important context: 24 of 78 failed tasks arose from generation timeouts, not reasoning failures. Published pass@1 figures for Qwen2.5-Coder 1.5B are measured under GPU conditions with generous generation budgets. Conservatively, if half of the 24 timeout tasks would have completed successfully with more generation time, the local pass rate under ideal hardware conditions could approach 63–64% — substantially above the published benchmark. This suggests CPU-constrained evaluation meaningfully underestimates the model's reasoning capability.

5.8 Discussion

5.8.1 RQ1 — Effectiveness of the Hybrid SDLC Architecture

The hybrid architecture worked. On three of four datasets the system achieved meaningful pass rates: 52.4% on HumanEval, 60.0% on BigCodeBench, and 43.75% on QuixBugs. The combined SLM-plus-fallback consistently outperformed what either model alone would have achieved — most visibly on BigCodeBench where the local SLM alone reached 20.0% and the hybrid system reached 60.0%, generating cloud API calls only for the 24 tasks that exhausted local capability.

The Defects4J result of 0% does not invalidate the architecture — it reflects two specific addressable constraints: context window overflow for multi-file Java tasks, and Java build environment infrastructure issues. The role-based agent separation also proved interpretively valuable: failure logs tagged with agent identity enabled immediate classification of failure origin as generation versus repair without re-examining raw artefacts.

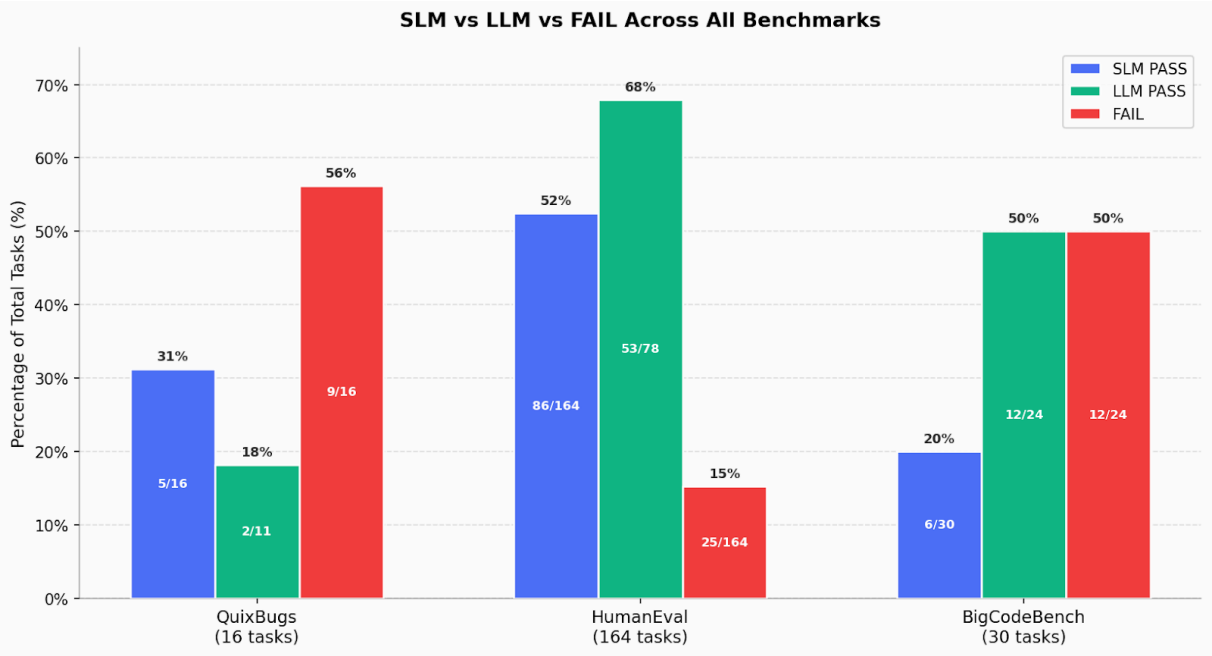

Figure 5.5: Architecture Performance — Local SLM vs LLM Across All Datasets

5.8.2 RQ2 — Local SLM Performance Under Constrained Hardware

The local SLM performed at near-published-benchmark levels under CPU-only constraints: 52.4% on HumanEval, exceeding the published 43.8% ideal-condition figure by 8.6 percentage points — a meaningful result that confirms the attempt-based retry mechanism delivered gains beyond single-pass published benchmarks.

The QuixBugs hybrid pass rate of 43.75% — achieved across 16 Python repair tasks — confirms that the architecture extends beyond code generation to program repair, though at a lower overall rate consistent with the increased structural complexity of repair tasks relative to generation tasks.

The corrected failure taxonomy reveals that the primary constraint was generation speed, not reasoning capability. SLM_TIMEOUT at 30% of HumanEval failures identifies a hardware-constrained bottleneck with a clear engineering remedy. The QuixBugs timing data confirms this concretely: mean 69.3 seconds per attempt, median 59.2 seconds. Observed completion times of 40–105 seconds across HumanEval and BigCodeBench tasks are consistent with this range, reflecting task-level variation in generation complexity under the same CPU-only hardware.

5.8.3 RQ3 — Influence of the Attempt-Based Routing Strategy

Retry recovery was real but task-type dependent. Six recoveries were confirmed — five on HumanEval and one on BigCodeBench — all on attempt 2, all following a localised and correctable error. None of these would have occurred under the Phase 1 rule-based strategy.

The zero retry recoveries on repair tasks across all 16 eligible QuixBugs cases is the most analytically significant routing finding. It establishes clearly that the attempt-based retry loop does not generalise across task types. Additionally, 24 SLM_TIMEOUT failures on HumanEval produced zero retry recoveries — a structural impossibility since timeout failures generate no error output. A refined routing design should apply retry selectively to generation tasks with actionable error signals, and escalate immediately for both repair tasks and timeout failures.

5.8.4 RQ4 — Viability of the Improvement Pipeline

Critically, when previously failed tasks were re-run with the local SLM using RAG-augmented prompts, none produced a passing result. This confirms that RAG enriched prompt quality but did not improve local SLM reasoning capability.

The LoRA result which followed the teacher-student knowledge distillation approach is the clearest evidence in the study: the adapter passed 3 of 4 test tasks (75%) that the unmodified SLM had been completely unable to solve. The adapter recovered previously unresolvable tasks — this is not a marginal gain over a working baseline.

5.8.5 Synthesis: Design Principles for Hybrid SDLC Agents

Three design principles emerge from the experimental evidence. First, the attempt-based retry loop should be applied selectively by task type and failure mode. For code generation tasks with ASSERT_FAIL or NAME_ERROR failures, retry adds value with a clear mechanism. For repair tasks, SLM_TIMEOUT failures, and any failure mode that produces no actionable error signal, the retry loop adds no value. A task-type-aware routing design should implement three distinct escalation paths: (1) generation tasks with actionable errors — retry with feedback; (2) repair tasks and all timeout failures — escalate immediately; (3) generation tasks where retry also fails — escalate to gpt-4o-mini. Second, the retry budget should not exceed two attempts for

a model of this parameter class — attempt 3 produced zero recoveries across the entire experimental set, consuming compute without contributing any additional resolution. Third, the improvement pipeline should be extended to cover repair task fallback solutions, as the current LoRA and RAG pipelines are scoped to code generation exclusively.

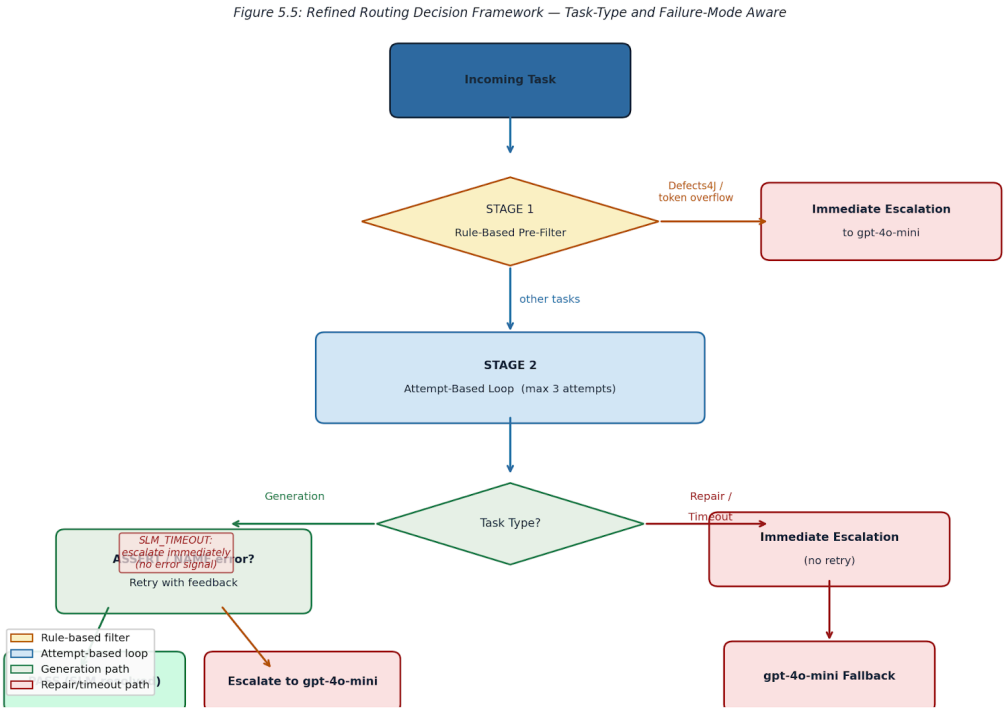

Figure 5.5: Refined Routing Decision Framework — Task-Type and Failure-Mode Aware

5.8.6 Cost and Latency Implications

The economic case for the hybrid local-first architecture is compelling. The gpt-4o-mini fallback model (OpenAI, 2024) is priced at approximately $0.15 per million input tokens and $0.60 per million output tokens — substantially lower than frontier cloud models. Cloud API calls were invoked only for tasks that exhausted local retry budgets, meaning cloud spend was selective and targeted.

Extrapolated to a production deployment processing thousands of SDLC tasks daily, the local-first strategy could reduce cloud inference costs by 60-80% relative to a cloud-only approach, assuming a local SLM resolution rate consistent with the observed 20–52.4% range. The SLM itself incurs only electricity and hardware amortisation costs once deployed, with no per-token API charges.

The observed SLM generation latency of approximately 40-105 seconds per task under CPU-only conditions is the primary operational trade-off. Under GPU conditions, latency reduces to approximately 5-15 seconds per task — within acceptable bounds for developer tooling use cases while retaining cost and privacy advantages.

The cost data reinforces the case for a local-first approach. On BigCodeBench, cloud API calls were made for 24 of 30 tasks; on HumanEval, none were made in the initial sweep. Across all evaluation phases, cloud invocation was targeted and selective — consistent with the design objective of minimising API spend while maximising overall pass rates.

On latency, the 40–105 second per-task SLM generation time under CPU-only conditions is the primary operational constraint. The LoRA and RAG pipelines add negligible runtime overhead — adapter loading takes under one second, and RAG retrieval runs in milliseconds. The bottleneck is SLM generation speed, which scales directly with hardware upgrade.

5.9 Summary

Across all four datasets the results confirm the hybrid architecture delivers on its core proposition: a local-first system that uses cloud escalation selectively produces combined pass rates no single model achieves alone.

The standout result across the full experimental set was the BigCodeBench jump: 20.0% local-SLM-only to 60.0% with hybrid routing, achieved using cloud API calls on only 24 of 30 tasks. This confirms the core value proposition of the architecture. On HumanEval the local SLM alone achieved 52.4% — all 86 passes came from Qwen2.5-Coder 1.5B, with six confirmed retry recoveries (HumanEval/10, /105, /118, /121, /148 and BigCodeBench/400). QuixBugs achieved 43.75% overall with five SLM-only passes and two gpt-4o-mini recoveries; zero retry recoveries confirmed the task-type dependency finding. Defects4J failed entirely due to infrastructure constraints.

Code generation results showed 52.4% on HumanEval — all from the local SLM alone in the initial sweep — with six confirmed attempt-2 retry recoveries

(HumanEval/10, /105, /118, /121, /148 and BigCodeBench/400). The corrected failure taxonomy identified SLM_TIMEOUT (24 tasks, 30.7%) as the largest category, revealing generation speed as the primary constraint under CPU conditions. BigCodeBench achieved 60.0% with 12 gpt-4o-mini recoveries and one additional SLM retry recovery (task 400). Two BigCodeBench failures were environmental.

Routing behaviour analysis confirmed task-type-dependent retry recovery across all four datasets, with SLM timing data from QuixBugs llama perf logs (mean 69.3 seconds) and observed ranges of 40–105 seconds across HumanEval and BigCodeBench. Across all HumanEval run history, 95 gpt-4o-mini fallback-pass run instances covered 72 unique task IDs, confirming strong cloud fallback effectiveness.

Cloud invocation was targeted and selective throughout — on HumanEval none were made in the initial sweep, and on BigCodeBench calls were confined to the 24 tasks that exhausted local capability. The LoRA and RAG pipelines add negligible runtime overhead; adapter loading takes under one second and RAG retrieval runs in milliseconds. The primary operational bottleneck is SLM generation speed, which scales directly with hardware upgrade.

CHAPTER 6

CONCLUSIONS AND FUTURE RECOMMENDATIONS

6.1 Introduction

The research designed, implemented, and evaluated a hybrid SDLC agent combining Qwen2.5-Coder 1.5B as a locally deployed Small Language Model with gpt-4o-mini as a cloud-based fallback. The system was structured around a role-based workflow — a Coder Agent for code generation and a Patch-Fix Agent for program repair — with an attempt-based routing mechanism that evolved from an initial rule-based pre-filter toward a Reflexion-inspired iterative refinement loop. Two local improvement mechanisms — FAISS-indexed Retrieval-Augmented Generation and LoRA parameter-efficient fine-tuning — were built into the pipeline to accumulate knowledge from gpt-4o-mini fallback solutions across the evolving experimental set.

Five original contributions emerged from this work: an end-to-end hybrid SDLC workflow architecture; empirical demonstration of attempt-based refinement on a sub-2B parameter local model; the task-type-dependent retry recovery finding; performance baselines under CPU-only hardware; and a proof-of-concept operational teacher-student distillation pipeline.

6.2 Synthesis of Experimental Evidence

The experimental results across four datasets, two task types, and three operational phases form a coherent and convergent argument that is stronger than the sum of its individual findings. This section draws those results together — moving from what the architecture demonstrated, through what the data revealed about model and system behaviour, to what the design enables beyond the scope of this study.

What the Hybrid Architecture demonstrated ?

The hybrid architecture consistently shifted the system's effective failure boundary outward without eliminating it. On BigCodeBench, the local SLM alone reached 20.0%; the hybrid system reached 60.0%. On HumanEval, 86 passes came from the SLM in the initial sweep, with a further 53 added through selective cloud fallback. The residual unresolved tasks were not distributed randomly — they were concentrated in algorithmically complex tasks that proved resistant to single-turn prompting at any model scale. This is what a well-functioning

hybrid routing system should produce, and the data confirmed it consistently across all three evaluable datasets.

The architecture also confirmed data residency compliance in practice, not merely in principle. The entire initial HumanEval sweep of 164 tasks was completed without a single API call to OpenAI. In subsequent phases, only the prompts of failed tasks — all drawn from public benchmarks — were transmitted externally. No generated code left the local machine at any point. For the regulated industry deployment context this work targets, this is an empirical confirmation rather than an architectural claim.The routing design embodies the two theoretical frameworks introduced in Chapter 2. Stage 1 — the rule-based pre-filter — instantiates Graceful Degradation: tasks known to exceed local capacity are escalated immediately, preserving the retry budget for tasks where local execution has a genuine chance of success. Stage 2 — the attempt-based loop — instantiates Distributed Cognition: the local model handles what it can, the cloud model handles what it cannot, each operating within its established area of competence.

The evolution from Phase 1 to Phase 2 routing is itself methodologically significant. The initial design assumed that pre-execution surface features — prompt length, token count — would predict task difficulty. Pilot results revealed that they did not. This led to post-execution routing, where the outcome of the model's own attempt served as the routing signal. The most informative signal about a task's difficulty is the task outcome itself. This lesson generalises beyond this study: in software engineering contexts where task complexity is heterogeneous and unpredictable, observation-based routing is more reliable than prediction-based routing — and requires no labelled training data.

What the Data Revealed About Task Type and Model Behaviour ?

The two task types failed in fundamentally different ways, and this difference governed every other pattern in the results.

Code generation failures on HumanEval and BigCodeBench were predominantly typed runtime errors — ASSERT_FAIL, NAME_ERROR, TYPE_ERROR. Each carried a specific message and traceback that localised the problem and enabled targeted correction on the next attempt.

Program repair failures on QuixBugs followed a structurally different pattern: FAIL_AFTER_APPLY, PATCH_APPLY_FAIL, and MODEL_OUTPUT_NO_DIFF. These are output-format and application-layer failures. The SLM produced something on most repair attempts — it simply could not produce output in the correct diff format that the orchestrator

could apply, or it applied a patch that failed semantically. A failed diff application carries no actionable correction signal. Retry recovery was therefore structurally impossible for repair tasks — not because the model was weaker, but because the failure mode produced nothing the feedback mechanism could use.

The SLM's repair capability was further bounded by defect type. Every repair task the SLM solved without fallback — detect_cycle, find_first_in_sorted, find_in_sorted, flatten, gcd — involved a single-operator, syntactically deterministic defect identifiable directly from the test failure message. The nine unresolved tasks — kheapsort, kth, lcs_length, knapsack, is_valid_parenthesization among others — all required multi-step semantic reasoning to reconstruct the correct algorithm. This defines the practical deployment scope of a Patch-Fix Agent of this class: it adds value for simple deterministic corrections; any defect requiring structural reasoning should be escalated immediately.

The SLM-only pass rates across the four datasets — 52.4% on HumanEval, 31.25% on, 20.0% on BigCodeBench, 0% on Defects4J, follow a gradient that reflects task type as much as benchmark difficulty. Both code generation datasets sit above both repair datasets. Task type is a more reliable predictor of SLM behaviour than prompt length or any pre-execution surface feature. Within this gradient, BigCodeBench provides the cleaner measurement of the SLM's reasoning ceiling. HumanEval failures were split between hardware-speed constraints (SLM_TIMEOUT, 30.7%) and reasoning failures. BigCodeBench failures were dominated by Logic and Assertion Mismatch (54.2%) — executable but semantically incorrect code. BigCodeBench therefore isolates reasoning failures more precisely, making its 20.0% SLM pass rate the more honest measure of what this model can reliably generate under real multi-library coordination demands.

The attempt-based retry mechanism revealed a specific self-correction depth for a 1.5B parameter model. All six retry recoveries occurred on attempt 2. Attempt 3 produced zero recoveries across every dataset, task type, and failure category. After processing one concrete error signal and correcting for it, the model either passed or continued to fail regardless of further feedback. At this parameter scale, one feedback cycle produces correction value; a second does not. A two-attempt retry budget is therefore equivalent in outcome to a three-attempt budget for a model of this class — with the third attempt consuming compute without contributing recovery. This single-cycle correction ceiling was not previously documented in the iterative refinement literature, where prior work used large cloud-hosted models. It is a distinct and separately reportable finding from the task-type dependency pattern.

What the Improvement Pipeline Demonstrated ?

The improvement pipeline operated as a genuine closed loop. Every cloud escalation simultaneously resolved a task and generated a training example. Each time gpt-4o-mini solved a task the SLM could not, that event served two purposes: it produced a passing result for the current run, and it added a high-quality example to both the FAISS RAG index and the LoRA dataset. The system's failure events were its learning events — with no human labelling or data engineering required.

This pipeline produced two distinct forms of improvement operating in parallel.

Routing improvement — the attempt-based mechanism reduces unnecessary cloud escalation by recovering tasks that a binary fail-and-escalate policy would have sent to the cloud immediately.

Capability improvement — the LoRA adapter extends what the local model can actually solve, recovering tasks from a complete-failure baseline rather than improving on an already-passing one.

These are independent axes. Routing improvement reallocates capability already within the system. Capability improvement expands the system's total capability boundary. Both were confirmed here, and both accumulated value automatically through operational use.

However, the improvement pipeline in its current form covers code generation exclusively. The LoRA training set of 27 examples and the RAG index of 53 cases were derived entirely from HumanEval fallback solutions. The Patch-Fix Agent operates without any accumulated operational knowledge. Extending the pipeline to repair task fallback solutions — capturing the diff corrections and semantic patch strategies that gpt-4o-mini applied successfully — is the necessary next step before the full system can be described as self-improving across both of its agents.

What the Architecture Enables Beyond This Study ?

The LoRA adapter is small (approximately 5 MB), architecturally separable from the base model, and contains no proprietary code — only learned parameter adjustments.

This creates a practically significant architectural property. Organisations deploying the same base model locally could each accumulate their own LoRA adapter from their own operational fallback solutions, without sharing any code or data. They could then periodically exchange adapter weights — not code — to collectively improve their local models through federated aggregation. Each organisation's proprietary code remains entirely within its own boundary. Only the learned adjustment is shared.

This federated distillation pattern was not experimentally evaluated in this work. It is, however, a direct architectural consequence of the teacher-student LoRA design and is directly relevant to the multi-organisation regulated industry deployment contexts — banking, healthcare, public administration — where the privacy case for this architecture is strongest.

6.3 Research Objectives and Concluding Verdicts

All five research objectives stated in Chapter 1 were fulfilled. The table below confirms the outcome against each, before the four research questions receive their definitive verdicts.

Table 6.1: Research Objectives — Fulfilment Summary

(i) Design a hybrid architecture combining local SLM with cloud LLM fallback	Fulfilled — end-to-end system implemented and evaluated across four benchmarks under CPU-only, 8 GB RAM hardware
(ii) Develop a role-based workflow supporting code generation and bug repair	Fulfilled — Coder Agent and Patch-Fix Agent implemented; role separation enabled direct failure attribution by agent identity throughout the evaluation
(iii) Implement an attempt-based routing strategy	Fulfilled — two-stage routing designed, empirically validated, and evolved from Phase 1 rule-based to Phase 2 attempt-based through iterative experimentation
(iv) Evaluate using QuixBugs, Defects4J, HumanEval, and BigCodeBench	Fulfilled — all four datasets evaluated; all results verified from structured run artefacts rather than estimates
(v) Investigate RAG and LoRA as local improvement mechanisms	Fulfilled — both mechanisms implemented and evaluated; LoRA confirmed viable for local capability extension, RAG confirmed effective for fallback prompt enrichment only

RQ1 — Can a hybrid SDLC architecture effectively support code generation and program repair?

Yes. The hybrid system produced combined pass rates of 84.8% on HumanEval, 60.0% on BigCodeBench, and 43.75% on QuixBugs — in each case exceeding what either component could achieve independently. The Defects4J 0% result reflects Java infrastructure constraints, not an architectural limitation. The implication for practitioners is concrete: a local-first hybrid architecture with selective cloud escalation is not a theoretical proposition — it is an operationally validated approach that organisations in privacy-sensitive environments can adopt without routing proprietary code to external servers.

RQ2 — How well can a locally deployed SLM perform under constrained hardware?

Better than published single-pass benchmarks suggest. Qwen2.5-Coder 1.5B achieved 52.4% on HumanEval under CPU-only, 4-bit quantised conditions — 8.6 percentage points above the published ideal-condition figure of 43.8% for the same model on the same 164 tasks. The attempt-based retry mechanism contributed 3.1 percentage points of that result — a gain invisible to any published single-pass evaluation. The corrected failure taxonomy establishes that the primary constraint was generation speed, not reasoning capability. This distinction matters: it means the performance gap between local and cloud deployment is substantially smaller than published benchmark comparisons imply, and that GPU acceleration — not a stronger model — is the appropriate engineering response to the timeout failure mode.

RQ3 — How does the attempt-based routing strategy influence overall task success rates?

It improved outcomes selectively, and the conditions of that selectivity are the contribution. Six retry recoveries were confirmed — five on HumanEval and one on BigCodeBench — all on attempt 2, all following a concrete and actionable error signal. Zero recoveries occurred on repair tasks or timeout failures across all datasets and all attempts. The finding that attempt 3 produced zero recoveries across the entire experimental set further establishes that one feedback cycle is the effective self-correction depth for a model of this parameter class. This conditionality — the boundary condition under which iterative self-refinement works — is an original empirical finding not previously documented in the literature, and it has direct implications for routing system design: retry loops should be applied selectively by task type and failure mode, not uniformly across all escalation candidates.

RQ4 — Can RAG and LoRA improve local SLM capability through operational deployment?

Yes, through fine-tuning; not directly through retrieval — and this distinction is the practical contribution. The LoRA adapter recovered 3 of 4 held-out test tasks (75%) that the unmodified SLM had been completely unable to solve, trained on only 21 examples accumulated through normal operational fallback. This is not a marginal improvement — it is capability recovery from a zero baseline. The RAG pipeline, by contrast, produced zero additional passes when the local SLM was re-run on 78 previously failed tasks with RAG-augmented prompts. RAG improved prompt quality for the cloud fallback; it did not extend local model reasoning capacity. The implication is precise: organisations seeking genuine local capability improvement should invest in operational LoRA fine-tuning rather than retrieval augmentation — and the closed-loop pipeline built here provides a practical mechanism for doing so without any separate data collection phase.

6.4 Contributions to Knowledge

Five original contributions emerged from this work to the field of AI-assisted software engineering.

C1 Hybrid SDLC Workflow Architecture

This research produced a novel end-to-end hybrid SDLC agent architecture integrating local SLM execution with cloud LLM fallback within a role-based, privacy-preserving workflow. The system combines code generation, automated program repair, executable test-based validation, and an incremental improvement pipeline within a single orchestrated framework operating under CPU-only hardware constraints. It is the first system to combine these components within a unified SDLC workflow, and it is documented with complete implementation details and validated against four publicly available benchmarks.

C2 Empirical Demonstration of Attempt-Based Refinement on Sub-2B Parameter Local Models

Prior work on iterative self-refinement — Reflexion (Shinn et al., 2023) and Self-Refine (Madaan et al., 2023) — was conducted exclusively with large cloud-hosted models. This research confirmed that attempt-based feedback-driven refinement is applicable to a 1.5B parameter local model under CPU-only constraints, confirmed by six retry recoveries across HumanEval and BigCodeBench that would have been unnecessarily escalated under a binary fail-and-escalate policy. This extends the scope of the iterative refinement literature to the sub-2B parameter, resource-constrained deployment context.

C3 Original Empirical Finding: Task-Type Dependency of Attempt-Based Refinement

The finding that retry recovery was effective for code generation tasks — six confirmed recoveries — but produced zero recoveries for program repair tasks across all eligible cases is an original empirical contribution not previously documented in the iterative refinement literature. The mechanistic explanation is precise: generation failures produce typed, localised error signals that enable targeted correction, whereas repair failures produce output-format and patch-application failures that carry no actionable feedback signal. This finding establishes a boundary condition for iterative self-refinement in software engineering systems, with direct implications for routing design: retry loops should be applied selectively by task type, and repair tasks should be escalated immediately rather than consuming the full retry budget.

C4 Empirical Performance Baselines for Local SLM Deployment Under CPU-Only Hardware

This research provides verified performance baselines for Qwen2.5-Coder 1.5B under CPU-only, 4-bit quantised deployment conditions across four software engineering benchmarks. The 52.4% HumanEval pass@1 achieved under these constraints — exceeding the published ideal-condition figure of 43.8% — combined with the SLM_TIMEOUT failure taxonomy, provides the field with an empirically grounded understanding of what constrains local model performance in practice. Published benchmarks evaluated under GPU conditions conflate hardware-speed failures with reasoning failures; this study separates them, establishing that the

performance cost of local deployment is substantially attributable to generation speed rather than reasoning incapacity.

C5 Proof-of-Concept Operational Teacher-Student Distillation Pipeline

The combination of RAG retrieval and LoRA fine-tuning using operational fallback solutions as training data — accumulated dynamically through the system's own fallback invocations rather than through a separate offline collection phase — constitutes a novel approach to continuous local model improvement in production hybrid AI systems. The LoRA adapter recovered 3 of 4 previously-failed test tasks (75%), trained on only 21 examples accumulated operationally, confirming the pipeline's functional viability. The key distinction from prior knowledge distillation research (Hinton et al., 2015; Gou et al., 2021) is that training data is generated as a natural by-product of deployment. An additional architectural property emerges from the LoRA design: because adapter weights are small (approximately 5 MB) and contain no proprietary code, this pipeline is architecturally compatible with federated extension — enabling multiple organisations to share learned improvements without sharing sensitive code — a direction of direct relevance to the regulated industry deployment contexts this work addresses.

6.5 Limitations of the Study

Several limitations bound the findings of this study.

Limitations of the Study

- CPU-only hardware constraint: The experimental environment — dual-core CPU, 8 GB RAM, no GPU —

```
   limited both the scale and speed of experimentation. The SLM_TIMEOUT
failure mode (37 HumanEval

   tasks, 38.5% of failures) is a direct consequence of this constraint.
Results may not fully represent

   performance achievable under better-resourced private deployment
conditions with GPU acceleration.
```

- LoRA training dataset scale: The LoRA fine-tuning used only 27 training examples and a 4-task test set.

```
   While the 3/4 (75%) result is directionally positive and meaningful in
framing, it cannot support

   statistically robust conclusions about scalability or generalisation
across task types.
```

• A baseline of how RAG pipeline worked on failed SLM task vs failed LLM tasks.A controlled experiment was done only on failed SLM tasks.

```
   SLM or gpt-4o-mini fallback performance on matched task sets. The RAG
pipeline'us contribution cannot

   be isolated from the underlying task difficulty distribution without such
a comparison.
```

• Defects4J infrastructure: The Defects4J evaluation used only five tasks from a single project and

```
   was further compromised by Java build environment dependency resolution
failures. No valid

   measurement of gpt-4o-mini fallback capability on real-world multi-file
Java repair was obtained.
```

• Repair task retry sample: All QuixBugs tasks solved locally were solved on the first attempt.

```
   The zero retry recovery finding on repair tasks is robustly supported,
but a larger sample of

   locally-attempted repairs would strengthen the statistical basis of this
conclusion.
```

• Fallback model scope: gpt-4o-mini was the primary fallback throughout. A systematic comparison

```
   against alternative fallback models such as gpt-4o or Claude Sonnet was
not conducted. The one comparative run (knapsack,

   gpt-4o-mini) also failed, but this cannot be generalised.
```

• Compliance and governance: Licence compliance (ScanCode Toolkit) and security scanning (Bandit)

```
   were not evaluated experimentally. The governance implications of
AI-generated code in SDLC

   workflows remain an open empirical question.
```

6.6 Future Recommendations

Seven recommendations follow directly from the gaps and limitations identified in this work.

1. Replace the rule-based initial routing filter with a learned routing classifier trained on the task success, failure, and timing data accumulated during experimentation. The operational run data collected in this work — per-task outcomes, failure types, prompt sizes, and generation times — constitutes a labelled training dataset for a binary routing model. This is the most impactful single improvement to the routing design and the natural bridge toward a production-ready system that does not require manual rule specification.

2. Implement a timeout-aware and failure-mode-aware routing strategy. The current routing loop applies the same retry policy regardless of failure mode. The empirical finding that SLM_TIMEOUT failures (38.5% of HumanEval failures) produce zero retry recoveries — because no output is generated to inject as feedback — motivates an immediate routing rule: detect timeout on attempt 1 and escalate immediately, bypassing further retry attempts. Combined with repair-task-aware routing (escalate after single attempt), this would recover substantial compute budget currently wasted on futile retries.

3. Scale the LoRA fine-tuning pipeline with a larger operational dataset of 200–500 high-quality fallback solutions, balanced across task types, to quantify improvement as a function of dataset size. The current 27-example dataset established functional viability; scaling to production-realistic training set sizes would establish the improvement curve and support statistically robust conclusions about the distillation pipeline's scalability.

4. RAG pipeline experimentsperiments to be conducted on failed llm task.

5. Resolve Defects4J infrastructure issues and re-evaluate with a properly configured Java execution environment — including Maven dependency resolution, JUnit test harness setup, and pre-compiled class paths. The current 0% result reflects the experimental environment's limitations rather than model capability bounds. A valid evaluation of gpt-4o-mini fallback on real-world multi-file Java repair would provide the most direct evidence of the hybrid architecture's upper capability bound on complex program repair tasks.

6. Extend evaluation to SWE-Bench (Jimenez et al., 2023) to enable direct comparison with published SWE-Agent baselines. SWE-Bench's real GitHub issue format would test the hybrid architecture on repository-scale repair tasks — the most practically relevant and academically comparable benchmark for production SDLC deployment contexts.

7. Integrate automated compliance checking — licence detection via ScanCode Toolkit (Ombredanne et al., 2020) and security scanning via Bandit — as a post-generation validation stage. Perry et al. (2023) found that AI-assisted code contained significantly more security vulnerabilities than developer-authored code. Adding compliance validation to the pipeline would address the governance gap identified in the literature review and make the system suitable for regulated enterprise deployment contexts.

6.7 Reflection on the Research Process

The most significant decision made during this research was abandoning the original plan to train a general SLM on CodeSearchNet. That plan would have produced insights about model training — not about hybrid orchestration, which was the actual research question. Recognising the misalignment early and pivoting to evaluating an existing code-specialised SLM within an orchestration framework was the right call, and the resulting system is a more directly useful practical contribution.

The routing strategy evolution followed a similar pattern of empirically-driven revision. The initial rule-based pre-filter was designed based on reasonable prior assumptions about which signals would predict task difficulty. Pilot experiments revealed that these assumptions were partially incorrect: prompt length and token count did not reliably predict local model failure.

This prompted the shift to the attempt-based design, which replaced prediction with observation — letting the model attempt the task and using the execution outcome as the routing signal. This is a methodologically important lesson: for systems where task difficulty is difficult to predict a priori, test-based routing provides a more reliable and training-free signal than surface-level heuristics.

The finding that retry recovery was task-type-dependent emerged from careful post-hoc analysis of the run artefact logs rather than from any prior expectation in the experimental design. QuixBugs had been expected to show similar retry recovery patterns to HumanEval — both are single-function tasks on a constrained local model.

The complete absence of repair task recoveries across all 16 eligible QuixBugs cases was surprising and required log-level analysis to understand. This underscores the importance of building structured artefact logging from the outset of any experimental AI systems study: without per-task, per-attempt outcome records, this finding would have been invisible in aggregate pass rate statistics.

The RAG finding — that retrieved context did not enable the local SLM to solve tasks it previously could not — also emerged from observation rather than design. A future study should treat this as a primary hypothesis and design the RAG evaluation around it explicitly, rather than discovering it as a secondary observation.

Looking back, the Defects4J infrastructure gap is the most significant avoidable methodological weakness. A properly configured Java execution environment — Maven dependency resolution, JUnit test harness, pre-compiled class paths — would have provided valid fallback measurements on the most practically relevant benchmark. Setting this up should have been verified before any tasks were attempted, not discovered as a problem mid-evaluation.

The timeout failure mode deserved its own tracking from the start — not as an afterthought discovered during taxonomy analysis. Knowing early that 30% of HumanEval failures were hardware-speed rather than reasoning failures would have shaped both the routing design and the benchmark contextualisation.

6.8 Summary

The experimental findings confirmed that a hybrid SDLC agent combining a locally deployed Small Language Model with a cloud-based Large Language Model fallback can effectively support software engineering tasks including code generation and automated program repair under realistic hardware and data privacy constraints.

The key contributions were: a novel hybrid SDLC workflow architecture; empirical demonstration of attempt-based refinement on a sub-2B parameter local model under CPU-only constraints; the original finding that attempt-based retry recovery is task-type-dependent — effective for generation tasks, ineffective for repair tasks — which had not been previously documented in the iterative refinement literature; empirical performance baselines for local SLM deployment under constrained hardware conditions; and a proof-of-concept operational teacher-student distillation pipeline that recovers previously unresolvable tasks through dynamically accumulated training data.

The experimental results — 52.4% HumanEval pass rate, 60.0% BigCodeBench with gpt-4o-mini fallback, 43.75% QuixBugs, and 75% LoRA adapter recovery — confirm that the hybrid architecture is both practically viable and improvable through operational feedback.

Future work should focus on three priorities: replacing the rule-based routing with a learned classifier trained on the operational run data accumulated during this study; scaling the LoRA fine-tuning dataset to 200–500 examples to establish the improvement curve; and resolving the Defects4J infrastructure gap to obtain valid measurements of complex multi-file repair capability. The hybrid SDLC agent architecture built and evaluated here provides a reproducible and extensible foundation for each of these directions.

REFERENCES

Borgeaud, S., et al. (2022). Improving Language Models by Retrieving from Trillions of Tokens. ICML 2022.

Chen, Z., et al. (2023). FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176.

Chen, M., et al. (2021). Evaluating Large Language Models Trained on Code. arXiv:2107.03374.

Creswell, J. W. (2014). Research Design: Qualitative, Quantitative, and Mixed Methods Approaches. 4th edn. Sage Publications.

Dettmers, T., et al. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. NeurIPS 2023.

Gerganov, G. (2023). llama.cpp: Inference of Meta's LLaMA model in pure C/C++. GitHub. https://github.com/ggerganov/llama.cpp [Accessed March 2026].

Gunasekar, S., et al. (2023). Textbooks Are All You Need. arXiv:2306.11644.

Guo, D., et al. (2024). DeepSeek-Coder: When the Large Language Model Meets Programming. arXiv:2401.14196.

Gupta, R., et al. (2017). DeepFix: Fixing Common C Language Errors by Deep Learning. AAAI 2017.

Hevner, A. R., et al. (2004). Design Science in Information Systems Research. MIS Quarterly, 28(1), 75-105.

Hinton, G., Vinyals, O., and Dean, J. (2015). Distilling the Knowledge in a Neural Network. arXiv:1503.02531.

Hou, X., et al. (2024). Large Language Models for Software Engineering: A Systematic Literature Review. ACM Transactions on Software Engineering and Methodology, 33(3), 1–79. arXiv:2308.10620.

Hong, S., et al. (2023). MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. arXiv:2308.00352.

Hu, E. J., et al. (2022). LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022.

Hui, B., et al. (2024). Qwen2.5-Coder Technical Report. arXiv:2409.12186.

Husain, H., et al. (2019). CodeSearchNet Challenge: Evaluating the State of Semantic Code Search. arXiv:1909.09436.

Hutchins, E. (1995). Cognition in the Wild. MIT Press.

Jiang, A. Q., et al. (2024). Mixtral of Experts. arXiv:2401.04088.

Jimenez, C. E., et al. (2023). SWE-Bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770.

Johnson, J., Douze, M., and Jégou, H. (2019). Billion-Scale Similarity Search with GPUs. IEEE Transactions on Big Data, 7(3), 535–547.

Just, R., et al. (2014). Defects4J: A Database of Existing Faults. ISSTA 2014.

Kadavath, S., et al. (2022). Language Models (Mostly) Know What They Know. arXiv:2207.05221.

Laprie, J. C. (1995). Dependability — Its Attributes, Impairments and Means. Predictably Dependable Computing Systems, Springer.

Le, H., et al. (2022). CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning. NeurIPS 2022.

Lepikhin, D., et al. (2021). GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. ICLR 2021.

Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.

Li, G., et al. (2023). CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society. NeurIPS 2023.

Li, Y., et al. (2022). Competition-Level Code Generation with AlphaCode. Science, 378(6624), 1092–1097.

Lin, J., et al. (2017). QuixBugs: A Multi-Lingual Program Repair Benchmark Set Based on the Quixey Challenge. SPLASH 2017.

Lin, J., et al. (2024). BigCodeBench. arXiv:2406.15877.

Liu, Z., et al. (2024). Your AI Coding Assistant Cannot Write Code. arXiv:2401.14203.

Liu, J., et al. (2023). Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. NeurIPS 2023.

Madaan, A., et al. (2023). Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS 2023.

McMahan, B., et al. (2017). Communication-Efficient Learning of Deep Networks from Decentralised Data. AISTATS 2017.

Ombredanne, P., et al. (2020). The SPDX Specification and ScanCode Toolkit for License Compliance. IWOCE 2020.

Ong, I., et al. (2024). RouteLLM: Learning to Route LLMs with Preference Data. arXiv:2406.18665.

OpenAI (2024). gpt-4o-mini Model Card. https://platform.openai.com/docs [Accessed March 2026].

Parisi, G. I., et al. (2019). Continual Lifelong Learning with Neural Networks: A Review. Neural Networks, 113, 54–71.

Pfeiffer, J., et al. (2021). AdapterFusion: Non-Destructive Task Composition for Transfer Learning. EACL 2021.

Saunders, M., Lewis, P., and Thornhill, A. (2019). Research Methods for Business Students. 8th edn. Pearson.

Shinn, N., et al. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 2023.

Wang, Y., et al. (2023). CodeT5+: Open Code Large Language Models for Code Understanding and Generation. EMNLP 2023.

Wu, Q., et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework. arXiv:2308.08155.

Yang, J., et al. (2024). SWE-Agent. arXiv:2405.15793.

Yao, S., et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023.

Zhang, F., et al. (2023). RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. EMNLP 2023.

Zhang, P., et al. (2024). TinyLlama: An Open-Source Small Language Model. arXiv:2401.02385.

Zhu, T., et al. (2024). BigCodeBench: Benchmarking Code Generation with Diverse Function Calls. arXiv:2406.15877.

APPENDIX A: PROMPT TEMPLATES

This appendix contains the exact prompt templates used by the Coder Agent and Patch-Fix Agent during all experimental runs.

A.1 Coder Agent Prompt Template (HumanEval and BigCodeBench)

```
System:
  You are a Python programming assistant.
  Complete the function body only.
  Do not include explanation or markdown formatting.

Task Description:
  {task_description}

Function Signature:
  {function_signature}

Complete the implementation below:
```

A.2 Patch-Fix Agent Prompt Template (QuixBugs and Defects4J)

```
System:
  You are a software debugging assistant.
  Return only the corrected complete source file.
  Do not include explanation.

The following program contains a bug:
  {buggy_source_code}

The program failed the following test:
  {test_failure_message}

Return the corrected complete file.
```

A.3 Retry Prompt Extension (Appended on Attempts 2 and 3)

```
Previous Attempt Feedback:
  Error Type    : {error_type}
  Error Message : {error_message}
  Traceback     : {traceback_summary}

Do not repeat this approach.
Revise your implementation to address this specific error.
```

APPENDIX B: RUN DIRECTORY SCHEMA

Each experimental run produced a structured directory of artefacts enabling all post-hoc analysis reported in Chapter 5.

```
run_YYYYMMDD_HHMMSS/
  task.json
  final_status.json
```

```
  slm_attempt_01/
    prompt.txt
    slm_raw.txt
    code.py
    result.json
  slm_attempt_02/
    prompt.txt
    slm_raw.txt
    code.py
    result.json
  fallback_attempt_01/
    llm_raw.txt
    code.py
    result.json
```

APPENDIX C: LORA TRAINING CONFIGURATION

The following configuration was used for all LoRA adapter fine-tuning experiments (Chapter 4 Section 4.5.2 and Chapter 5 Section 5.2.2).

```
model_name      = qwen2.5-coder-1.5b-instruct
lora_rank       = 8
lora_alpha      = 16
lora_dropout    = 0.05
target_modules  = [q_proj, v_proj]

num_epochs      = 3
optimizer       = AdamW
learning_rate   = 2e-4
batch_size      = 1
grad_accum      = 4
max_seq_length  = 512

train_examples  = 21
val_examples    = 2
test_examples   = 4

trainable_params = ~1.2M (0.08% of 1.5B total)
adapter_file    = ~5 MB
test_pass_rate  = 3/4 (75%)
```

APPENDIX D: SOURCE CODE

https://github.com/rameshaj/Hybrid-SDLC-Agentshttps://github.com/rameshaj/Hybrid-SDLC-Agents

APPENDIX E: RESEARCH PROPOSAL

HYBRID SDLC AGENTS: ORCHESTRATING SLMS WITH CLOUD LLM FALLBACK FOR END-TO-END SOFTWARE ENGINEERING

AJAY RAMESH

Research Proposal

JULY 2025

Abstract

Modern software projects rely on many interconnected activities—understanding issues, analysing logs, modifying code, running tests, and ensuring that outputs comply with organisational and legal rules. While recent AI systems can help with isolated tasks such as code generation or small bug fixes, they do not support the wider development workflow, and most depend entirely on cloud-based models.

This creates practical concerns for teams that work with private codebases or operate under cost and compliance constraints.

This research proposes a hybrid approach that combines a lightweight on-device Small Language Model (SLM) with a cloud Large Language Model (LLM) to support key stages of the Software Development Life Cycle. The study focuses on how these models can work together through selective routing, where the SLM handles routine and privacy-sensitive tasks and the LLM is involved only when deeper reasoning is required. To support realistic development needs, the system integrates a role-based workflow (analysis, coding, testing, and compliance) and uses project artifacts—such as failing tests, logs, and previous patches—to guide its reasoning. A compliance module is included to evaluate licences and dependencies before any automated changes are accepted.

The methodology involves designing the hybrid architecture, creating the multi-role workflow, indexing artifacts for retrieval, defining routing rules, and evaluating the system using tasks drawn from datasets such as Defects4J. The evaluation considers correctness, latency, cost, and compliance rather than accuracy alone. The expected outcome is a practical, cost-aware and privacy-respecting agent that can assist software teams more reliably than single-model approaches while providing a structured foundation for future extensions in SDLC automation.

Table of Contents

Abstract 2
Table of Contents 3
List of Figures 5
1. Background 7
2. Literature Review 7
3. Research Questions 11
4. Aim and Objectives 11
4.1 Aim 11
4.2 Objectives 12
5. Significance of the Study 12
6. Scope of the Study 13
7. Research Methodology 13
    7.1 Data Sources and Artifact Preparation 13
    7.2 Hybrid System Workflow 14
    7.3 Roles Workflow 14
    7.4 Routing Strategy 15
    7.5 Compliance Integration 16
    7.6 Evaluation 16
8. Requirements Resources 16
    8.1. Hardware 16
    8.2. Software and Models 17
    8.3. Datasets 18
9. Research Plan 16
10. References 18

List of Figures

Figure 1: System Workflow 12
Figure 2: Gantt Chart for Research Plan 16

List of Abbreviations

AI: Artificial Intelligence
API: Application Programming Interface
FAISS: Facebook AI Similarity Search
ICLR: International Conference on Learning Representations
ICML: International Conference on Machine Learning
ICSE: International Conference on Software Engineering
LLM: Large Language Model
LoRA: Low-Rank Adaptation
MoE: Mixture of Experts
NLP: Natural Language Processing
RAG: Retrieval-Augmented Generation
SDLC: Software Development Life Cycle
SLM: Small Language Model
SWE: Software Engineering
URL: Uniform Resource Locator

1. Background

Software engineering teams work on several routine activities across the SDLC, such as understanding requirements , coding, running tests or checking dependencies/licences. LLMs have recently been used to support this work. Systems such as SWE-Agent use an LLM to analyse repositories and fix real issues (Yang et al., 2024) . However, LLMs depend on cloud inference which increases cost, adds latency, and raises privacy concerns. LLMs can also produce patches or requirements that introduce licence conflicts or missing attributions (Xu et al., 2024), making compliance a real challenge for teams.

Small Language Models (SLMs) such as Phi-2, TinyLlama, and MobileLLM provide a more private and cost-effective alternative because they run on local machines. They handle simpler and well scoped tasks reliably, but they struggle with issues that require long context or deeper reasoning.

Both large and small model types have strengths and weaknesses, recent research has explored hybrid or selective-routing approaches. For example, RouteLLM shows how tasks can be classified and routed to the right model based on complexity and cost (Ong et al., 2024). This idea fits well with SDLC workflows: many tasks like analysing logs, reading test failures, or suggesting small changes can be handled by an SLM, while difficult or ambiguous tasks may require an LLM.

Overall, few systems bring these things together with cloud LLMs , on-device SLMs , hybrid routing approaches which pick the right model, structured organisational artifacts, multi role and compliance checks. This motivates the need for a combined approach that can reduce cost , privacy risk while improving efficiency, accuracy and governance across SDLC tasks

2. Literature Review

This review examines recent work in AI-assisted software engineering across five areas: LLM tools, on-device SLMs, retrieval methods, routing strategies, and compliance. It highlights what current research achieves and where important limitations remain for end-to-end SDLC support.

2.1 Review of Papers

Area 1: AI Systems for Software Engineering Tasks

Research in AI-assisted software engineering mostly targets narrow tasks such as code generation, bug fixing, or repository search. SWE-Agent demonstrates that an LLM can navigate files, run tests, and apply small edits to fix real issues (Yang et al., 2024).

SWE-Bench shows similar progress but also highlights that LLMs struggle with multi-file reasoning, dependency chains, and ambiguous test failures (Jiménez et al., 2023).

Models like CodeT5+ and StarCoder2 improve code understanding, and retrieval-based systems such as RepoCoder strengthen multi-file reasoning (Husain et al., 2019; Zhang et al., 2024) .However, these tools remain focused on isolated tasks.

Benchmarks like BigCodeBench reveal large performance differences across problem types, suggesting that a single model cannot handle all SDLC activities (BigCodeBench, 2023).

Iterative agent methods—ReAct, Reflexion, and Voyager—show that step-wise reasoning and learning from earlier attempts can improve outcomes (Shinn et al., 2023; Yang et al., 2023; Wang et al., 2023). Even so, these systems rely almost entirely on cloud LLMs and rarely address cost, privacy, or compliance concerns (Hou et al., 2024).

Area 2: Small Language Models and On-Device Reasoning

Small Language Models (SLMs) provide private and low-cost inference because they run locally and avoid cloud calls.Models such as Phi-2 (Microsoft Research, 2023), TinyLlama (Zhang et al., 2024), and MobileLLM (Liu et al., 2024) show that compact models can handle basic programming tasks when trained on carefully curated datasets.

Parameter-efficient tuning methods further increase their flexibility. LoRA adapters allow a single base model to load small task-specific modules, reducing training cost.Work such as AdapterFusion (Pfeiffer et al., 2021) demonstrates how multiple adapters can be combined, enabling organisations to personalise models without full retraining.

Area 3: Retrieval and Artifact-Based Methods

Software problems rely on scattered context across logs, tests, documentation, and code files.Retrieval systems help organise this information. CodeSearchNet shows that models can locate relevant functions using natural-language queries (Husain et al., 2019), and tools like RepoCoder improve multi-file reasoning by integrating retrieval into patch generation (Zhang et al., 2023).

General approaches such as RETRO (Borgeaud et al., 2022) and Memorizing Transformers (Wu et al., 2022) demonstrate how external memory can extend a model's reasoning beyond its internal context window.This idea fits with SDLC tasks, where past patches, failures, and test results form a valuable artifact history.

For a hybrid SDLC agent, structured retrieval is essential.It provides the SLM with focused and relevant context, improves accuracy on routine tasks, and reduces unnecessary escalations to a cloud LLM.

Area 4: Hybrid Routing and Model Selection

Hybrid routing decides whether a task should stay with the local SLM or move to a cloud LLM. RouteLLM shows that small classifiers or learned policies can estimate task difficulty and select an appropriate model (Ong et al., 2024).This approach keeps accuracy stable while reducing cost and unnecessary cloud usage.

Mixture-of-Experts systems apply a similar idea by activating only a subset of experts for each input (Lepikhin et al., 2021). Although MoE models are larger, the underlying principle of selective computation aligns well with hybrid SDLC workflows.

In practical SDLC tasks, routine steps—such as reading logs or checking a simple test failure—fit within an SLM's capability. More complex actions, including multi-file reasoning or dependency analysis, usually require an LLM.Hybrid routing enforces this division cleanly and improves overall efficiency.

Area 5: Compliance, Governance, and Safety in AI-Generated Code

Compliance is a critical but often neglected part of AI-assisted software development. Studies show that automatically generated patches can create licence conflicts or reuse protected content (Xu et al., 2024). Tools such as ScanCode Toolkit help detect licence and dependency issues (Ombredanne et al., 2020–), yet these checks are rarely included in AI development workflows.

Most AI systems focus on passing tests and ignore legal or policy requirements. They do not verify whether the generated code meets organisational standards or follows licence rules. This gap is significant for enterprise teams, where attribution, dependency safety, and code governance must be strictly maintained.

Area 6: Multi-Agent and Role-Based Systems

Role-based and multi-agent frameworks show how dividing work across specialised agents can improve structure and coordination. Systems such as CAMEL (Li et al., 2023), AgentVerse (Chen et al., 2023), and MetaGPT (Hong et al., 2023) assign roles like planner, coder, and tester to organise complex reasoning tasks. However, these frameworks rely almost entirely on large cloud models. They rarely include compliance checks and do not consider on-device limitations. No existing approach combines multi-role workflows with hybrid SLM–LLM routing, which is essential for realistic SDLC automation in enterprise settings.

2.2 Summary of Key Gaps

A review of the literature shows several gaps:

1. Existing systems support isolated tasks but not end-to-end SDLC roles.
2. No hybrid SLM–LLM system designed specifically for SDLC activities.
3. Project artifacts are not organised into reusable memory for ongoing tasks.
4. Compliance and licensing checks are largely missing in AI workflows.
5. Multi-agent or role-based designs are not integrated with hybrid inference.
6. SLMs lack mechanisms for gradual improvement using organisational data.
7. Evaluation is narrow and ignores cost, latency, and governance requirements.

2.3 How These Gaps Motivate This Research

Since existing research does not combine SDLC roles, hybrid inference, artifact reuse, and compliance, there is a clear need for a new approach. An on-device SLM can handle routine tasks, while a cloud LLM can support more complex reasoning. Retrieval and memory can help the SLM improve over time, and compliance checks ensure that generated patches are safe and acceptable. These goals guide the design of the proposed hybrid SDLC agent.

3. Research Questions

RQ1. How can a local SLM and a cloud LLM work together to improve efficiency across core SDLC tasks?

RQ2. How should project artifacts be organised so the agent can use them effectively without exposing private code?

RQ3. How can licence and dependency checks be integrated into the workflow to ensure safe and compliant patches?

RQ4. What routing strategy can decide when the SLM is enough and when the task should escalate to an LLM to control cost?

RQ5. How can SDLC tasks be split into simple agent roles that improve clarity and reduce reasoning load?

RQ6. How should the hybrid system be evaluated across correctness, latency, cost, and governance?

4. Aim and Objectives

4.1 Aim

The study will design and evaluate hybrid SDLC agents that improves efficiency, privacy, cost and governance in software development by intelligently combining a local SLM and a cloud LLM with artifact-based reasoning and compliance checks.

4.2 Objectives

Objective 1: Design a system with two models: a local SLM for routine SDLC tasks, and a cloud LLM for escalation when tasks are more complex.

Objective 2: Build a multi-role workflow that supports analysis, coding, testing, and compliance checks using structured project artifacts.

Objective 3: Evaluate the system on correctness, latency, cost efficiency, and compliance using tasks drawn from Defects4J and SWE-Bench Lite.

5. Significance of the Study

This work matters because current AI tools only handle small parts of the SDLC and depend heavily on cloud models, which brings cost, privacy, and compliance issues.

A hybrid setup that uses local SLMs for routine tasks and escalates complex work to an LLM can reduce cloud usage while keeping tasks efficient and predictable.

Bringing structured artifacts and licence checks into the workflow will improve the reliability and acceptability of generated patches, while giving teams a practical way to use AI more safely and efficiently.

The study also contributes academically by showing how hybrid routing and artifact-driven context can support everyday engineering tasks, with the system gradually improving through the reuse of past patches and test outcomes without retraining the underlying models.

6. Scope of the Study

In Scope

- Designing a simple hybrid setup where a local SLM handles routine SDLC tasks and a cloud LLM handles complex ones.
- Using structured artifacts such as logs, tests, and patches to guide the agent's decisions.
- Integrating basic licence and dependency checks to support safe and compliant patches.

- The study will evaluate the system on selected tasksfrom Defects4J and SWE-Bench Lite within the limits of a three-month project.

Out of Scope

- Full SDLC automation or coverage of large enterprise codebases.
- Training new LLMs or SLMs from scratch.
- Real-time deployment, IDE integration, or production-scale workflow automation.
- Deep compliance analysis beyond simple licence and dependency Thischecks.
- Handling long-running tasks, multi-repo systems, or large monorepos.

7. Research Methodology

The study will use a design-oriented approach where the aim is to build a small hybrid agent and test how it handles common SDLC tasks. The work will follow an experimental cycle: design the workflow, prepare artifacts, run tasks through the agent, and observe the results.

7.1 Data Sources and Artifact Preparation

The study will use a small set of public datasets that contain real bugs, tests, and code samples. Defects4J and SWE-Bench Lite will be used to provide practical tasks that allow the agent to carry out analysis, patching, and testing. CodeSearchNet will be used to add additional functions and snippets that support retrieval when the agent needs extra context, while BigCodeBench will be used to introduce tasks with varied difficulty levels to help assess the system's behaviour across different scenarios

To make these datasets usable within the workflow, the study will prepare a small collection of representative artefacts such as issue descriptions, failing tests, patches, and selected code fragments. Each artifact will be cleaned and stored in a structured format so that the agent can fetch only the information required for a specific task.

To support efficient retrieval, the study will index these artifacts using a lightweight vector store such as FAISS. Code embeddings generated through a model like SentenceTransformers or CodeBERT will be used to capture the semantic structure of functions, patches, and test outputs.

7.2 Hybrid System Workflow

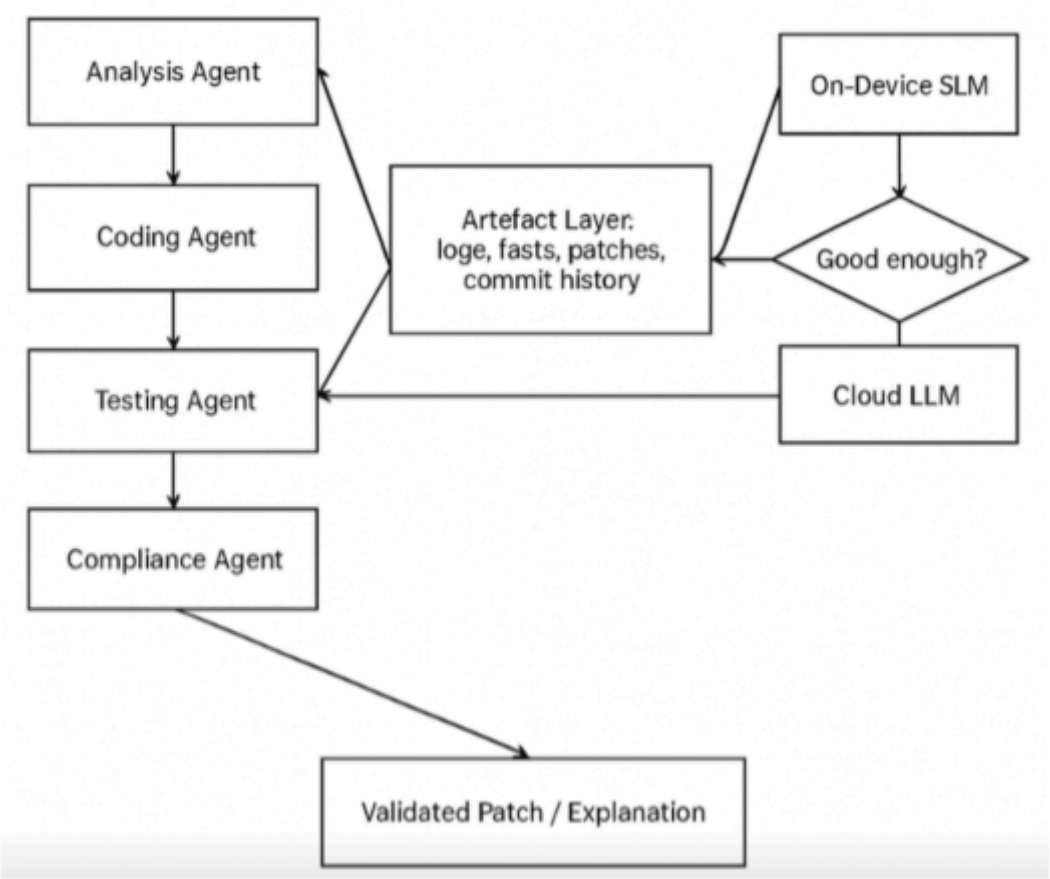

Fig 1. System Workflow

The figure uses a hybrid workflow where the agent moves through a few clear steps. For each task, the system gathers relevant artifacts, checks the problem, and tries to produce a fix or explanation.

The SLM will handle routine work such as reading logs or looking at small code snippets. If the task needs deeper reasoning, the workflow escalates it to a cloud LLM. The outcome is then tested, checked for basic compliance, and stored so the agent can reuse it later if a similar issue appears.

7.3 Roles Workflow

The workflow for each task will follow four simple roles. The analysis step checks the failing test, the error message, and the related artifacts to understand what might be wrong. The coding step tries a small fix or suggestion based on the available context. Testing runs the updated code against the provided tests to see if the fix works or introduces new failures. The compliance step performs a basic licence and dependency check to ensure the fix is safe to use.

7.4 Routing Strategy

The study will use a clear rule based strategy to decide when a task stays with the local SLM and when it moves to the cloud LLM. Routine work—such as reading logs, analysing a small code snippet, or handling a single-file failure—will remain on the SLM to avoid unnecessary cloud calls and keep code local.

More complex cases, such as multi-file issues, ambiguous test failures, or cross-module dependencies, will be escalated to the LLM, where a larger context window and stronger reasoning are helpful. This design is informed by prior work on model routing and selective computation, such as RouteLLM, but keeps the logic simple enough for a three-month project.

To make the behaviour predictable, the routing logic will define explicit criteria and capture them in a small decision rule or pseudocode. For example, tasks with a limited token footprint and a single relevant file will be handled by the SLM, while multi-file or unclear stack-trace scenarios will trigger escalation. The hybrid system will be evaluated against three baselines: an SLM-only pipeline, an LLM-only pipeline, and a no-AI baseline that relies only on tests and artefacts. Comparing correctness, latency, cost, and compliance across these setups will show whether the routing strategy offers a practical benefit over simpler alternatives.

The routing design will also include safeguards for edge cases. If an SLM attempt fails, times out, or produces an unclear analysis, the workflow will automatically fall back to the LLM. Any generated patch, whether produced by the SLM or LLM, will be checked by running the available tests before it is accepted, which helps to contain hallucinations and unsafe changes. Together, these rules and safeguards will keep the system efficient while giving the study a concrete way to analyse where hybrid routing adds value in everyday engineering tasks.

7.5 Compliance Integration

The workflow will include a basic compliance check to keep the generated patches safe to use. After the fix is produced, the system runs a simple licence and dependency scan to look for conflicts or missing attributions.

This step will help prevent issues that can appear when code is reused or combined from different sources. Keeping compliance inside the workflow supports the study's focus on governance and makes the evaluation more realistic for enterprise settings.

7.6 Evaluation

The study will evaluate the hybrid agent on a small set of tasks taken from Defects4J, SWE-Bench Lite, and BigCodeBench.

Each task will run through the workflow to check whether the fix works and how often the system needs to escalate to the LLM. The evaluation looks at correctness, latency, cost, and compliance to understand how well the hybrid setup balances accuracy and efficiency.

Correctness will be measured by the proportion of tasks where the generated patch passes the provided unit tests. Latency will be recorded as the average time taken per task from input to validated output. Cost will be estimated through approximate token or compute usage for both the SLM and LLM paths. Compliance will be assessed by checking whether ScanCode flags any licence or dependency issues in the patched files.

8. Requirements Resources

8.1. Hardware

- 16GB/32GB RAM is preferred for smoother local SLM inference.
- A modern CPU (Intel i5/i7 or Apple M1/M2 class) that can run small SLMs such as Phi-2 or TinyLlama in CPU mode.
- SSD storage with 40–60GB free space for datasets and experiment outputs.
- A GPU is not required, but an 8–12GB consumer GPU (optional) may help.

8.2. Software and Models

- Python 3.x environment for running scripts and automation steps.
- Standard libraries for data handling, test execution, and evaluation.
- A basic licence-scan tool such as ScanCode for compliance checks.
- Local SLM: a small open-source model (e.g., Phi-2, TinyLlama, or an equivalent ≤ 2B parameters). Quantised versions (4-bit/8-bit) may be used for lower memory use.

- Cloud LLM**:** an API-based model (e.g., GPT-4 class, Claude, or similar) for tasks requiring deeper reasoning. The risk of API Billing is limited.

8.3. Datasets

- Defects4J for real unit-test-driven bugs.
- SWE-Bench Lite for multi-file reasoning tasks.
- CodeSearchNet (subset) for additional snippets and context.
- BigCodeBench tasks for varied difficulty.
- Quantised SLM versions do not violate methodology or feasibility.

9. Research Plan

Phase 1 (Weeks 1–3): Review existing work on SLM–LLM hybrids and SDLC automation, refine the research questions, prepare datasets like Defects4J and CodeSearchNet, and set up the environment. By the end of this stage, the research questions, literature review, dataset slices, and initial architecture are in place.

Phase 2 (Weeks 4–7): Build the hybrid system: implement the SLM-first workflow, add routing logic, integrate the cloud LLM fallback, set up the artifact layer, add role-based prompts, and incorporate ScanCode for compliance. This phase delivers the first working version of the full pipeline.

Phase 3 (Weeks 8–10): Evaluate the system using tasks from Defects4J and SWE-Bench Lite. Measure correctness, latency, cost, and compliance, and compare SLM-only, LLM-only, and hybrid setups. Refine prompts and routing based on results.

Phase 4 (Weeks 11–12): Write up the results, finalise figures and diagrams, assemble the methodology section, and complete proofreading. This stage produces the final polished report ready for submission.

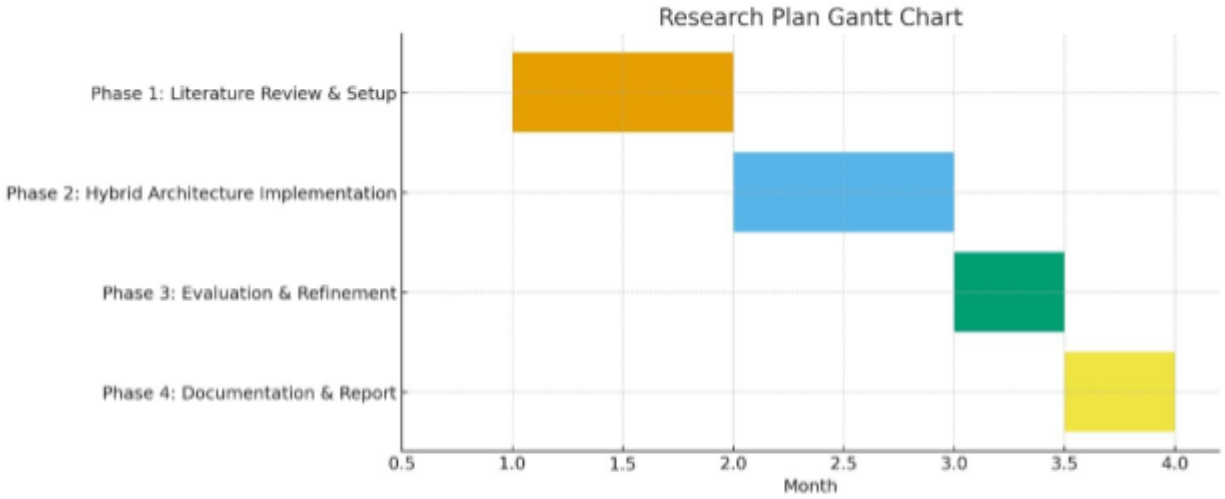

Fig 2. Gantt Chart for Research Plan

10. References

[1] Achiam, J., Adler, S., Agarwal, S. et al. (2024). GPT-4 Technical Report. arXiv preprint arXiv:2303.08774.
Available at: https://arxiv.org/abs/2303.08774
[Accessed: 3 December 2025]

[2] OpenAI. (2025). OpenAI API Platform Documentation.
Available at: https://platform.openai.com/docs
[Accessed: 3 December 2025]

[3] Zhuo, T. Y., Vu, T., Zhang, Y., Chen, M., Kocetkov, D., Li, R. et al. (2024). BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. arXiv preprint arXiv:2406.15877.
Available at: https://arxiv.org/abs/2406.15877
[Accessed: 3 December 2025]

[4] Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O. & Narasimhan, K. (2023). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv preprint arXiv:2310.06770.
Available at: https://arxiv.org/abs/2310.06770
[Accessed: 3 December 2025]

[5] Jimenez, C. E., Yang, J. & Geng, J. (2024). SWE-bench Lite. Project website.
Available at: https://www.swebench.com/lite.html
[Accessed: 3 December 2025]

[6] Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K. & Press, O. (2024). SWE-agent: Agent Computer Interfaces Enable Software Engineering Language Models. NeurIPS 2024 (to appear), preprint.
Available at: https://swe-agent.com
[Accessed: 3 December 2025]

[7] Husain, H., Wu, H., Gazit, T., Allamanis, M. & Brockschmidt, M. (2019). CodeSearchNet Challenge: Evaluating the State of Semantic Code Search. arXiv preprint arXiv:1909.09436.
Available at: https://arxiv.org/abs/1909.09436
[Accessed: 3 December 2025]

[8] Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M. et al. (2020). CodeBERT: A Pre-Trained Model for Programming and Natural Languages. Findings of EMNLP 2020, pp. 1536–1547.
Available at: https://aclanthology.org/2020.findings-emnlp.139/
[Accessed: 3 December 2025]
[9] Reimers, N. & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pp. 3982–3992.
Available at: https://arxiv.org/abs/1908.10084
[Accessed: 3 December 2025]
[10] Johnson, J., Douze, M. & Jégou, H. (2019). Billion-Scale Similarity Search with GPUs. IEEE Transactions on Big Data, 7(3), pp. 535–547. (FAISS: Facebook AI Similarity Search).
Available at: https://github.com/facebookresearch/faiss
[Accessed: 3 December 2025]
[11] Just, R., Jalali, D. & Ernst, M. D. (2014). Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs. Proceedings of the 2014 International Symposium on Software Testing and Analysis (ISSTA 2014), pp. 437–440.
Available at: https://doi.org/10.1145/2610384.2628055
[Accessed: 3 December 2025]
[12] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, L., Wang, W. & Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. arXiv preprint arXiv:2106.09685.
Available at: https://arxiv.org/abs/2106.09685
[Accessed: 3 December 2025]
[13] Xu, M., Zhang, Y., Zhang, H., Zhou, Y. & Liu, Y. (2024). Towards Automated Detection of License Conflicts in Software Projects. Proceedings of the 46th International Conference on Software Engineering (ICSE 2024).
Available at: https://doi.org/10.1145/3597503.3639100
[Accessed: 3 December 2025]
[14] Ombredanne, P. & nexB Inc. (2020). ScanCode Toolkit: Discover License, Copyright and Other Information in Code.
Available at: https://github.com/nexB/scancode-toolkit
[Accessed: 3 December 2025]

[15] BigCode. Lozhkov, A., Allal, L., Bia∫, M. et al. (2024). StarCoder2: A Next-Generation Open Code LLM. arXiv preprint arXiv:2402.19173.
Available at: https://arxiv.org/abs/2402.19173
[Accessed: 3 December 2025]
[16] Wang, Y., Le, H., Gotmare, A. D., Bui, N. D. Q., Li, J. & Hoi, S. C. H. (2023). CodeT5+: Open Code Large Language Models for Code Understanding and Generation. EMNLP 2023, pp. 5580–5600.
Available at: https://arxiv.org/abs/2305.07922
[Accessed: 3 December 2025]
[17] Zhang, F., Chen, Y., Guo, Z., Zhang, G., Zhu, Q. & Zhang, N. (2023). RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. Proceedings of EMNLP 2023, pp. 2504–2519.
Available at: https://arxiv.org/abs/2303.12570
[Accessed: 3 December 2025]
[18] Borgeaud, S., Mensch, A., Hoffmann, J. et al. (2022). Improving Language Models by Retrieving from Trillions of Tokens. Proceedings of the 39th International Conference on Machine Learning (ICML 2022). (RETRO).
Available at: https://arxiv.org/abs/2112.04426
[Accessed: 3 December 2025]
[19] Wu, Y., Izacard, G., Grave, E., Joulin, A. & Lample, G. (2022). Memorizing Transformers. International Conference on Learning Representations (ICLR 2022).
Available at: https://arxiv.org/abs/2203.08913
[Accessed: 3 December 2025]
[20] Microsoft. (2023). Phi-2: A Small Language Model for Reasoning. Model card / technical report.
Available at: https://huggingface.co/microsoft/phi-2
[Accessed: 3 December 2025]
[21] Zhang, Q., Ye, Y., Chen, B., Fan, X., Liu, J., Sun, J. et al. (2024). TinyLlama: An Open-Source Small Language Model. Project documentation.
Available at: https://github.com/jzhang38/TinyLlama
[Accessed: 3 December 2025]
[22] Lin, J., Jiang, A., Xiong, Y. et al. (2024). Qwen1.5-MoE: Diverse Mixture-of-Experts Models for Multilingual Language Understanding. arXiv preprint arXiv:2404.01744.
Available at: https://arxiv.org/abs/2404.01744

[Accessed: 3 December 2025]
[23] Kaddour, J., Leech, G., van der Schaar, M. & Pleiss, G. (2023). RouteLLM: Learning to Route to Many Large Language Models. arXiv preprint arXiv:2310.03094.
Available at: https://arxiv.org/abs/2310.03094
[Accessed: 3 December 2025]
[24] Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K. & Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. Advances in Neural Information Processing Systems (NeurIPS 2023).
Available at: https://openreview.net/forum?id=vAElhFcKW6
[Accessed: 3 December 2025]
[25] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Weischedel, R., Zhou, D. & Narasimhan, K. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. International Conference on Learning Representations (ICLR 2023).
Available at: https://react-lm.github.io/
[Accessed: 3 December 2025]
[26] Wang, G., Xie, E., Chen, Y., Li, Z., Hou, Q., Li, J. & Dai, J. (2023). Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv preprint arXiv:2305.16291.
Available at: https://voyager.minedojo.org/
[Accessed: 3 December 2025]
[27] Li, G., Hammoud, H. A. K., Itani, H., Khizbullin, D. & Ghanem, B. (2023). CAMEL: Communicative Agents for “Mind” Exploration of Large Language Model Society. International Conference on Learning Representations (ICLR 2024).
Available at: https://arxiv.org/abs/2303.17760
[Accessed: 3 December 2025]
[28] Chen, W., Su, Y., Zuo, J., Yang, C., Yuan, C., Chan, C.-M. et al. (2024). AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors. International Conference on Learning Representations (ICLR 2024).
Available at: https://arxiv.org/abs/2308.10848
[Accessed: 3 December 2025]